

# FROST Guide

Threshold RedPallas: FROST and its Zcash integration surfaces

m@rek.onl

## Abstract

This volume of *The Zcash Arboretum* constructs threshold spend authorisation for Orchard-protocol notes: FROST, the two-round threshold Schnorr protocol of RFC 9591, with key generation by a trusted dealer and by a distributed protocol, and its re-randomised variant specified in ZIP 312 over the ciphersuite FROST(Pallas, BLAKE2b-512), whose aggregate is an ordinary RedPallas spend-authorisation signature. Any threshold number of participants signs, and no signing run reconstructs the signing key. The volume proves correctness of each protocol and states its security with the assumptions and the status of each result, marking open problems as such. It assumes the *Math Guide* and the *Crypto Guide*, and cites the *Ironwood Guide* for RedPallas, randomised validating keys, the signature digest and the construction of a transaction. It is non-normative: the protocol specification, the ZIPs and RFC 9591 are authoritative.

# Contents

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                 | <b>3</b>  |
| 1.1      | Scope . . . . .                                                     | 3         |
| 1.2      | Sources and their standing . . . . .                                | 4         |
| 1.3      | Notation . . . . .                                                  | 5         |
| <b>2</b> | <b>Threshold Schnorr signatures</b>                                 | <b>6</b>  |
| 2.1      | Participants, identifiers and signing sets . . . . .                | 6         |
| 2.2      | Key material and group information . . . . .                        | 8         |
| 2.3      | Two-round threshold signature schemes . . . . .                     | 9         |
| 2.4      | Channels and corruption . . . . .                                   | 10        |
| <b>3</b> | <b>Key generation</b>                                               | <b>11</b> |
| 3.1      | Trusted-dealer key generation . . . . .                             | 11        |
| 3.2      | Distributed key generation . . . . .                                | 14        |
| <b>4</b> | <b>Signing and aggregation</b>                                      | <b>16</b> |
| 4.1      | Ciphersuites and encodings . . . . .                                | 17        |
| 4.2      | Round One: nonces and commitments . . . . .                         | 18        |
| 4.3      | Round Two: binding factors and signature shares . . . . .           | 20        |
| 4.4      | Aggregation and share verification . . . . .                        | 22        |
| 4.5      | Correctness of FROST signing . . . . .                              | 23        |
| 4.6      | Single use of nonces . . . . .                                      | 24        |
| <b>5</b> | <b>Re-randomised FROST for spend authorisation</b>                  | <b>26</b> |
| 5.1      | Spend authorisation under randomised validating keys . . . . .      | 26        |
| 5.2      | Re-randomisable threshold signatures . . . . .                      | 28        |
| 5.3      | Randomiser generation . . . . .                                     | 29        |
| 5.4      | Authorisation of the signed message . . . . .                       | 32        |
| 5.5      | Re-randomised signing, aggregation and share verification . . . . . | 32        |
| 5.6      | Trust model of ZIP 312 . . . . .                                    | 34        |
| <b>6</b> | <b>The aggregate as a spend-authorisation signature</b>             | <b>35</b> |
| 6.1      | The ciphersuite FROST(Pallas, BLAKE2b-512) . . . . .                | 35        |
| 6.2      | Validity of the aggregate . . . . .                                 | 37        |
| 6.3      | Sign normalisation of a shared key . . . . .                        | 38        |
| 6.4      | The key tree of an Orchard-protocol FROST key . . . . .             | 39        |
| 6.5      | Order of randomiser, validating key and signature digest . . . . .  | 41        |
| 6.6      | Threshold signing of an Action . . . . .                            | 43        |
| <b>7</b> | <b>Security</b>                                                     | <b>44</b> |
| 7.1      | One-more discrete logarithms . . . . .                              | 44        |
| 7.2      | Security of FROST signing . . . . .                                 | 45        |
| 7.3      | Unforgeability and key generation . . . . .                         | 49        |
| 7.4      | Unforgeability of re-randomised FROST . . . . .                     | 51        |
| 7.5      | Threshold spend authority . . . . .                                 | 52        |
| 7.6      | Unlinkability . . . . .                                             | 55        |
| 7.7      | Status of the constructions . . . . .                               | 57        |

# 1 Introduction

This section fixes the object of the volume and its scope, the standing of the normative and scholarly sources with the classes into which its claims fall, and the notation shared with the lower volumes.

## 1.1 Scope

This volume of *The Zcash Arboretum* constructs threshold spend authorisation for Orchard-protocol notes, for Actions of either pool of the Orchard protocol (*Ironwood Guide*, §“The Orchard protocol and its two pools”, Definitions “Orchard protocol” and “Orchard pool, Ironwood pool”). Any threshold number of the participants, the threshold and the number of participants being those of Definition 2.1, jointly produce, for an Action spending a note of a threshold key, a signature that consensus accepts as an ordinary RedPallas spend-authorisation signature, with the same format and the same validation as a signature by a single holder of the spend authorising key ask. No signing run reconstructs the signing key; a trusted dealer computes it at key generation; no step of distributed key generation computes it, and no source cited in this volume proves that protocol secure (Remark 3.9). RedPallas and spend authorisation are cited, not re-derived: *Ironwood Guide*, §“The RedPallas signature scheme” and §“Randomised validating keys”.

The construction has two layers. The first is FROST, the two-round threshold Schnorr protocol of Komlo and Goldberg as RFC 9591 specifies it, run on a Shamir sharing of ask. The second is its re-randomised variant (ePrint 2024/436), specified for Zcash in ZIP 312, which shifts the shares, the verification shares and the group key of each signing run by one randomiser so that the aggregate verifies under the Action’s randomised validating key; it is defined in §5.2 and constructed in §5.5 (Construction 5.10) with the randomiser of §5.3, and its correctness is Theorem 5.11.

The Orchard pool is spend-only: no transaction increases its value, and every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 258, Draft, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“The Orchard protocol and its two pools”), so no value enters the pool and, by consensus, only a party able to authorise spends for an address can create an Orchard-pool note to it. ZIP 326 (Draft) forbids conforming wallets to send Orchard-pool funds to a key derived with `use_qsk = true` (ZIP 2005), which every key from distributed key generation must use, so under that rule such a key holds no Orchard-pool notes, a rule that consensus does not enforce, while a dealer-shared key whose ask is derived from a spending key may spend notes of either pool; both facts are established, with their sources, in §6.4.

The volume constructs FROST as RFC 9591 specifies it; key generation by a trusted dealer and by a distributed protocol; the re-randomised variant for RedPallas that ZIP 312 specifies, with its ciphersuite FROST(Pallas, BLAKE2b-512); the proof that its output is an ordinary spend-authorisation signature; the key-generation constraints of ZIP 2005; and the security of each part, with its assumptions and status. ZIP 312 also defines the ciphersuite FROST(Jubjub, BLAKE2b-512) for Sapling spend authorisation by RedJubjub; it is named here and not developed. Transparent inputs, transports and particular tools are outside the scope. The volume assumes the *Math Guide* and the *Crypto Guide*, and cites the *Ironwood Guide* for RedPallas, randomised validating keys, the signature digest and the construction of a transaction. It is non-normative: the protocol specification, the ZIPs and RFC 9591 are authoritative.

The volume describes the protocol as specified for NU7 (*Ironwood Guide*, §“Scope and status”, Definition “Network upgrade”), whose deployment ZIP 259 has status Draft; NU7 changes neither spend authorisation nor the transaction formats on which the volume relies, version-5 and version-6 transactions remaining valid (ZIP 259, Abstract). NU6.3 is active, and its rules, including those of ZIP 2005 that ZIP 258 deploys, are part of the protocol so described.

## 1.2 Sources and their standing

The *Ironwood Guide*, §“Scope and status”, Definition “ZIP”, names the ZIP statuses Draft and Reserved. A ZIP has status *Proposed* when, after Draft, it has been advanced upon consideration, feedback and rough consensus of the community (ZIP 0, “ZIP Status Field”). A ZIP of status Draft, Proposed, Active, Implemented or Final carries normative text; a Reserved, Withdrawn, Rejected or Obsolete ZIP carries none.

### Definition 1.1 (Classes of claims).

- (i) A claim about the deployed protocol is placed in one of the two classes of the *Ironwood Guide*, §“Scope and status”, Definition “Epistemic classes”, with their meaning unchanged. It is *specified* when normative text of the protocol specification or of a ZIP fixes it. It is *designed but unspecified* when the deployed construction is designed to satisfy it but neither the protocol specification, nor a ZIP, nor a published proof establishes it for the deployed instance; knowledge soundness of the Action proof is that volume’s example.
- (ii) *Normative text* comprises the protocol specification, RFC 9591, an Informational RFC of the IRTF stream, and every ZIP of status Draft, Proposed, Active, Implemented or Final.
- (iii) A claim about a construction outside the deployed protocol, namely FROST, its key generation and its re-randomised variant, to which no consensus rule refers, is *specified* when normative text fixes it, *established* when a published proof establishes it, stated with that proof’s assumptions and the venue status of its source (§1.2), *proved here* when this volume proves it, under the hypotheses it states, and an *open problem* when neither normative text, nor a published proof, nor this volume establishes it.
- (iv) A construction itself is *specified* when normative text fixes it; *designed but unspecified* when a published design, or this volume, fixes it and no normative text does, any part first stated here, such as Construction 6.7, being marked as the volume’s own; and an *open problem* when nothing fixes it.

Every design-stage claim carries its class where it is stated; Table 3 collects them.

The sources of the volume, each with its standing, are the following.

- (a) D. Connolly, C. Komlo, I. Goldberg and C. A. Wood, “The Flexible Round-Optimized Schnorr Threshold (FROST) Protocol for Two-Round Schnorr Signatures”, RFC 9591, June 2024, a product of the IRTF Crypto Forum Research Group in the category Informational, not an Internet Standards Track specification, specifies two-round FROST signing over any prime-order group and hash, with ciphersuites for none of the Zcash groups, leaves key generation out of scope, gives a trusted dealer for completeness in its Appendix C, and is normative text in the sense of Definition 1.1.
- (b) ZIP 312, “FROST for Spend Authorization Multisignatures”, status Draft, category Wallet, specifies the re-randomised variant for Zcash as changes to RFC 9591 (randomiser generation, Round Two, and re-randomised signing, aggregation and share verification) together with the ciphersuites FROST(Jubjub, BLAKE2b-512) and FROST(Pallas, BLAKE2b-512), names ask as the key to be shared, requires key generation to be consistent with FROST without specifying one, and specifies no consensus rule.
- (c) ZIP 2005, “Ironwood Quantum Recoverability”, status Proposed, category Consensus, whose key rules take effect with NU6.3, which ZIP 258 (Draft) deploys, and stand in the protocol as specified for NU7, is used for its constraints on FROST key generation and the

keys they produce: “Usage with FROST”, its replacement text for §4.2.3 “Orchard Key Components” (“Changes to the Protocol Specification”), and “Deployment”.

- (d) C. P. L. Gouvêa and C. Komlo, “Re-Randomized FROST”, IACR ePrint 2024/436, an unrefereed preprint with no peer-reviewed venue, defines re-randomisable threshold signatures and their unforgeability game and claims unforgeability of Rerandomized FROST in the random-oracle model under the algebraic one-more discrete logarithm assumption, with a proof that §7.4 finds incomplete (Remark 7.17), so that the claim is an open problem (Table 3); a result that rests on it alone is stated as the preprint’s, not as established.
- (e) C. Komlo and I. Goldberg, “FROST: Flexible Round-Optimized Schnorr Threshold Signatures”, IACR ePrint 2020/852, published at SAC 2020 (LNCS 12804, pp. 34–65) and cited by RFC 9591 as [FROST20], gives the original scheme, its distributed key generation, in which every participant acts as a Feldman dealer and proves knowledge of its constant coefficient by a Schnorr proof (its Figure 1), and the original security argument.
- (f) M. Bellare, S. Tessaro and C. Zhu, “Stronger Security for Non-Interactive Threshold Signatures: BLS and FROST”, IACR ePrint 2022/833, cited in its current revision, has a preliminary version that is part of M. Bellare, E. Crites, C. Komlo, M. Maller, S. Tessaro and C. Zhu, “Better than Advertised Security for Non-Interactive Threshold Signatures”, CRYPTO 2022, which RFC 9591 cites as [StrongerSec22]; each work is cited by its own title and authors.
- (g) E. Crites, C. Komlo and M. Maller, “How to Prove Schnorr Assuming Schnorr: Security of Multi- and Threshold Signatures”, IACR ePrint 2021/1375, which defines FROST2 and analyses it with its own key generation, is cited only for FROST2 in §7.2 and for the nearest result recorded in Remark 7.13.
- (h) The revision of ZIP 312 proposed in zcash/zips pull request 895, linked from ZIP 2005, “Deployment”, is unmerged and not normative text, and is cited only as a published design.

### 1.3 Notation

Notation is additive throughout; the multiplicative forms of the papers are converted at citation. The generic sections work in a group  $\mathbb{G}$  of prime order  $q$  with generator  $B$  and identity  $O$ , whose discrete logarithm problem and random oracles are those of the *Crypto Guide*, §“The discrete logarithm problem”, Definition “Discrete logarithm problem, DLP”, and §“The random oracle model”. The Pallas instance sets

$$\mathbb{G} = \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}), \quad q = p_{\text{Vesta}}, \quad B = G^{\text{SpendAuth}}$$

(*Ironwood Guide*, §“Fields, groups, and encodings”, and §“The spending key and the spend-side secrets”, Construction “Spend validating key and sign normalisation”). Scalar multiplication is written  $[k]P$ , and scalars are elements of  $\mathbb{F}_q$ . The star encoding  $P^*$ , the encodings  $\text{LE}_k$  and  $\text{I2LEOSP}_k$ , concatenation  $\parallel$  and monospace byte-string literals are those of the *Ironwood Guide*, §“Fields, groups, and encodings”. The element and scalar serialisations  $\text{ser}_{\mathbb{G}}$ ,  $\text{ser}_{\mathbb{F}_q}$  and their inverses  $\text{deser}_{\mathbb{G}}$ ,  $\text{deser}_{\mathbb{F}_q}$  are components of a ciphersuite (Definition 4.1).

The spend-side symbols follow the *Ironwood Guide*, §“The spending key and the spend-side secrets”:  $\text{ask}$  is the spend authorising key,  $\text{ak}^{\mathbb{P}} = [\text{ask}] G^{\text{SpendAuth}}$  the spend validating key as a point,  $\text{ak} = \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$  its extracted coordinate, and  $\text{nk}$ ,  $\text{rsk}$  and  $\text{rk}$  are as there. ZIP 2005 uses the same notation.

A signing set is written  $I$  and its Lagrange coefficients  $\lambda_{I,i}$  (Definition 2.3), the symbols of the *Crypto Guide*, §“Secret sharing”, in Construction “Shamir  $t$ -of- $n$  secret sharing” and in Remark “The leak, and where it is harmless”, which names this volume as the consumer of  $\lambda_{I,i}$ -weighted recombination. The generator  $G$  of that Guide’s Construction “Feldman verifiable secret sharing”

| Object                            | This volume                                                                                                                                                                          | RFC 9591                                             | ZIP 312                                              | Papers                                              |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------------|-----------------------------------------------------|
| Group order                       | $q$                                                                                                                                                                                  | $p, \text{G.Order}()$                                | $\text{G.Order}(); r_{\mathbb{P}}$ for Pallas        | $q$ (2020/852); $p$ (2022/833, 2024/436)            |
| Generator                         | $B; G^{\text{SpendAuth}}$ for Pallas                                                                                                                                                 | $B$                                                  | $G^{\text{Orchard}}$                                 | $g$                                                 |
| Scalar base multiplication        | $[k]B$                                                                                                                                                                               | $\text{ScalarBaseMult}(k)$                           | $\text{G.ScalarBaseMult}(k)$                         | $g^k$                                               |
| Scalar multiplication             | $[k]A$                                                                                                                                                                               | $\text{ScalarMult}(A, k)$                            | as RFC 9591                                          | $A^k$                                               |
| Element encoding                  | $\text{ser}_{\mathbb{G}}, \text{deser}_{\mathbb{G}}$ ; on Pallas LEBS2OSP <sub>256</sub> ( $P^*$ ) and $\text{abst}_{\mathbb{P}}$ , failing on $\perp$ and on $O$ (Construction 6.1) | $\text{SerializeElement}, \text{DeserializeElement}$ | $\text{repr}_{\mathbb{P}}, \text{abst}_{\mathbb{P}}$ | —                                                   |
| Scalar encoding                   | $\text{ser}_{\mathbb{F}_q}, \text{deser}_{\mathbb{F}_q}$ ; on Pallas l2LEOSP <sub>256</sub>                                                                                          | $\text{SerializeScalar}, \text{DeserializeScalar}$   | 32-byte little-endian                                | —                                                   |
| Threshold, number of participants | $t, n$                                                                                                                                                                               | $\text{MIN\_PARTICIPANTS}, \text{MAX\_PARTICIPANTS}$ | as RFC 9591                                          | $t, n$                                              |
| Randomiser                        | effective randomiser $\hat{\alpha}$ , the scalar that shifts the key material; in an accepted Action the Action's randomiser $\alpha$ (Corollary 6.4)                                | —                                                    | randomizer                                           | $\hat{\alpha} = H_r(\alpha, \text{aux})$ (2024/436) |
| Hash roles                        | $H_1, \dots, H_5, H_R$                                                                                                                                                               | $H1, \dots, H5$                                      | $H1, \dots, H5, HR$                                  | —                                                   |

Table 1: Notation of the sources. Each row names an object in this volume, in RFC 9591, in ZIP 312 and in the multiplicative notation of the papers (ePrints 2020/852, 2022/833 and 2024/436). The names  $r_{\mathbb{P}}, \text{repr}_{\mathbb{P}}, \text{abst}_{\mathbb{P}}$  and  $G^{\text{Orchard}}$  are those of the protocol specification, which ZIP 312 uses, and which §6 also writes where it follows the protocol specification.

is  $B$  here. The group key and verification shares are  $PK$  and  $PK_i$ , the secret shares  $sk_i$  (Definition 2.5). Query counts of cited bounds are written  $Q$ , or  $q_s$  and  $q_h$  with subscripts, never a bare  $q$ , which is the group order. The British spelling *randomiser* replaces ZIP 312's *randomizer*; titles of ZIPs and papers keep their own spelling. Table 1 maps the names of the sources to those of the volume; later text uses the latter, and §6 also the protocol specification's names where it follows that text.

## 2 Threshold Schnorr signatures

This section fixes the setting that every later protocol assumes: the participants, their identifiers, the signing sets and their Lagrange coefficients; the threshold key material and the group information that coefficient commitments determine; the syntax and correctness of a two-round threshold Schnorr signature scheme with a Coordinator; and the channel types and the static corruption model. Notation is that of §1.3.

### 2.1 Participants, identifiers and signing sets

**Definition 2.1** (Threshold parameters and signing sets). Let  $\mathbb{G}$  be a group of prime order  $q$  with generator  $B$  and scalar field  $\mathbb{F}_q$  (§1.3). *Threshold parameters* are integers  $t, n$  with

$$1 \leq t \leq n < q.$$

There are  $n$  participants  $P_1, \dots, P_n$ . The *identifier* of  $P_i$  is  $i \in \{1, \dots, n\}$ , read as an element of  $\mathbb{F}_q$ ;

identifiers are distinct and nonzero in  $\mathbb{F}_q$  because  $n < q$ . A *signing set* is a set  $I \subseteq \{1, \dots, n\}$  with  $t \leq |I| \leq n$ . In the notation of RFC 9591 (Table 1),  $t$  is MIN\_PARTICIPANTS,  $n$  is MAX\_PARTICIPANTS and  $|I|$  is NUM\_PARTICIPANTS.

The constraints are those of RFC 9591, Section 5 (“Two-Round FROST Signing Protocol”): each identifier is a nonzero scalar in  $[1, \text{MAX\_PARTICIPANTS}]$ , distinct from every other; MAX\_PARTICIPANTS is less than the group order; and MIN\_PARTICIPANTS  $\leq$  NUM\_PARTICIPANTS  $\leq$  MAX\_PARTICIPANTS. The dealer of RFC 9591, Appendix C (“Trusted Dealer Key Generation”), shares at the identifiers  $1, \dots, n$ . Every statement below that uses only interpolation holds verbatim for any distinct nonzero identifiers; the identifiers  $1, \dots, n$  are the ones RFC 9591 requires.

**Remark 2.2** (Shamir sharing). Shamir sharing is cited, not re-derived: *Crypto Guide*, §“Secret sharing”, Construction “Shamir  $t$ -of- $n$  secret sharing”. Its parameters  $1 \leq t \leq n < q$  and identifiers  $1, \dots, n$  are those of Definition 2.1; its share symbol  $s_i$  is this volume’s  $sk_i$ , and the generator  $G$  of the *Crypto Guide*’s Construction “Feldman verifiable secret sharing” is  $B$ . Any  $t$  shares determine the sharing polynomial and its constant term (Lemma 2.4 gives this for every signing set). Fewer than  $t$  shares are jointly uniform whatever the secret (*Crypto Guide*, Theorem “Perfect secrecy below the threshold”). That theorem concerns the shares alone. For key material shared by a dealer, once the coefficient commitments of §2.2 are published, the point  $[s]B$  is public and secrecy below the threshold is computational: the view of  $t - 1$  participants is computable from  $[s]B$  and  $t - 1$  uniform scalars. Since  $[s]B$  is the group key, which is published in any case, nothing further escapes (*Crypto Guide*, Remark “The leak, and where it is harmless”). Distributed key generation is treated in §7.3.

**Definition 2.3** (Lagrange coefficients of a signing set). Let  $I$  be a signing set and  $i \in I$ . The *Lagrange coefficient* of  $i$  in  $I$  is

$$\lambda_{I,i} := \prod_{j \in I, j \neq i} \frac{j}{j - i} \in \mathbb{F}_q,$$

whose denominators are nonzero because identifiers are distinct. For the Lagrange basis polynomial  $\ell_i(X) = \prod_{j \in I, j \neq i} (X - j)/(i - j)$  of the nodes  $I$  (*Math Guide*, Theorem “Lagrange interpolation”),  $\lambda_{I,i} = \ell_i(0)$ , since  $(0 - j)/(i - j) = j/(j - i)$ ; it is the reconstruction weight of the *Crypto Guide*’s Construction “Shamir  $t$ -of- $n$  secret sharing”. The specified computation takes the list of identifiers of  $I$  and the identifier  $i$ , and fails with the error “invalid parameters” if  $i$  is absent from the list or some identifier occurs in it more than once (RFC 9591, Section 4.2, “Polynomials”).

**Lemma 2.4** (Recombination over a signing set). Let  $I$  be a signing set. For every  $f \in \mathbb{F}_q[X]$  with  $\deg f \leq t - 1$ ,

$$\sum_{i \in I} \lambda_{I,i} f(i) = f(0).$$

In particular  $\sum_{i \in I} \lambda_{I,i} = 1$ . The statement and its proof hold verbatim for any finite set  $I$  of distinct nonzero elements of  $\mathbb{F}_q$  with  $|I| \geq \deg f + 1$ , the coefficients being defined by the same product.

*Proof.* Put  $d := |I| - 1$ , so  $d \geq t - 1 \geq \deg f$ . By *Math Guide*, Theorem “Lagrange interpolation”, with nodes  $I$  and targets  $f(i)$ , the polynomial  $g := \sum_{i \in I} f(i) \ell_i$  has degree at most  $d$  and satisfies  $g(i) = f(i)$  for every  $i \in I$ , that is at  $d + 1$  distinct points. The polynomial  $f$  also has degree at most  $d$ , so  $g = f$  by *Math Guide*, Corollary “Few points determine a low-degree polynomial”. Evaluating at 0 and using  $\ell_i(0) = \lambda_{I,i}$  (Definition 2.3) gives the identity. The constant polynomial  $f = 1$  has degree  $0 \leq t - 1$ , because  $t \geq 1$ , and gives the sum one.  $\square$

The proof is given in place: the *Math Guide*’s Proposition “Lagrange polynomials sum to one” concerns its roots-of-unity domain and does not apply to the nodes  $I$ . ZIP 312, “Rationale”, uses the sum  $\sum_{i \in I} \lambda_{I,i} = 1$  and refers to an external note for its proof; ePrint 2024/436, Section 5, justifies it in one line as the interpolation of the constant polynomial 1.

The coefficients  $\lambda_{I,i}$  depend on the identifiers of  $I$  alone, not on the shares or on the polynomial (Definition 2.3). Hence the signing set is chosen per signing run, after key generation, by the Coordinator of Definition 2.8, which determines the participants, at least MIN\_PARTICIPANTS in number (RFC 9591, Section 5).

## 2.2 Key material and group information

**Definition 2.5** (Threshold key material). Let  $(t, n)$  be threshold parameters. *Threshold key material* consists of:

1. a *sharing polynomial*  $f(X) = \sum_{k=0}^{t-1} a_k X^k \in \mathbb{F}_q[X]$  with  $f(0) = a_0 = s$ , the *signing key*;
2. for each identifier  $j$ , the *secret share*  $(j, sk_j)$  with  $sk_j = f(j)$ , held by  $P_j$  alone;
3. the *coefficient commitments*  $A_k := [a_k]B$  for  $k = 0, \dots, t-1$ ;
4. the *group information*  $(PK, (PK_j)_{j=1}^n)$ , with *group key*  $PK := [s]B$  and *verification shares*  $PK_j := [sk_j]B$ , held by every party, the Coordinator included.

Shares  $(sk_j)_{j=1}^n \in \mathbb{F}_q^n$  and group information  $(PK, (PK_j)_{j=1}^n) \in \mathbb{G}^{n+1}$  are *valid key material* if some  $f \in \mathbb{F}_q[X]$  with  $\deg f \leq t-1$  has  $f(j) = sk_j$  for every  $j \in \{1, \dots, n\}$ ,  $PK = [f(0)]B$  and  $PK_j = [sk_j]B$  for every  $j$ . Such  $f$  is unique, since  $n \geq t$  (*Math Guide*, Corollary “Few points determine a low-degree polynomial”). No party is given  $s$  or  $f$ ; the definition names them. Valid key material is the output of both key generations (§3) and the input of signing.

The components are those of RFC 9591, Section 5: each participant is configured with its identifier, the share  $sk_i = f(i)$  and  $PK_i = [sk_i]B$ ; the group information consists of  $PK$  and  $PK_i$  for each  $i$  in  $[1, \text{MAX\_PARTICIPANTS}]$ .

**Lemma 2.6** (Group information from coefficient commitments). Let  $f = \sum_{k=0}^{t-1} a_k X^k$  and  $A_k = [a_k]B$ .

(i) The group key is  $PK = [f(0)]B = A_0$ , and for every identifier  $j$

$$PK_j = [f(j)]B = \sum_{k=0}^{t-1} [j^k]A_k.$$

Every party holding  $(A_0, \dots, A_{t-1})$  therefore computes the group information from the commitments alone.

(ii) Let  $(A_0, \dots, A_{t-1}) \in \mathbb{G}^t$ , however chosen, and let  $sk_1, \dots, sk_n \in \mathbb{F}_q$  satisfy  $[sk_j]B = \sum_{k=0}^{t-1} [j^k]A_k$  for every  $j \in \{1, \dots, n\}$ . Then  $(sk_j)_{j=1}^n$ , with the group information computed by (i), is *valid key material* (Definition 2.5) for the unique polynomial  $f^* = \sum_k a_k^* X^k$  with  $[a_k^*]B = A_k$ .

*Proof.* Part (i) is the identity displayed in *Crypto Guide*, Construction “Feldman verifiable secret sharing”, where it follows from the homomorphism  $a \mapsto [a]B$ ; that construction’s  $G$  and  $\Phi_k$  are this volume’s  $B$  and  $A_k$ . For (ii), by *Crypto Guide*, Proposition “What the Feldman check

certifies”, the vector  $(A_0, \dots, A_{t-1})$  determines a unique polynomial  $f^*$  of degree at most  $t - 1$ , and each passing check gives  $sk_j = f^*(j)$ . Then  $A_0 = [f^*(0)]B$  and  $\sum_k [j^k]A_k = [sk_j]B$ , which are the conditions of Definition 2.5.  $\square$

RFC 9591, Appendix C.2 (“Verifiable Secret Sharing”), derives the group information from the commitment vector in this way. Both key generations of §3 cite the lemma.

### 2.3 Two-round threshold signature schemes

**Remark 2.7** (Prerequisites). The syntax of a signature scheme, existential unforgeability under chosen-message attack and strong unforgeability are those of *Crypto Guide*, §“Syntax and security goal” (Definitions “Digital signature scheme”, “Existential unforgeability under chosen-message attack” and “Strong unforgeability”), which admits correctness with overwhelming probability. The single-party Schnorr signature is *Crypto Guide*, §“From identification to signature via the Fiat–Shamir transform”, Construction “Schnorr signature”, perfectly correct by its Proposition “Perfect correctness”. In additive notation with generator  $B$ , signing key  $s$  and public key  $PK = [s]B$ , a signature on a message  $m$  is

$$r \xleftarrow{\$} \mathbb{F}_q, \quad R = [r]B, \quad c = H(R \parallel PK \parallel m), \quad z = r + cs, \quad \sigma = (R, z),$$

and verification accepts  $(R, z)$  if and only if

$$[z]B = R + [c]PK.$$

The *Crypto Guide*’s response  $s$  is written  $z$  here,  $s$  being the signing key. The challenge input order (nonce commitment, public key, message) is that of RFC 9591, Section 4.6 (“Signature Challenge Computation”), and of RedPallas; the hash and the encodings of FROST are fixed by the ciphersuite (Definition 4.1). Discrete logarithms and random oracles are those of the *Crypto Guide*, §“The discrete logarithm problem” and §“The random oracle model”, as §1.3 records.

**Definition 2.8** (Two-round threshold signature scheme with a Coordinator). Let  $(t, n)$  be threshold parameters (Definition 2.1). A *two-round threshold signature scheme with a Coordinator* is run by the participants  $P_1, \dots, P_n$  and one *Coordinator*: a role, which a participant may also play, that uses no share of the signing key and no nonce, and that chooses the signing set, relays messages, aggregates and publishes the signature. It consists of:

1. *Key generation*, a dealer algorithm or a protocol, which outputs valid key material (Definition 2.5):  $sk_j$  to  $P_j$ , and the group information to every party.
2. *Round One*, run by each  $P_i$  independently of any message and of any signing set, which outputs a public *commitment*  $com_i$ , sent to the Coordinator, and a secret *state*  $st_i$ , which  $P_i$  keeps and uses in at most one Round Two.
3. *Round Two*. The Coordinator chooses a signing set  $I$  and a message  $m$ , and sends the request  $(m, (com_j)_{j \in I})$  to each  $P_i$  with  $i \in I$ . Participant  $P_i$  either returns a *signature share*  $z_i$ , computed from  $sk_i, st_i$ , the request and the group information, or aborts; in both cases it deletes  $st_i$ .
4. *Aggregation*, by which the Coordinator maps  $m$ , the commitment list  $(com_j)_{j \in I}$  and the shares  $(z_j)_{j \in I}$  to a signature  $\sigma$  or to  $\perp$ .
5. *Verification*, the single-party Schnorr verification of a signature on  $m$  under the group key  $PK$ .

The scheme is *correct* if, for all valid key material output by key generation, every signing set  $I$  and every message  $m$ , a run in which every party is honest outputs either  $\perp$  or a signature that verification accepts under  $PK$ , and outputs  $\perp$  with probability negligible in the security parameter.

The parties and the rounds are those of RFC 9591, Section 5: the Coordinator determines the participants, coordinates the rounds, aggregates the signature shares and publishes the signature; Round One produces commitments; Round Two produces a signature share over the message and the commitments that the Coordinator supplies. Theorem 4.11 bounds the probability of  $\perp$  for FROST. Unforgeability notions are defined in §7, before any use.

**Remark 2.9** (Scope of the syntax). The two-round form is the one RFC 9591 specifies (Section 5). Single-round signing with commitments preprocessed in batches (ePrint 2020/852) is out of the scope of RFC 9591 (Section 1, “Introduction”) and of this volume.

The parties compute  $(R, z)$  jointly, and no step computes  $s$  or  $r$ . Round Two binds each share to the message and the whole commitment list (Construction 4.8); unforgeability under concurrent runs is Theorem 7.9, under its assumptions (Remark 7.10).

## 2.4 Channels and corruption

**Definition 2.10** (Channels). Every message of every protocol in this volume travels on a channel of one of the following four types, and the protocol names the type.

- (a) An *authenticated channel with reliable delivery*: the receiver learns the sender and the unmodified message; delivery is assured, as completion of the protocol requires; the content is visible to an observer of the network. It carries the signing messages: the Round One commitments, the Round Two request and the returned shares.
- (b) A *confidential and authenticated channel*: a channel of type (a) whose content is hidden from every party but its endpoints. It carries the Round Two request of the re-randomised variant with its confidential inputs, named in Construction 5.10.
- (c) A *mutually authenticated confidential channel*: a channel of type (b) in which each endpoint is authenticated to the other. It carries dealt shares and the evaluations of round two of distributed key generation.
- (d) *Consistent broadcast*: every honest recipient outputs the same value from the sender, whether or not the sender is honest, or every honest recipient aborts. If the sender is honest, an honest recipient that does not abort outputs the value sent. It carries the coefficient commitments and the proofs of knowledge of key generation.

Realisations are not described. For what authentication provides against an adversary that controls the network, see *Crypto Guide*, §“Active attacks and the role of authentication”.

Each type is tied to its source. Type (a) is that of RFC 9591, Section 5, which assumes reliable delivery for completion and an authenticated channel, confidentiality not being required; Section 7 (“Security Considerations”) adds that signing may run over a public channel provided it is authenticated and reliable. Authentication is what identifies a misbehaving participant: without it, a party masquerading as another causes only an invalid signature (RFC 9591, Section 5). Type (b) is that of ZIP 312, “Round Two - Signature Share Generation”, which requires a confidential and authenticated channel and records the difference from RFC 9591. Type (c) is that of RFC 9591, Appendix C, which requires a mutually authenticated secure channel with confidentiality and integrity for dealt shares, and of ePrint 2020/852, Figure 1, in which round two of key

generation sends the evaluations securely. Type (d) is required by RFC 9591, Appendix C, under which participants must abort if they do not hold the same view of the commitment vector, and by ePrint 2020/852, Figure 1, which broadcasts commitments and proofs.

**Definition 2.11** (Static corruption). Before key generation the adversary fixes a set  $C \subseteq \{1, \dots, n\}$  with  $|C| \leq t - 1$ . It receives the entire state of each  $P_j$  with  $j \in C$  throughout, and chooses their messages. It may in addition control the Coordinator, which holds no share of the signing key and no nonce; in the re-randomised variant it also holds the randomiser and the transaction data (Definition 5.14). It observes and schedules the network, subject to the guarantees of Definition 2.10. The set  $C$  never changes, and every other participant follows the protocol.

A malicious Coordinator can deny service, by withholding requests, the aggregate or its publication, and can falsely report a participant as misbehaving; no honest party can prevent either, since FROST provides no robustness (RFC 9591, Section 7; ePrint 2020/852, Section 5, “Signature Aggregator Role”). What a malicious Coordinator cannot do is stated in §7, either as a theorem (Theorem 7.9, Theorem 7.20) or as an open problem (Remark 7.22); Definition 2.11 asserts none of it. The re-randomised variant further trusts the Coordinator and every share holder with the privacy of the transaction (Definition 5.14). Adaptive corruption, metadata privacy and network privacy are out of scope. The model is that of RFC 9591, Section 7, under which the Coordinator and at most  $\text{MIN\_PARTICIPANTS} - 1$  participants may be corrupted and robustness and metadata protection are not goals, and of ZIP 312, “Threat Model”.

### 3 Key generation

This section states the status of key generation in RFC 9591 and ZIP 312 and the two protocols that output the threshold key material of Definition 2.5: key generation by a trusted dealer and distributed key generation. Each is proved to output valid key material and is classified by Definition 1.1.

#### 3.1 Trusted-dealer key generation

RFC 9591 places key generation out of its scope and, for completeness, specifies key generation by a trusted dealer in Appendix C (“Trusted Dealer Key Generation”); Section 1 (“Introduction”) says so. Its Section 5 (“Two-Round FROST Signing Protocol”) names the two mechanisms by which the key material of Definition 2.5 may be configured, a trusted dealer and a distributed key generation protocol, and specifies neither beyond that appendix. ZIP 312 specifies no key generation (“Non-requirements”). Its “Key Generation” requires key generation to be consistent with FROST, referring to the dealer appendix of RFC 9591 for guidance, and names the spend authorising key ask as the key to be shared. It records that ask is usually derived from the spending key  $sk$  but need not be; that not deriving it permits distributed key generation, since the key so generated is unpredictable; and that not deriving it prevents recovery of the secret from a seed phrase, which the ZIP notes may be desirable for FROST. The security analysis of Bellare, Tessaro and Zhu (ePrint 2022/833, in its current revision, whose preliminary version is part of the analysis RFC 9591 cites) models key generation as an algorithm run by a trusted party. Two protocols output key material: the dealer of RFC 9591, Appendix C, in this subsection, and the distributed key generation of ePrint 2020/852, Figure 1, in §3.2.

**Assumption 3.1** (Honest dealer). The dealer runs Construction 3.2 as written:

1. it samples  $s$  and the coefficients  $a_1, \dots, a_{t-1}$  uniformly and independently from its own randomness;
2. it reveals  $s$ , the coefficients and the shares only as the construction prescribes, the share of  $P_j$  to  $P_j$  alone;
3. it deletes  $s$ , the coefficients and the shares once the shares are distributed.

The adversary of Definition 2.11 does not control the dealer and never obtains its state.

The three clauses are the trust requirements of RFC 9591, Appendix C: the dealer is trusted to generate good randomness, to delete secret values after distributing the shares, and to keep secret values confidential; the appendix further requires that the dealer delete the secret key and the secret key shares upon completion.

**Construction 3.2** (Dealer sharing of a threshold key). Let  $(t, n)$  be threshold parameters with identifiers  $1, \dots, n$  (Definition 2.1), over the group  $\mathbb{G}$  of prime order  $q$  with generator  $B$  and scalar field  $\mathbb{F}_q$ . The dealer proceeds as follows.

1. It samples the signing key  $s \xleftarrow{\$} \mathbb{F}_q$  uniformly and  $a_1, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$  independently and uniformly.
2. It sets  $a_0 := s$  and

$$f(X) := \sum_{k=0}^{t-1} a_k X^k \in \mathbb{F}_q[X],$$

a sharing of  $s$  by *Crypto Guide*, Construction “Shamir  $t$ -of- $n$  secret sharing”.

3. For  $j = 1, \dots, n$  it sends the secret share  $(j, sk_j)$ , with  $sk_j := f(j)$ , to  $P_j$  over a mutually authenticated confidential channel (Definition 2.10, type (c)).
4. It sends the coefficient commitments

$$A_k := [a_k]B, \quad k = 0, \dots, t-1,$$

to every party, the participants and the Coordinator, by consistent broadcast (Definition 2.10, type (d)).

5. It deletes  $s$ , the coefficients and the shares.

Participant  $P_j$  aborts if its view of  $(A_0, \dots, A_{t-1})$  differs from that of another party, and aborts if

$$[sk_j]B \neq \sum_{k=0}^{t-1} [j^k]A_k.$$

Every party computes the group information

$$PK := A_0, \quad PK_i := \sum_{k=0}^{t-1} [i^k]A_k \quad (i = 1, \dots, n)$$

by Lemma 2.6; by this computation every party learns every verification share. *Output*: the share  $(j, sk_j)$  to  $P_j$ , and  $(PK, PK_1, \dots, PK_n)$  to every party. The dealer knows  $s$ . Class (Definition 1.1): specified.

The construction is that of RFC 9591, Appendix C, with its Appendices C.1 (“Shamir Secret Sharing”) and C.2 (“Verifiable Secret Sharing”). The secret is generated uniformly at random

and must be derived from at least  $N_s$  bytes of entropy,  $N_s$  being the length of an encoded scalar; Appendix D (“Random Scalar Generation”) gives the generation of uniform scalars. The shares are the evaluations of  $f$  at  $1, \dots, n$ , the secret prepended to the coefficients, and an identifier is never 0 (Appendix C.1). The commitment is the vector of the  $[a_k]B$ ; a participant must abort if it does not hold the same view of it as the other participants, which a secure broadcast channel ensures (Appendix C), and must abort if its share fails the check, the failure being investigated out of band (Appendix C.2). The channel for shares is mutually authenticated and provides confidentiality and integrity (Appendix C). Consistent broadcast realises the first abort rule: under Definition 2.10, type (d), every honest recipient outputs the same vector or every honest recipient aborts.

**Proposition 3.3** (Validity of dealt key material). *Let the commitments of Construction 3.2 travel by consistent broadcast (Definition 2.10, type (d)).*

- (i) *Under Assumption 3.1, if the broadcast does not abort, no honest participant aborts, and the shares  $(j, sk_j)$  with the group information  $(PK, PK_1, \dots, PK_n)$  are valid key material for  $s$  (Definition 2.5), with  $PK = [s]B$ .*
- (ii) *Without Assumption 3.1, the commitments determine a unique polynomial  $f^* = \sum_k a_k^* X^k$  of degree at most  $t - 1$  with  $[a_k^*]B = A_k$  for every  $k$ ; a participant whose check passes holds  $sk_j = f^*(j)$ ; and every honest party computes the same group information, that of  $f^*$ . The shares that pass and the group information are thus consistent with the one polynomial  $f^*$ , whose constant term  $\log_B A_0$  the dealer chooses freely.*
- (iii) *Under Assumption 3.1, for every set  $C$  of fewer than  $t$  participants there is a randomised algorithm that, given  $PK = [s]B$  alone, outputs a tuple distributed identically to the joint view of  $C$ : its shares and the commitments. The commitments reveal  $[s]B$  and nothing further to  $C$ .*

*Proof.* (i) By steps (1) to (3),  $sk_j = f(j)$  for the polynomial  $f$  of degree at most  $t - 1$  with  $f(0) = s$ . Each honest share passes its check by the identity  $\sum_k [j^k]A_k = [f(j)]B$  displayed in *Crypto Guide*, Construction “Feldman verifiable secret sharing”, and consistent broadcast from an honest dealer that does not abort gives every honest party the dealer’s vector, so no honest participant aborts. Lemma 2.6, part (i), gives  $PK = A_0 = [f(0)]B = [s]B$  and  $PK_j = [f(j)]B = [sk_j]B$ , which are the conditions of Definition 2.5.

(ii) Consistent broadcast gives every honest party the same vector  $(A_0, \dots, A_{t-1})$ , or every honest party aborts. By *Crypto Guide*, Proposition “What the Feldman check certifies”, that vector determines the unique  $f^*$  with  $a_k^* = \log_B A_k$ , a check passes if and only if  $sk_j = f^*(j)$ , and nothing constrains  $f^*(0)$ . Lemma 2.6, part (i), applied to  $f^*$ , gives the common group information  $PK = [f^*(0)]B$  and  $PK_j = [f^*(j)]B$ .

(iii) Suppose first  $|C| = t - 1$ . By *Crypto Guide*, Theorem “Perfect secrecy below the threshold”, the shares  $(sk_i)_{i \in C}$  are jointly uniform on  $\mathbb{F}_q^{t-1}$  whatever  $s$  is. Given the shares, the points  $[f(0)]B = PK$  and  $[f(i)]B = [sk_i]B$ ,  $i \in C$ , are  $t$  values of the map  $x \mapsto [f(x)]B$  at distinct points; since each  $A_k$  is a fixed  $\mathbb{F}_q$ -linear combination of these values (interpolation in the exponent), the commitments are a function of  $PK$  and the shares. The algorithm samples  $t - 1$  uniform scalars as the shares and computes the commitments so; this is the argument of *Crypto Guide*, Remark “The leak, and where it is harmless”. If  $|C| < t - 1$ , then, since  $t \leq n$ , some set  $C' \supseteq C$  of  $t - 1$  participants exists; the view of  $C$  is a projection of that of  $C'$ , and the algorithm for  $C'$  followed by the projection serves for  $C$ .  $\square$

**Remark 3.4** (Key material of an Orchard-protocol key). For an Orchard-protocol key the shared secret is  $ask$  (ZIP 312, “Key Generation”). The output of Construction 3.2 then passes through Construction 6.7 in §6.3 and is completed by Construction 6.8. ZIP 312 also admits a dealer that

shares an ask derived from a spending key; that ask is not sampled as in Assumption 3.1, clause (1), and §7.3 treats both cases.

### 3.2 Distributed key generation

**Assumption 3.5** (The proof-of-knowledge hash). The challenge hash

$$H_{\text{dkg}} : \{1, \dots, n\} \times \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{F}_q$$

of Construction 3.6, applied to an injective encoding of its input (identifier, context string, point, point), is modelled as a random oracle (*Crypto Guide*, §“The random oracle model”) independent of every other hash of the volume, in particular of the ciphersuite hashes of Assumption 4.2.

An instantiation achieves the independence by domain separation: a random oracle restricted to prefix-free tags yields independent random oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”). No ZIP fixes  $H_{\text{dkg}}$  or the context string for the Pallas ciphersuite of ZIP 312: that ciphersuite defines the hashes  $H_1$  to  $H_5$  and  $H_R$  only (“FROST(Pallas, BLAKE2b-512)”), and key generation is outside the ZIP’s scope (“Key Generation”). The volume therefore takes  $H_{\text{dkg}}$  as the named oracle of Assumption 3.5.

**Construction 3.6** (Pedersen key generation with proofs of knowledge). Let  $(t, n)$  be threshold parameters (Definition 2.1) and  $H_{\text{dkg}}$  the hash of Assumption 3.5. Before Round One the participants agree, by means outside the construction, on a context string  $\Phi \in \{0, 1\}^*$  unique to the ceremony.

*Round One*, participant  $P_i$ :

1. It samples  $a_{i,0}, \dots, a_{i,t-1} \xleftarrow{\$} \mathbb{F}_q$  independently and uniformly and sets  $f_i(X) := \sum_{k=0}^{t-1} a_{i,k} X^k$ .

2. It proves knowledge of  $a_{i,0}$ : it samples  $\nu_i \xleftarrow{\$} \mathbb{F}_q$  and sets

$$\begin{aligned} R_i &:= [\nu_i]B, & c_i &:= H_{\text{dkg}}(i, \Phi, [a_{i,0}]B, R_i), \\ \mu_i &:= \nu_i + a_{i,0} c_i, & \sigma_i &:= (R_i, \mu_i). \end{aligned}$$

3. It computes  $A_{i,k} := [a_{i,k}]B$  for  $k = 0, \dots, t-1$ .
4. It sends  $(A_{i,0}, \dots, A_{i,t-1}, \sigma_i)$  to every other party, the Coordinator included, by consistent broadcast (Definition 2.10, type (d)).
5. For every  $\ell \neq i$  it computes  $c_\ell := H_{\text{dkg}}(\ell, \Phi, A_{\ell,0}, R_\ell)$  and aborts unless

$$R_\ell = [\mu_\ell]B - [c_\ell]A_{\ell,0}.$$

If every check passes, it deletes the proofs  $\sigma_\ell$ .

*Round Two*, participant  $P_i$ :

1. It sends  $f_i(\ell)$  to each  $P_\ell$ ,  $\ell \neq i$ , over a mutually authenticated confidential channel (Definition 2.10, type (c)), keeps  $f_i(i)$ , and deletes  $f_i$  and the values sent.

2. For every  $\ell \neq i$  it aborts unless

$$[f_\ell(i)]B = \sum_{k=0}^{t-1} [i^k]A_{\ell,k}.$$

3. It sets  $sk_i := \sum_{\ell=1}^n f_\ell(i)$  and deletes the  $f_\ell(i)$ .

*Group information.* Every party sets  $A_k := \sum_{\ell=1}^n A_{\ell,k}$  for  $k = 0, \dots, t-1$  and computes

$$PK := A_0, \quad PK_j := \sum_{k=0}^{t-1} [j^k]A_k \quad (j = 1, \dots, n)$$

by Lemma 2.6.

*Output:*  $(i, sk_i)$  to  $P_i$ , and  $(PK, PK_1, \dots, PK_n)$  to every party. A failed check leads to abort only; its cause is investigated out of band.

The construction is that of ePrint 2020/852, Figure 1, in additive notation: the paper's  $g, \phi_{i,k}, k, s_i, Y_i$  and  $Y$  are  $B, A_{i,k}, v_i, sk_i, PK_i$  and  $PK$ . The paper's Section 5.1 describes it as Pedersen's protocol,  $n$  parallel runs of *Crypto Guide*, Construction "Feldman verifiable secret sharing", one with each participant as dealer, each share being the sum of the  $n$  shares received. Round One, step (2), is the Fiat–Shamir transform of *Crypto Guide*, Construction "Schnorr identification" (§"The Schnorr identification protocol"; §"The Fiat–Shamir transform: from interactive to non-interactive"), with statement  $A_{i,0}$ , witness  $a_{i,0}$ , and the prover's identifier and  $\Phi$  bound into the challenge; the verification equation of step (5) is that of the identification protocol rearranged. ePrint 2020/852, Figure 1, binds the context string into the challenge to prevent replay of proofs across ceremonies, a design claim that no cited source proves (Remark 3.9). The last part of Figure 1 computes every verification share from the broadcast commitments, which is how parties other than  $P_j$  learn  $PK_j$ .

Consistent broadcast of Round One is a hypothesis of the construction: ePrint 2020/852, Section 5.1, assumes that participants hold a consistent view of the commitments and does not provide it. A participant that sent different commitment vectors to different parties would lead them to compute different group information: different verification shares, and different group keys when the constant terms  $A_{\ell,0}$  differ.

**Lemma 3.7** (Sum of dealt sharings). *Let  $f_1, \dots, f_n \in \mathbb{F}_q[X]$  have degree at most  $t-1$ , with coefficients  $a_{\ell,k}$  and coefficient commitments  $A_{\ell,k} = [a_{\ell,k}]B$ . Then  $f := \sum_{\ell=1}^n f_\ell$  has degree at most  $t-1$ , constant term  $f(0) = \sum_{\ell} a_{\ell,0}$ , and coefficient commitments*

$$\left[ \sum_{\ell=1}^n a_{\ell,k} \right] B = \sum_{\ell=1}^n A_{\ell,k}, \quad k = 0, \dots, t-1;$$

*and  $f(j) = \sum_{\ell} f_\ell(j)$  for every  $j \in \mathbb{F}_q$ .*

*Proof.* Polynomial addition adds coefficients and does not raise the degree, so the coefficient of  $X^k$  in  $f$  is  $\sum_{\ell} a_{\ell,k}$ , zero for  $k \geq t$ . Evaluation at a point is additive in the polynomial. The map  $a \mapsto [a]B$  is a group homomorphism  $\mathbb{F}_q \rightarrow \mathbb{G}$ , which gives the commitments.  $\square$

**Proposition 3.8** (Validity of distributed key material). *Consider an execution of Construction 3.6 without abort in which the Round One messages travel by consistent broadcast. For each  $\ell$  let  $f_\ell^*$  be*

the unique polynomial of degree at most  $t - 1$  whose coefficient commitments are  $A_{\ell,0}, \dots, A_{\ell,t-1}$ , let  $f^* := \sum_{\ell} f_{\ell}^*$ , and let  $sk_j := f^*(j)$  for every identifier  $j$ . Then:

- (i) every honest participant  $P_i$  outputs  $sk_i = f^*(i)$ ;
- (ii) every honest party computes the same group information,  $PK = [f^*(0)]B$  and  $PK_j = [f^*(j)]B$ ;
- (iii) the shares  $sk_j$  with this group information are valid key material (Definition 2.5) for the signing key

$$s = f^*(0) = \sum_{\ell=1}^n \log_B A_{\ell,0}.$$

If every participant is honest,  $f_{\ell}^* = f_{\ell}$  for every  $\ell$ .

*Proof.* The polynomials  $f_{\ell}^*$  exist and are unique by *Crypto Guide*, Proposition “What the Feldman check certifies”. An honest  $P_i$  broadcasts the commitments of its own  $f_i$ , so  $f_i = f_i^*$  and its kept value is  $f_i(i) = f_i^*(i)$ . Each received  $f_{\ell}(i)$ ,  $\ell \neq i$ , passed the check of Round Two, step (2), so equals  $f_{\ell}^*(i)$  by the same proposition. Hence  $sk_i = \sum_{\ell} f_{\ell}^*(i) = f^*(i)$  by Lemma 3.7, which is (i). Consistent broadcast gives every honest party the same  $A_{\ell,k}$ , hence the same  $A_k = \sum_{\ell} A_{\ell,k}$ , which by Lemma 3.7 are the coefficient commitments of  $f^*$ , of degree at most  $t - 1$ . Lemma 2.6, part (i), applied to  $f^*$  gives  $PK = [f^*(0)]B$  and  $PK_j = [f^*(j)]B = [sk_j]B$ , which is (ii); with the shares  $sk_j = f^*(j)$  these are the conditions of Definition 2.5, which is (iii), and  $f^*(0) = \sum_{\ell} f_{\ell}^*(0) = \sum_{\ell} \log_B A_{\ell,0}$ . If every participant is honest, each broadcasts the commitments of its own polynomial, so  $f_{\ell}^* = f_{\ell}$ .  $\square$

**Remark 3.9** (Rogue keys and robustness). The statements of this remark are design claims of ePrint 2020/852; none is a theorem. The proofs of knowledge of Round One, step (2), are the change from Pedersen’s protocol: the paper adds them against rogue-key attacks, in which a participant chooses its commitment as a function of the others’ commitments to bias the group key, in the setting  $t \geq n/2$  (Section 5.1). No source cited in this volume proves Construction 3.6 secure, or proves FROST as RFC 9591 specifies it unforgeable with it; §7.3 states what is known (Remark 7.13). The protocol is not robust: any failed proof or share check aborts the whole execution and its cause is investigated out of band (Figure 1; Section 3), so one participant can prevent key generation.

Construction 3.6 is designed but unspecified for Zcash (Definition 1.1). The protocol is published (ePrint 2020/852, Figure 1), but no RFC or ZIP specifies a distributed key generation: RFC 9591 places key generation out of scope (Section 1), and ZIP 312 specifies none (“Key Generation”). ZIP 2005 (Proposed), “Usage with FROST”, names the FROST distributed key generation protocol without a specification behind the reference; the revision of ZIP 312 proposed in pull request 895 names a different key-generation protocol, so its adoption would not specify this one. Its Zcash instantiation, the hash  $H_{\text{dkg}}$  and the context string  $\Phi$ , is fixed by no ZIP (Assumption 3.5).

## 4 Signing and aggregation

This section states FROST signing as RFC 9591 specifies it, in execution order: the ciphersuite interface with its hash roles and the encoding of a commitment list; Round One, which draws hedged nonces and commits to them; Round Two, which binds each signature share to the message and to the whole commitment list; aggregation with share verification; the correctness of an honest run; and the single use of nonces, with the exact condition under which reuse of a nonce pair exposes a share. Threshold parameters, signing sets, Lagrange coefficients and key material are those of §2; the key material is the output of either key generation of §3.

## 4.1 Ciphersuites and encodings

**Definition 4.1** (Ciphersuite). A *ciphersuite* fixes the following data.

- (a) A group  $\mathbb{G}$  of prime order  $q$ , written additively, with identity  $O$  and generator  $B$ , and its scalar field  $\mathbb{F}_q$ . Write  $\mathbb{G}^* := \mathbb{G} \setminus \{O\}$ .
- (b) An *element serialisation*  $\text{ser}_{\mathbb{G}}$ , mapping either every element of  $\mathbb{G}$  or every element of  $\mathbb{G}^*$  injectively to a byte string of fixed length  $N_e$ , and an *element deserialisation*  $\text{deser}_{\mathbb{G}}$ , which maps a byte string to an element of  $\mathbb{G}^*$  or fails. It fails on every string that is not the canonical encoding  $\text{ser}_{\mathbb{G}}(P)$  of some  $P \in \mathbb{G}$ , and on the encoding of  $O$  where one exists; on every other string it inverts  $\text{ser}_{\mathbb{G}}$ . The serialisation is *total* if its domain is  $\mathbb{G}$ .
- (c) A *scalar serialisation*  $\text{ser}_{\mathbb{F}_q}$ , mapping  $\mathbb{F}_q$  injectively to byte strings of fixed length  $N_s$ , and a *scalar deserialisation*  $\text{deser}_{\mathbb{F}_q}$ , which fails on every string whose integer value is not below  $q$  and inverts  $\text{ser}_{\mathbb{F}_q}$  otherwise.
- (d) Hash functions by role:

$$H_1, H_2, H_3: \{0, 1\}^{8*} \rightarrow \mathbb{F}_q, \quad H_4, H_5: \{0, 1\}^{8*} \rightarrow \{0, 1\}^{8N_h},$$

where  $\{0, 1\}^{8*}$  denotes the byte strings and  $N_h$  is the digest length of the suite. The roles are:  $H_1$  the binding factor,  $H_2$  the challenge,  $H_3$  the nonce,  $H_4$  the message pre-hash and  $H_5$  the commitment-list pre-hash.

- (e) The *signature encoding*  $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$  of a pair  $(R, z) \in \mathbb{G} \times \mathbb{F}_q$ , and the *single-party verification*: a pair  $(R, z)$  is accepted on a message  $m$  under a public key  $PK$  if and only if

$$[z]B = R + [c]PK, \quad c := H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m).$$

This is the verification of the *Crypto Guide*'s Construction "Schnorr signature" (§"From identification to signature via the Fiat–Shamir transform") in additive form, with hash  $H_2$ .

A ciphersuite for *re-randomised signing* adds

- (f) the *randomiser hash*  $H_R: \{0, 1\}^{8*} \rightarrow \mathbb{F}_q$  (ZIP 312, "Specification" and "Randomizer Generation").

The ciphersuites of RFC 9591 do not define  $H_R$ .

The data are those of RFC 9591, Sections 3.1 ("Prime-Order Group") and 3.2 ("Cryptographic Hash Function"), and Section 6 ("Ciphersuites"), whose Appendix A ("Schnorr Signature Encoding") fixes the encoding and whose Appendix B ("Schnorr Signature Generation and Verification for Prime-Order Groups") the verification equation. The requirements that RFC 9591, Section 6.6 ("Ciphersuite Requirements"), places on every ciphersuite are four: the group is of prime order and every deserialisation outputs an element of its set or fails; every hash function is domain-separated with a per-suite context string, FROST(Ed25519, SHA-512) excepted for  $H_2$  for compatibility with RFC 8032;  $H_1$ ,  $H_2$  and  $H_3$  have output distributions close to uniform; and the signature encoding is specified. The suite FROST(Pallas, BLAKE2b-512) separates its hashes by BLAKE2b personalisations instead, its  $H_2$  being that of RedPallas (Construction 6.1).

The generic element serialisation of RFC 9591, Section 3.1, fails on  $O$ ; a suite may instead serialise every element, as FROST(Pallas, BLAKE2b-512) does with the star encoding (*Ironwood Guide*, Definition "Star encoding"; ZIP 312, "FROST(Pallas, BLAKE2b-512)"). In both cases the element deserialisation rejects  $O$ . Whether  $\text{ser}_{\mathbb{G}}$  is total is the one property of a suite on which

Theorem 4.11 distinguishes cases.

**Assumption 4.2** (The ciphersuite hashes). The hashes  $H_1, H_2, H_3$  and  $H_R$  are modelled as random oracles into  $\mathbb{F}_q$  (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”), independent of each other and of the proof-of-knowledge hash  $H_{\text{dkg}}$  (Assumption 3.5): each answer at a new input is an independent sample from a distribution on  $\mathbb{F}_q$  within statistical distance  $\delta$  of uniform (*Math Guide*, §“Statistical distance”), where  $\delta$  is a parameter of the suite. The hashes  $H_4$  and  $H_5$  are modelled as independent random oracles into  $\{0, 1\}^{8N_h}$ ; hence an algorithm making  $Q$  queries finds a collision of either with probability at most

$$\frac{Q(Q-1)}{2} \cdot 2^{-8N_h}$$

(*Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”). For a suite that builds every role from one hash function under distinct personalisations, the assumption is that this function, keyed by its personalisation, is a random oracle; distinct personalisations then give independent oracles (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”), and  $\delta$  is the bias of the reduction of its output into  $\mathbb{F}_q$  (*Crypto Guide*, §“Hashing to a field element”).

The assumption is the one hash assumption of every ciphersuite in this volume. RFC 9591, Section 3.2, models the ciphersuite hash as a random oracle in the security analyses it cites. The suite FROST(Pallas, BLAKE2b-512) meets the assumption with  $\delta \leq p_{\text{Vesta}}/2^{512} < 2^{-257}$  and no further assumption, by Proposition 6.2.

**Definition 4.3** (Commitment list and its encoding). Let a ciphersuite be fixed and let  $I$  be a signing set (Definition 2.1). A *commitment list* over  $I$  is a sequence

$$L = ((i, D_i, E_i))_{i \in I}, \quad D_i, E_i \in \mathbb{G},$$

with one triple for each member of  $I$ :  $D_i$  is the *hiding commitment* and  $E_i$  the *binding commitment* of member  $i$ . The identifiers are distinct and the triples are sorted in ascending order of the least non-negative representative of the identifier. The signing set is read off  $L$  as the set of its identifiers. The *encoding* of  $L$  is the byte string

$$\text{enc}(L) := \parallel_{i \in I, \text{ ascending}} \left( \text{ser}_{\mathbb{F}_q}(i) \parallel \text{ser}_{\mathbb{G}}(D_i) \parallel \text{ser}_{\mathbb{G}}(E_i) \right)$$

of length  $|I|(N_s + 2N_e)$ .

A commitment equal to  $O$  occurs only in a suite with total element serialisation, and check (3) of Construction 4.8 rejects it.

The ordering is a requirement of RFC 9591, which states that the list must be sorted in ascending order by identifier, and compares scalars by their least non-negative representative (Sections 3.1 and 4.3, “List Operations”); the encoding is that of Section 4.3. Every component has fixed length, each serialisation is injective, and the order of the triples is fixed; hence the length of  $\text{enc}(L)$  determines  $|I|$ , and the string parses uniquely into the triples. The map  $\text{enc}$  is therefore injective on commitment lists over one suite, which the binding factors of Definition 4.7 require.

## 4.2 Round One: nonces and commitments

**Construction 4.4** (Round One). Participant  $P_i$ , holding the share  $sk_i$  (Definition 2.5), proceeds as follows.

1. It draws two independent strings  $T_d, T_e$ , each uniform on the 32-byte strings, and sets the *hiding nonce* and the *binding nonce*

$$d_i := H_3(T_d \parallel \text{ser}_{\mathbb{F}_q}(sk_i)), \quad e_i := H_3(T_e \parallel \text{ser}_{\mathbb{F}_q}(sk_i)).$$

2. It sets  $D_i := [d_i]B$  and  $E_i := [e_i]B$ .
3. It stores  $(d_i, e_i, D_i, E_i)$ , keeps  $(d_i, e_i)$  secret, and sends  $(\text{ser}_{\mathbb{G}}(D_i), \text{ser}_{\mathbb{G}}(E_i))$  to the Coordinator over an authenticated channel (Definition 2.10, type (a)). If a serialisation fails, which happens only in a suite whose element serialisation is not total and only if  $D_i$  or  $E_i$  is  $O$ , the participant discards the pair and this execution of Round One fails.

The strings  $T_d$  and  $T_e$  must be sampled uniformly from a secure source of randomness, and a nonce pair must not be used in more than one signing run.

The construction is RFC 9591, Section 5.1 (“Round One - Commitment”), with the hedged nonce generation of Section 4.1 (“Nonce Generation”), which always samples 32 bytes of fresh randomness and hashes them together with the serialised share. Section 5.1 requires that the nonces not be used in more than one signing and be generated from a source of secure randomness; Section 7.3 (“Nonce Reuse Attacks”) requires the randomness to be sampled uniformly. Nonces lie in  $\mathbb{F}_q$ , not in  $\mathbb{F}_q^*$ : the zero nonce is not excluded, and its consequence is case (1) of Theorem 4.11. Round One does not depend on the message or on the signing set, so a participant may run it ahead of any signing run and hold several unused pairs. The paper that introduced FROST (ePrint 2020/852, Figure 2) samples the pair uniformly from  $\mathbb{F}_q^* \times \mathbb{F}_q^*$ ; the specified rule is the hedged derivation above, and this volume states it instead.

**Lemma 4.5** (Hedged nonces are close to uniform). *Under Assumption 4.2, let an algorithm, the observer, make at most  $q_h$  queries to  $H_3$  and, at one point, choose a byte string  $w$  and receive  $d := H_3(T \parallel w)$ , for  $T$  uniform on the 32-byte strings and independent of all else. The pair formed by the observer’s view and  $d$  is within statistical distance*

$$\delta + q_h \cdot 2^{-256}$$

*of the pair obtained when  $d$  is replaced by a uniform element of  $\mathbb{F}_q$  independent of all else. For the two nonces of one execution of Round One, received for one string  $w$  with independent strings  $T_d$  and  $T_e$ , the corresponding distance is at most*

$$2(\delta + q_h \cdot 2^{-256}) + 2^{-256}.$$

*Proof.* The argument is that of the *Ironwood Guide*’s Lemma “Distance of GenRandom from uniform” (§“Randomised validating keys”), with 32 fresh bytes in place of 80 and  $H_3$  in place of the RedPallas hash. Let  $E$  be the event that the observer queries  $H_3$  at  $T \parallel w$ . In a second experiment the answer  $H_3(T \parallel w)$  is replaced, in the computation of  $d$  only, by an independent sample of the answer distribution. In the lazy realisation of the random oracle the answers at other inputs are independent of the answer at  $T \parallel w$ , so the two experiments are identical until  $E$  occurs, and their outputs are within statistical distance  $\Pr[E]$  (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”). In the second experiment the observer’s view is independent of  $T$ , so each query has prefix  $T$  with probability at most  $2^{-256}$ , and  $\Pr[E] \leq q_h \cdot 2^{-256}$  by the union bound. There the sample is independent of everything else, and replacing it by a uniform scalar changes

the pair by at most  $\delta$  (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (4)). The triangle inequality, part (2) of the same theorem, gives the first bound.

For the pair, the event  $T_d = T_e$ , of probability  $2^{-256}$ , is excluded at that cost; outside it the inputs  $T_d \parallel w$  and  $T_e \parallel w$  are distinct. The event that the observer queries either input has probability at most  $2q_h \cdot 2^{-256}$ , and two independent samples are replaced by uniform scalars at cost  $2\delta$ .  $\square$

The nonces are therefore close to uniform, not uniform, whatever the share; RFC 9591, Section 7.3, states that they are indistinguishable from values sampled uniformly at random, and the lemma quantifies that statement. For FROST(Pallas, BLAKE2b-512), with the  $\delta$  of Proposition 6.2, the first bound at  $q_h = 2^{64}$  is below  $2^{-191.99}$ , the term  $q_h \cdot 2^{-256}$  dominating; in a suite with a larger  $\delta$ , such as a 48-byte hash to field,  $\delta$  dominates.

RFC 9591, Section 4.1, hashes the share with the fresh bytes to hedge against a generator whose output is predictable to an observer; Lemma 4.5 assumes  $T$  uniform, and the volume proves no statement for a predictable  $T$ . Nonces computed from the share and the message alone permit complete key recovery (RFC 9591, Sections 1 and 7.3), which is why fresh bytes are used.

Over  $N = 2^{64}$  executions of Round One by one participant, the 32-byte strings of one role, hiding or binding, repeat with probability at most  $\binom{N}{2} \cdot 2^{-256} < 2^{-129}$  (*Math Guide*, Proposition “Birthday bound”), so, by the union bound, a repeat in either role has probability at most  $2^{-128}(1 - 2^{-64}) < 2^{-128}$ , the figure of RFC 9591, Section 4.1. A repeated nonce *pair*, the hypothesis of Lemma 4.13, requires in the same two executions equal hiding nonces and equal binding nonces.

**Lemma 4.6** (Repetition of a nonce pair). *Under Assumption 4.2, two of  $N$  executions of Round One by one participant yield equal nonce pairs with probability at most*

$$\binom{N}{2}(2^{-255} + 1/q + \delta)^2.$$

*Proof.* For two executions  $j \neq k$ , the hiding strings are equal with probability  $2^{-256}$ , and so are the binding strings, independently. Two answers of  $H_3$  at distinct inputs are equal with probability at most  $1/q + \delta$ , since every value has probability at most  $1/q + \delta$  under a distribution within  $\delta$  of uniform (*Math Guide*, Theorem “Properties of statistical distance”, part (3)); and when both roles have distinct strings, the two equalities constrain answers at two distinct pairs of inputs, whose probability of holding together is at most  $(1/q + \delta)^2$ , except when the two pairs of inputs coincide, an event of probability at most  $2 \cdot 2^{-512}$ . Summing the cases gives at most  $2^{-512} + 2 \cdot 2^{-256}(1/q + \delta) + (1/q + \delta)^2 + 2^{-511}(1/q + \delta)$ , which is below the square displayed; the union bound over the  $\binom{N}{2}$  pairs of executions gives the claim.  $\square$

### 4.3 Round Two: binding factors and signature shares

**Definition 4.7** (Binding factors and group commitment). Let  $PK$  be the group key,  $m$  a message, that is a byte string, and  $L$  a commitment list over a signing set  $I$  (Definition 4.3). For  $i \in I$  the *binding factor* of  $i$  is

$$\rho_i := H_1(\text{ser}_{\mathbb{G}}(PK) \parallel H_4(m) \parallel H_5(\text{enc}(L)) \parallel \text{ser}_{\mathbb{F}_q}(i));$$

the *commitment share* of  $i$  is  $R_i := D_i + [\rho_i]E_i$ ; the *group commitment* is  $R := \sum_{i \in I} R_i$ ; and the *challenge* is

$$c := H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m),$$

with  $\rho_i$  defined whenever  $\text{ser}_{\mathbb{G}}(PK)$  is, and  $c$  whenever  $\text{ser}_{\mathbb{G}}(R)$  and  $\text{ser}_{\mathbb{G}}(PK)$  are. All four are functions of the public data  $(PK, m, L)$ , computed identically by every member of  $I$  and by the Coordinator.

The binding factor is that of RFC 9591, Section 4.4 (“Binding Factors Computation”), the group commitment that of Section 4.5 (“Group Commitment Computation”), and the challenge that of Section 4.6 (“Signature Challenge Computation”). The challenge input has the order nonce commitment, public key, message of the single-party challenge of Definition 4.1, part (e), so a verifier recomputes  $c$  from  $(R, PK, m)$  alone. The paper’s binding value  $\rho_i = H_1(i, m, B)$ , with  $B$  its commitment list, and its challenge  $c = H_2(R, Y, m)$  (ePrint 2020/852, Figure 3, in multiplicative notation with  $R = \prod_i D_i E_i^{\rho_i}$ ) differ as follows: the specified binding-factor input adds the group key and pre-hashes the message and the encoded list; the challenge input is the paper’s, in the same order. For FROST(Pallas, BLAKE2b-512),  $H_2$  is the RedPallas hash and  $\text{ser}_{\mathbb{G}}$  the star encoding, so  $c$  is the challenge that RedPallas validation recomputes (Construction 6.1; ZIP 312, “Rationale”).

**Construction 4.8** (Round Two). *Coordinator*. It selects a signing set  $I$  with  $t \leq |I| \leq n$ ; takes, for each  $i \in I$ , one commitment pair  $(D_i, E_i)$  received from  $P_i$  in Round One and not placed in any earlier request; forms the commitment list  $L$  over  $I$  (Definition 4.3); and sends  $(m, L)$ , serialised componentwise, to each  $P_i$  with  $i \in I$  over an authenticated channel (Definition 2.10, type (a)).

*Member  $P_i$* , on receiving  $(m, L)$ :

1. it checks that  $m$  is a message it is willing to sign; for spend authorisation this is the check of Definition 5.9;
2. it checks that its identifier  $i$  appears in  $L$  with a commitment pair  $(D_i, E_i)$  that it stores and has not used;
3. it deserialises every commitment of  $L$  with  $\text{deser}_{\mathbb{G}}$ , which fails on a non-canonical encoding and on the encoding of  $O$ ;
4. it checks that the identifiers of  $L$  are distinct, without which  $\lambda_{I,i}$  is undefined, and in ascending order, as RFC 9591 requires of a commitment list (Section 4.3).

It aborts if any check fails. Otherwise it computes  $\rho_i$ ,  $R$  and  $c$  (Definition 4.7), aborting if  $\rho_i$  or  $c$  is undefined, and  $\lambda_{I,i}$  (Definition 2.3), with  $I$  the set of identifiers of  $L$ ; sets the *signature share*

$$z_i := d_i + e_i \rho_i + \lambda_{I,i} sk_i c \in \mathbb{F}_q;$$

deletes  $(d_i, e_i)$  and  $(D_i, E_i)$ ; and only then sends  $\text{ser}_{\mathbb{F}_q}(z_i)$  to the Coordinator over an authenticated channel (Definition 2.10, type (a)).

The construction is RFC 9591, Section 5.2 (“Round Two - Signature Share Generation”). The Coordinator’s choice of at least  $t$  and at most  $n$  participants is required by Section 5. Each participant must validate the commitment list by deserialising each element, must abort if deserialisation fails, and must ensure that its identifier and commitments from Round One appear in the list (Section 5.2); applications that restrict the messages a participant signs perform further validation, which Section 7.7 (“Input Message Validation”) recommends so that participants do not act as signing oracles for arbitrary messages. The computation of  $\lambda_{I,i}$  fails on an absent or repeated identifier (Section 4.2, “Polynomials”). The share equation is that of Section 5.2. After computing its share, each participant must delete the nonce and the corresponding commitment, and must not use the nonce as input more than once. The deletion is the step on which Proposition 4.14 rests.

#### 4.4 Aggregation and share verification

**Construction 4.9** (Aggregation). The Coordinator holds  $(m, L)$  and the group information  $(PK, (PK_j)_{j=1}^n)$  stored from key generation (Definition 2.5). The shares arrive over authenticated channels without confidentiality (Definition 2.10, type (a)). The Coordinator proceeds as follows.

1. It deserialises each share with  $\text{deser}_{\mathbb{F}_q}$  and aborts if any deserialisation fails.
2. It computes  $\rho_i$  and  $R_i$  for  $i \in I, R$  and  $c$  (Definition 4.7), aborting if  $\rho_i$  or  $c$  is undefined.
3. It sets  $z := \sum_{i \in I} z_i$  and outputs the signature  $(R, z)$ , encoded  $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$ .

It should verify  $(R, z)$  on  $m$  under  $PK$  by the single-party verification (Definition 4.1, part (e)) before publishing it, and should abort if the signature is invalid. It may check each share by the *share equation*

$$[z_i]B = R_i + [c \lambda_{I,i}] PK_i,$$

in particular when the aggregate is invalid. The keys  $PK$  and  $PK_i$  in these checks are taken from the stored group information, never from the messages of the run.

The construction is RFC 9591, Section 5.3 (“Signature Share Aggregation”). The Coordinator must validate each share by deserialisation and must abort if validation fails; it should verify the signature under the group key before publishing or releasing it; if that verification fails, it may verify each share individually to identify and act on misbehaving participants; and the group key and the keys  $PK_i$  used there must come from the stored group information. Section 5 requires an authenticated channel for identifying misbehaving participants and does not require confidentiality. The paper checks every share unconditionally before aggregating (ePrint 2020/852, Figure 3); the specified duties are the recommendation and the permission above, and this volume states those.

**Proposition 4.10** (Identifiable abort). *Let the key material be valid (Definition 2.5), let the Coordinator be honest, and let  $c$  be defined.*

- (i) *If the share of every member satisfies its share equation, then  $[z]B = R + [c] PK$ , so the aggregate  $(R, z)$  is accepted by the single-party verification on  $m$  under  $PK$ .*
- (ii) *The share of every member that follows Construction 4.8 satisfies its share equation.*
- (iii) *Hence, if the aggregate is not accepted, the share equation of some member fails, and that member deviated from the protocol; the authenticated channel identifies it as the sender of the share.*

*Proof.* (ii) For a member following the construction,

$$[z_i]B = [d_i]B + [\rho_i][e_i]B + [c \lambda_{I,i}][sk_i]B = R_i + [c \lambda_{I,i}] PK_i,$$

since  $D_i = [d_i]B$ ,  $E_i = [e_i]B$  and, by validity,  $PK_i = [sk_i]B$ .

(i) Let  $f$  be the polynomial of degree at most  $t - 1$  that validity provides, with  $f(j) = sk_j$  and  $PK = [f(0)]B$ . Summing the share equations over  $I$  gives

$$[z]B = \sum_{i \in I} R_i + \left[ c \sum_{i \in I} \lambda_{I,i} sk_i \right] B = R + [cf(0)]B = R + [c] PK,$$

by Lemma 2.4 applied to  $f$ . The challenge  $c$  is the one the single-party verification recomputes from  $(R, PK, m)$  (Definition 4.7), so the verification accepts.

(iii) is the contrapositive of (i), with (ii). □

The mechanism for acting on an identified member is outside RFC 9591 (Section 5.4, “Identifiable Abort”). FROST provides no robustness (Sections 5.4 and 7): one member that withholds or corrupts its share prevents the signature, and a failed run is restarted from Round One with fresh nonces, since the consumed pairs are deleted.

#### 4.5 Correctness of FROST signing

**Theorem 4.11** (Correctness of FROST). *Let the key material be valid (Definition 2.5), as key generation by a dealer or by the distributed protocol outputs it (Propositions 3.3 and 3.8); let  $I$  be a signing set and  $m$  a message that every member of  $I$  accepts; and let every party follow Constructions 4.4, 4.8 and 4.9. Then, outside the three cases below, the run outputs  $(R, z)$  with*

$$R = [r]B, \quad z = r + cs, \quad r := \sum_{i \in I} (d_i + e_i \rho_i),$$

which the single-party verification accepts on  $m$  under  $PK$ . In the three cases the run aborts.

- (1) Zero nonce. Some  $d_i$  or  $e_i$ ,  $i \in I$ , is 0, so  $D_i$  or  $E_i$  is  $O$ . In a suite whose element serialisation is not total, Round One fails on it; in every suite the run aborts no later than check (3) of Round Two, since  $\text{deser}_{\mathbb{G}}$  rejects  $O$ . Under Assumption 4.2 the probability of this case is at most  $2|I|(\delta + 1/q)$  per run.
- (2) Identity group commitment. Only in a suite whose element serialisation is not total, as the generic serialisation of RFC 9591 is not:  $R = O$ , so  $c$  is undefined. Under Assumption 4.2 the probability of this case, outside case (1), is at most  $\delta + 1/q$  per run.
- (3) Identity group key. Only in a suite whose element serialisation is not total:  $s = 0$ , so  $PK = O$  and  $\rho_i$  and  $c$  are undefined. For  $s$  uniform in  $\mathbb{F}_q$ , as under Assumption 3.1, its probability is  $1/q$  per key generation.

A suite with a total element serialisation, such as FROST(Pallas, BLAKE2b-512), has only case (1).

*Proof.* Write  $r_i := d_i + e_i \rho_i$ , so that  $z_i = r_i + c \lambda_{I,i} sk_i$ . The nonce  $r = \sum_{i \in I} r_i$  is additively shared by construction: each summand is formed locally, with no dealer and no interaction; and  $R = \sum_{i \in I} R_i = \sum_{i \in I} [r_i]B = [r]B$ . The Lagrange coefficients turn the Shamir shares  $sk_i = f(i)$  of  $s = f(0)$  into additive shares  $\lambda_{I,i} sk_i$  with sum  $s$ , by Lemma 2.4, since  $f$  has degree at most  $t - 1$  by validity. Hence

$$z = \sum_{i \in I} z_i = r + cs, \quad [z]B = [r]B + [c][s]B = R + [c]PK,$$

where  $c = H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m)$  is the challenge the verifier recomputes (Definition 4.7). Outside the three cases, every check of Round Two passes:  $m$  is accepted, each member’s pair is stored and unused, every commitment lies in  $\mathbb{G}^*$  and deserialises, the identifiers of  $L$  are distinct and sorted, and  $\rho_i$  and  $c$  are defined. Every share deserialises, being an honest serialisation, so aggregation outputs  $(R, z)$ .

(1) Each of the  $2|I|$  nonces is an answer of  $H_3$ , equal to 0 with probability at most  $1/q + \delta$ , because under a distribution within  $\delta$  of uniform every value has probability at most  $1/q + \delta$  (Math Guide, Theorem “Properties of statistical distance”, part (3)). Since  $B$  generates a group of prime order,  $[d]B = O$  if and only if  $d = 0$ . The union bound gives  $2|I|(1/q + \delta)$ .

(2) Outside case (1), fix  $i \in I$ ; then  $E_i \neq O$ . Condition on all answers of  $H_3, H_4$  and  $H_5$ , and on the answers of  $H_1$  at every input other than that of  $\rho_i$ . Then  $R = O$  if and only if  $[\rho_i]E_i$  equals the element  $-D_i - \sum_{j \in I \setminus \{i\}} R_j$ , which is fixed; since  $E_i$  generates  $\mathbb{G}$ , this holds for exactly one value of  $\rho_i$ . The binding factor  $\rho_i$  is the answer of  $H_1$  at an input with suffix  $\text{ser}_{\mathbb{F}_q}(i)$ , distinct from

the inputs of the other members, whose suffixes differ; so it takes that value with probability at most  $1/q + \delta$ .

(3) Since  $B$  generates a group of prime order,  $PK = [s]B = O$  if and only if  $s = 0$ , and  $\text{ser}_{\mathbb{G}}(PK)$ , an input of every  $\rho_i$  and of  $c$ , is then undefined. A uniform  $s \in \mathbb{F}_q$  is 0 with probability  $1/q$ .  $\square$

For FROST(Pallas, BLAKE2b-512), with the  $\delta$  of Proposition 6.2, the bound of case (1) is below  $2^{-251.9}$  for  $|I| = 2$  and below  $2^{-251.3}$  for  $|I| = 3$ . In a suite with the parameters  $q = p_{\text{Vesta}}$  and that  $\delta$  but a serialisation that is not total, the bound of case (2) would be below  $2^{-253.9}$ .

The output of a threshold run is a single-party Schnorr signature under the group key. It carries no threshold marker: a verifier applies it, with  $(PK, m)$  alone and no knowledge of  $t$ ,  $n$  or  $I$ , the verification it applies to a signature of one signer. RFC 9591 states this in Section 5, under which signatures can be verified as if produced by a single signer holding  $s$ , and in Section 6, under which the verification instructions of each suite are equivalent to those for a single participant; Appendix B gives that verification.

**Example 4.12** (A two-of-three signing run). Every number below is output by the volume's toy script; none is computed by hand. The group is the curve  $y^2 = x^3 + 11$  over  $\mathbb{F}_{1009}$ , whose group of points has prime order  $q = 967$ , with generator  $B = (1, 298)$ . A point  $(x, y)$  is serialised as the two-byte little-endian integer  $x + 2^{15}(y \bmod 2)$ , and  $O$  as zero, the analogue of the star encoding; a scalar is serialised as a two-byte little-endian integer. The hashes are BLAKE2b-512 under the personalisations that FROST(Pallas, BLAKE2b-512) assigns to  $H_1$  to  $H_5$  (Construction 6.1), the scalar-valued ones read as little-endian integers and reduced modulo 967. The 32 fresh bytes of each nonce are fixed strings, so that the run is reproducible; the example therefore does not follow the sampling rule of Construction 4.4.

*Key material.* A dealer with  $t = 2$ ,  $n = 3$  uses  $f(X) = 321 + 778X$ , so  $s = 321$  and

$$sk_1 = 132, \quad sk_2 = 910, \quad sk_3 = 721.$$

The coefficient commitments are  $A_0 = (526, 125)$  and  $A_1 = (433, 783)$ , and by Lemma 2.6 the group information is  $PK = A_0$  and  $PK_j = A_0 + [j]A_1$ :

$$PK_1 = (963, 797), \quad PK_2 = (587, 458), \quad PK_3 = (30, 509),$$

each equal to  $[sk_j]B$ .

*Round One.* Members 1 and 3 draw

$$\begin{array}{llll} d_1 = 155, & e_1 = 457, & D_1 = (942, 518), & E_1 = (309, 697), \\ d_3 = 464, & e_3 = 724, & D_3 = (958, 797), & E_3 = (582, 220). \end{array}$$

*Round Two.* The signing set is  $I = \{1, 3\}$ , with

$$\lambda_{I,1} = 3 \cdot (3 - 1)^{-1} = 3/2 = 485, \quad \lambda_{I,3} = 1 \cdot (1 - 3)^{-1} = -1/2 = 483$$

modulo 967. Then  $\lambda_{I,1} + \lambda_{I,3} = 1$  and  $\lambda_{I,1} sk_1 + \lambda_{I,3} sk_3 = 321 = s$ , as Lemma 2.4 requires with  $f = 1$  and with the sharing polynomial. For the message  $m$  equal to the ASCII string `toy message`, the binding factors are  $\rho_1 = 869$  and  $\rho_3 = 285$ , the group commitment is  $R = (972, 62)$  and the challenge is  $c = 628$ . The shares are  $z_1 = 419$  and  $z_3 = 717$ , and each satisfies its share equation  $[z_i]B = D_i + [\rho_i]E_i + [c\lambda_{I,i}]PK_i$ .

*Aggregation.* The aggregate is  $z = 169$ , and  $[z]B = R + [c]PK$  holds. The toy group gives no security; the example checks the algebra only.

## 4.6 Single use of nonces

**Lemma 4.13** (Reuse of a nonce pair exposes the share). *Let member  $P_i$  answer  $k$  Round Two requests,  $j = 1, \dots, k$ , with one nonce pair  $(d, e)$  and share  $sk_i$ . Its responses are*

$$z_j = d + e\rho_j + \mu_j sk_i, \quad \mu_j := \lambda_{I_j, i} c_j,$$

where  $\rho_j$ ,  $c_j$  and  $I_j$  are the binding factor of  $i$ , the challenge and the signing set of request  $j$ , all computable from public data. Each response is affine in the unknown  $(d, e, sk_i) \in \mathbb{F}_q^3$  with coefficient vector  $v_j := (1, \rho_j, \mu_j)$ . Say that  $sk_i$  is determined by  $(z_j, v_j)_{j \leq k}$  if every  $x \in \mathbb{F}_q^3$  with  $v_j \cdot x = z_j$  for all  $j$  has third coordinate  $sk_i$ . The following are equivalent:

- (a)  $sk_i$  is determined by  $(z_j, v_j)_{j \leq k}$ ;
- (b)  $(0, 0, 1)$  lies in the span of  $v_1, \dots, v_k$  (Math Guide, §“Linear combinations, span, and linear independence”, Definition “Linear combination and span”);
- (c) the points  $(\rho_j, \mu_j)$  do not all lie on one line  $\mu = \alpha + \beta\rho$  with  $\alpha, \beta \in \mathbb{F}_q$ .

In particular, three responses with linearly independent coefficient vectors determine  $sk_i$ ; two responses with distinct binding factors never do, and every value of  $sk_i$  is then consistent with them; and two responses with equal binding factors and  $\mu_1 \neq \mu_2$  determine

$$sk_i = (z_1 - z_2) / (\mu_1 - \mu_2).$$

*Proof.* (b) implies (a): if  $(0, 0, 1) = \sum_j a_j v_j$ , then every consistent  $x$  has third coordinate  $\sum_j a_j (v_j \cdot x) = \sum_j a_j z_j$ , one value, which the true unknown attains.

(a) implies (c): suppose every point lies on  $\mu = \alpha + \beta\rho$ , and put  $u := (\alpha, \beta, -1)$ . Then  $v_j \cdot u = \alpha + \beta\rho_j - \mu_j = 0$  for every  $j$ , so  $(d, e, sk_i) + \gamma u$  is consistent for every  $\gamma \in \mathbb{F}_q$ , and its third coordinate  $sk_i - \gamma$  takes every value;  $sk_i$  is not determined.

(c) implies (b): if two points have equal  $\rho$  and distinct  $\mu$ , then  $v_1 - v_2$  (for those two) is a nonzero multiple of  $(0, 0, 1)$ . Otherwise distinct points have distinct  $\rho$ . Two points with distinct  $\rho$  lie on a line  $\mu = \alpha + \beta\rho$ , so (c) forces at least three distinct points, not all on one such line; no vertical line contains them either, since their  $\rho$  are distinct. Hence some three of them are not collinear, their vectors  $(1, \rho, \mu)$  are linearly independent, and they span  $\mathbb{F}_q^3$ , which contains  $(0, 0, 1)$ .

The particular cases follow: independent vectors span  $\mathbb{F}_q^3$ ; two points with distinct  $\rho$  lie on a line  $\mu = \alpha + \beta\rho$ , and the proof of (a) implies (c) shows every value of  $sk_i$  consistent; and with  $\rho_1 = \rho_2$ ,  $z_1 - z_2 = (\mu_1 - \mu_2) sk_i$ .  $\square$

The last case is the *Crypto Guide*’s Proposition “Nonce reuse is fatal” (§“Security in the random oracle model and the forking lemma”) applied to the effective nonce  $d + e\rho$ . Whoever sees the shares and the public data can compute  $sk_i$  in the determined cases, since the shares travel without confidentiality (Definition 2.10, type (a)). RFC 9591 states the risk (Sections 1 and 7.3) and forbids the reuse (Section 5.2).

In the toy group of Example 4.12, member 1 answers three requests on the message toy message over  $I = \{1, 3\}$  with its pair  $(d, e) = (155, 457)$ , the requests differing only in the commitments of member 3. The triples  $(\rho_j, \mu_j, z_j)$  are  $(682, 413, 819)$ ,  $(403, 331, 773)$  and  $(947, 46, 955)$ . The determinant of the three coefficient vectors is  $347 \neq 0$ , and solving the system gives  $sk_1 = 132$ . From the first two responses alone, the value 133 with  $(d, e) = (802, 325)$  is also consistent, so  $sk_1$  is not determined.

**Proposition 4.14** (Replayed commitments). *A Coordinator that places in a Round Two request a commitment pair of member  $P_i$  that  $P_i$  has already used to answer a request obtains no response from*

$P_i$ , unless  $P_i$  still stores a second copy of that pair from another execution of Round One, an event whose probability over  $N$  executions is at most  $\binom{N}{2}(2^{-255} + 1/q + \delta)^2$  (Lemma 4.6). Outside that event a replay denies service only, and yields no second response under one nonce pair, the hypothesis of Lemma 4.13.

*Proof.* A member responds only for a pair that it stores and has not used, by check (2) of Construction 4.8, and it deletes a pair when it answers with it. Outside that event no second copy of the pair is stored, so a used pair is no longer stored, check (2) fails and the member aborts.  $\square$

The Coordinator may in addition track, per group key, the commitments already used (RFC 9591, Section 7.3).

## 5 Re-randomised FROST for spend authorisation

This section states the re-randomised variant of FROST that ZIP 312 specifies, in the order of the mechanism: why spend authorisation requires a shifted key; the re-randomisable threshold scheme; the derivations of the randomiser, with the bound showing that the specified derivation, fed the signature digest as its message, succeeds only with negligible probability; the signers' duty to check the signed message; the re-randomised rounds with their correctness; and the requirements and trust model of ZIP 312 that this flow forces. The ciphersuite is generic, with the randomiser hash  $H_R$  of Definition 4.1, part (f); the Pallas instance is Construction 6.1. ZIP 312 is Draft.

### 5.1 Spend authorisation under randomised validating keys

The single-signer objects are cited, not re-derived. The spend-authorisation instance of RedPallas (*Ironwood Guide*, §“The RedPallas signature scheme”, Construction “RedPallas” and Definition “Spend-authorisation and binding-signature instances”) works in the Pallas group, of prime order  $q = p_{\text{Vesta}}$ , with base

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:Orchard}, G).$$

A key pair is  $(x, X = [x]B)$ . A signature on a message  $M$  is  $\sigma = \underline{R} \parallel \text{l2LEOSP}_{256}(S)$ , with challenge  $c = H^{\circ}(\underline{R} \parallel \underline{X} \parallel M)$ , where  $H^{\circ}$  reduces modulo  $p_{\text{Vesta}}$  the BLAKE2b-512 output under the personalisation  $\text{Zcash\_RedPallasH}$ . Validation accepts if and only if the first 32 bytes decode to a point  $R$ , a non-canonical encoding decoding to  $\perp$  and being rejected;  $S < p_{\text{Vesta}}$ ; and  $[S]B = R + [c]X$ , the cofactor being one on Pallas. The nonce and challenge hashes take  $\underline{X}$ : the scheme is key-prefixed (Remark “Key prefixing” there). The message of every spend-authorisation signature is the signature digest  $\text{SigHash}$  (*Ironwood Guide*, §“Transaction digests and signatures”, Definition “Signature digest”). The deployed rules are those of the protocol specification, §“RedDSA, RedJubjub, and RedPallas” and §“Spend Authorization Signature (Sapling and Orchard)”.

Re-randomisation by a randomiser  $\alpha \in \mathbb{F}_q$  maps a key pair  $(x, X)$  to  $(x + \alpha, X + [\alpha]B)$  (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Definition “Key re-randomisation”); the identity randomiser is 0. The randomiser generator  $\text{GenRandom}$  draws  $T$  uniformly from the 80-byte strings and returns  $H^{\circ}(T)$  (*Ironwood Guide*, Definition “Randomiser generator and key re-randomisation”).

For each Action the spender draws  $\alpha := \text{GenRandom}()$  and sets

$$\text{rsk} := \text{ask} + \alpha, \quad \text{rk} := \text{ak}^{\mathbb{P}} + [\alpha]G^{\text{SpendAuth}} = [\text{rsk}]G^{\text{SpendAuth}},$$

with  $\text{ask}$  sign-normalised; the Action publishes  $\text{rk}$  and the signature under  $\text{rsk}$  on  $\text{SigHash}$  (*Ironwood Guide*, §“Randomised validating keys”, Construction “Randomised validating key”). Two

consensus rules apply:  $rk \neq O$ , and the signature is valid under  $rk$  on  $\text{SigHash}$  (protocol specification, §“Action Descriptions” and §“Spend Authorization Signature (Sapling and Orchard)”). The verifier’s key is therefore  $rk$ , which equals  $ak^{\mathbb{P}}$  only for  $\alpha = 0$ , an event of probability at most  $1/p_{\text{Vesta}} + 2^{-512}$  under  $\text{GenRandom}$  (*Ironwood Guide*, Assumption “The RedPallas hash as a random oracle”). Only these scheme-level statements are inherited. The *Ironwood Guide* constructs ask only for keys with  $\text{use\_qsk} = \text{false}$  (Remark “Scope of the key derivations”), so its key-derived theorems are not cited for threshold keys (§7.5).

For every Action of the Orchard protocol, in either pool, the single signer proceeds in the order: draw  $\alpha$ ; compute  $rk$ ; compute  $\text{SigHash}$  of the transaction, which commits to  $rk$ ; sign it under  $rsk$ . The Action proof takes  $\alpha$  as auxiliary (secret) input and  $rk$  as primary input and, being authorising data, may be computed after signing (*Ironwood Guide*, §“Construction of a transaction”, Construction “Ironwood-pool bundle”, steps “Randomised keys”, “Proof”, “Digest” and “Spend authorisation”; ZIP 229, “Motivation”). Sapling spends are outside the volume.

**Definition 5.1** (Strong unforgeability under re-randomised keys). A *re-randomisable signature scheme* is a signature scheme ( $\text{GenPrivate}$ ,  $\text{DerivePublic}$ ,  $\text{Sign}$ ,  $\text{Validate}$ ) with private-key set  $\text{Private}$  and public-key set  $\text{Public}$ , together with a randomiser set  $\mathcal{R}$ , a randomised generator  $\text{GenRandom}$  with values in  $\mathcal{R}$ , maps

$$\text{RandomizePrivate}: \mathcal{R} \times \text{Private} \rightarrow \text{Private}, \quad \text{RandomizePublic}: \mathcal{R} \times \text{Public} \rightarrow \text{Public},$$

and an identity randomiser  $\alpha_{\text{id}} \in \mathcal{R}$ , such that:

1. each  $\text{RandomizePrivate}(\alpha, \cdot)$  is injective and efficiently invertible;
2.  $\text{RandomizePrivate}(\alpha_{\text{id}}, \cdot)$  is the identity;
3. for every  $sk$ , the value  $\text{RandomizePrivate}(\alpha, sk)$  with  $\alpha \leftarrow \text{GenRandom}()$  is distributed as  $\text{GenPrivate}()$ ;
4.  $\text{RandomizePublic}(\alpha, \text{DerivePublic}(sk)) = \text{DerivePublic}(\text{RandomizePrivate}(\alpha, sk))$  for every  $\alpha$  and  $sk$ .

Let  $sk \leftarrow \text{GenPrivate}()$  and  $vk := \text{DerivePublic}(sk)$ . The signing oracle keeps a set  $\mathcal{Q}$ , initially empty; on a query  $(m, \alpha)$  it returns  $\sigma := \text{Sign}_{\text{RandomizePrivate}(\alpha, sk)}(m)$  and adds  $(m, \sigma)$  to  $\mathcal{Q}$ . The scheme is *SURK-CMA secure* if no efficient adversary, given  $vk$  and the oracle, outputs, except with negligible probability, a triple  $(m', \sigma', \alpha')$  with

$$\text{Validate}_{\text{RandomizePublic}(\alpha', vk)}(m', \sigma') = 1 \quad \text{and} \quad (m', \sigma') \notin \mathcal{Q}.$$

Freshness is on the message–signature pair; the forgery may be under any re-randomisation of  $vk$ , the identity included.

The definition is that of the protocol specification, §“Signature with Re-Randomizable Keys”, adapted there from Fleischhacker et al. (PKC 2016, Section 3) with the randomiser and key arguments swapped; the specification requires every spend-authorisation instance to meet it (§“Spend Authorization Signature (Sapling and Orchard)”). The single-signer spend-authorisation instance meets it: the *Ironwood Guide*’s Proposition “Unforgeability under re-randomisation” (“RedPallas unforgeability”), part (ii), under discrete logarithms on Pallas and the RedPallas hash as a random oracle, requires freshness only of the triple of key, message and signature, and states the implication. The threshold form of the notion is an open problem (Definition 1.1; Table 3).

**Proposition 5.2** (A fixed-key aggregate under a randomised key). *Let key material with signing key  $s$  and group key  $PK = [s]B$  be valid (Definition 2.5), over a ciphersuite meeting Assumption 4.2 with distance  $\delta$ .*

- (i) *For every signing set  $I$ ,  $\sum_{i \in I} \lambda_{I,i} sk_i = s$ . Hence every honest run of FROST (Constructions 4.4, 4.8 and 4.9) outputs a signature valid under the one key  $PK$ , whatever  $I$ , apart from the failed runs of Theorem 4.11.*
- (ii) *Fix  $\alpha \in \mathbb{F}_q$  with  $\alpha \neq 0$  and  $PK + [\alpha]B \neq O$  before the experiment. In any experiment whose parties make at most  $Q_2$  queries to  $H_2$  in total, the probability that one output pair  $(m, (R, z))$ , designated in advance (such as the aggregate of a run), is accepted by the single-party verification (Definition 4.1, part (e)) both under  $PK$  and under  $PK + [\alpha]B$  is at most*

$$(Q_2 + 2)(1/q + \delta).$$

*Proof.* (i) Lemma 2.4 with  $f$  the sharing polynomial gives the sum; Theorem 4.11 gives the rest.

(ii) Write  $PK' := PK + [\alpha]B$ . Acceptance of  $(R, z)$  on  $m$  under both keys means  $[z]B = R + [c]PK = R + [c']PK'$ , where

$$c = H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m), \quad c' = H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK') \parallel m);$$

hence  $[c]PK = [c']PK'$ , that is  $cs = c'(s + \alpha)$ . The two inputs differ, since  $PK' \neq PK$  and  $\text{ser}_{\mathbb{G}}$  is injective. Add the two queries at the inputs of the designated pair if they are missing, so that at most  $Q_2 + 2$  queries are made. Call a query with key component  $PK$  and one with key component  $PK'$  and the same  $R$  and  $m$  partners; each query has at most one partner. For each query define a target, where  $s + \alpha \neq 0$  because  $PK' \neq O$ . If  $s \neq 0$  and the partner was answered earlier, the target is the value the query's own answer must take for the equation to hold:  $cs/(s + \alpha)$  for a query with key  $PK'$ , and  $c'(s + \alpha)/s$  for a query with key  $PK$ . If  $s = 0$ , the equation reads  $c'\alpha = 0$ , and every query with key  $PK'$  has target 0. A pair valid under both keys makes some query hit its target: the later-answered query of the pair if  $s \neq 0$ , the query with key  $PK'$  if  $s = 0$ . Each target is fixed before the fresh answer is drawn, and that answer takes any fixed value with probability at most  $1/q + \delta$  (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (3)). The union bound over the at most  $Q_2 + 2$  queries gives the claim.  $\square$

The argument is that of the *Ironwood Guide*'s Remark “Key prefixing” for the unshifted response. The bound is for  $\alpha$  fixed in advance; with  $\alpha$  chosen adaptively, one query may pair with many. For FROST(Pallas, BLAKE2b-512) the per-query term  $1/q + \delta$  is below  $2^{-253.9}$ .

Spend authorisation re-randomises the validating key of every Action with a randomiser from GenRandom, which makes validating keys unlinkable (*Ironwood Guide*, Proposition “Unlinkability of randomised validating keys”); a signer bound to one key cannot follow it (ZIP 312, “Motivation”). By part (i), a plain FROST aggregate is valid under  $PK = ak^{\mathbb{P}}$ , which consensus accepts only in an Action with  $rk = ak^{\mathbb{P}}$ , that is with randomiser zero, publishing the spend validating key in every such Action. By part (ii), it is valid under  $rk$  for a nonzero randomiser only with the stated probability. The re-randomised variant shifts every participant's share, every verification share and the group key by one common scalar.

## 5.2 Re-randomisable threshold signatures

**Definition 5.3** (Re-randomisable threshold signature scheme). *A re-randomisable threshold signature scheme is a scheme of Definition 2.8 over a group  $\mathbb{G}$  of prime order  $q$  with generator  $B$ , together with a randomiser set  $\mathcal{X}$  and:*

1. a randomised algorithm GenRand with values in  $\mathcal{X}$ ;
2. an *effective-randomiser map*  $H_r: \mathcal{X} \times \{0,1\}^* \rightarrow \mathbb{F}_q$ , sending a randomiser  $\alpha$  and an *auxiliary string*  $aux$  to the *effective randomiser*  $\hat{\alpha} := H_r(\alpha, aux)$ ;
3. the *shifts*

$$\begin{aligned} \text{RandShare}(sk_i, \alpha, aux) &:= sk_i + \hat{\alpha}, & \text{RandPKShare}(PK_i, \alpha, aux) &:= PK_i + [\hat{\alpha}]B, \\ \text{RandPK}(PK, \alpha, aux) &:= PK + [\hat{\alpha}]B; \end{aligned}$$

such that the following *correctness* holds. For valid key material with signing key  $s$ , every  $(\alpha, aux)$  with  $PK + [\hat{\alpha}]B$  in the domain of  $\text{ser}_{\mathbb{G}}$  (every  $(\alpha, aux)$  when  $\text{ser}_{\mathbb{G}}$  is total), every signing set  $I$  and every message  $m$ : the shifted shares, verification shares and group key are valid key material for  $s + \hat{\alpha}$ ; and a run of the signing rounds on the shifted material over  $I$  on  $m$  in which every party is honest outputs either  $\perp$  or a signature that verification accepts under  $PK + [\hat{\alpha}]B$ , and outputs  $\perp$  with negligible probability.

The definition is that of ePrint 2024/436, Definition 5, which names the algorithms, with general RandShare, RandPKShare and RandPK and a distribution on  $\mathcal{X}$ , without a correctness relation; the correctness clause is this volume's. The auxiliary string models a contribution from the transcript of another protocol, so that a new session changes the shifted key even under a repeated  $\alpha$  (Remark 1 there).

FROST is an instance, with the identity exception of Theorem 5.11 in a suite whose element serialisation is not total, with  $H_r$  a hash into scalars, whose role the randomiser hash  $H_R$  of ZIP 312 plays, and with Rounds One and Two and aggregation run unchanged on the shifted material (ePrint 2024/436, Figure 5, read in additive notation and with the challenge input order of RFC 9591, nonce commitment, key, message, in place of the figure's key, nonce commitment, message; ZIP 312, "Re-randomizable FROST"); its rounds are constructed once, in Construction 5.10. The shifted material is valid key material: for the sharing polynomial  $f$ , the polynomial  $f + \hat{\alpha}$  has degree at most  $t - 1$ , constant term  $s + \hat{\alpha}$  and values  $sk_i + \hat{\alpha}$ , and  $[s + \hat{\alpha}]B = PK + [\hat{\alpha}]B$ ,  $[sk_i + \hat{\alpha}]B = PK_i + [\hat{\alpha}]B$ . Recombination on it is Lemma 2.4 applied to  $f + \hat{\alpha}$ , whose case  $f = 1$  gives  $\sum_{i \in I} \lambda_{I,i} = 1$ . Correctness of the instance is Theorem 5.11.

### 5.3 Randomiser generation

**Construction 5.4** (Randomiser generation of ZIP 312). After Round One, let  $m$  be the message and  $L$  the commitment list of Definition 4.3, sorted in ascending order of the nonzero identifiers. The Coordinator draws  $u$  uniformly from the  $N_s$ -byte strings,  $N_s$  the length of a serialised scalar (32 for FROST(Pallas, BLAKE2b-512)), and sets

$$\hat{\alpha} := H_R(u \parallel \text{enc}(L, m)),$$

where  $\text{enc}$  is any encoding from which  $m$  and, for each member of  $L$ , its identifier, hiding commitment and binding commitment are recovered uniquely. It sends  $\hat{\alpha}$  itself to the signers. Class (Definition 1.1): specified (ZIP 312, Draft).

The construction is that of ZIP 312, "Randomizer Generation" and "Round Two - Signature Share Generation". The ZIP's default encoding is a byte serialisation of the message and the commitment list; it permits any other encoding under which all these values are unambiguously encoded.

In spend authorisation the message is  $m = \text{SigHash}(T)$  for the transaction  $T$ , and SigHash commits to the randomised validating key of every Action: ZIP 244, "T.4c: orchard\_actions\_noncompact\_digest", field T.4c.ii, kept for version 6, and with the same structure

for the Ironwood pool, by ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests” (*Ironwood Guide*, Definition “Signature digest”). The aggregate is accepted only under  $rk$ , which for the shifted key of Definition 5.3 is  $PK + [\hat{a}]B$ . A valid threshold spend by Construction 5.4 with this message therefore needs a fixed point: a randomiser  $\hat{a}$  equal to  $H_R$  of an input containing the digest of a transaction that contains  $PK + [\hat{a}]B$ .

**Proposition 5.5** (No randomiser derived from the signature digest). *Let  $PK$  be the group key of an Orchard-protocol threshold key, with  $q = p_{\text{Vesta}}$  and  $B = G^{\text{SpendAuth}}$ . Let  $H_R = \text{ToScalar} \circ H'_R$ , where  $H'_R$  is a random oracle onto a set of size  $h$  and  $\text{ToScalar}$  maps a uniform element of that set to within  $\delta$  of uniform on  $\mathbb{F}_q$ , the one-function form of Assumption 4.2; for  $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$ ,  $h = 2^{512}$ . Let an algorithm  $A$ , given  $PK$  and making at most  $q_h$  queries to  $H'_R$ , output a transaction  $T$  with at most  $N_A$  Actions, an index  $j$ , and a string  $x = u \parallel \text{enc}(L, m)$  with  $u$  of  $N_s$  bytes, as in Construction 5.4. It wins if  $m = \text{SigHash}(T)$  and the randomised validating key of Action  $j$  of  $T$  equals  $PK + [H_R(x)]B$ . With  $Q := q_h + 1$ ,*

$$\Pr[A \text{ wins}] \leq \frac{Q}{h} + \sqrt{Q \cdot \text{Adv}^{\text{cr}}(\mathcal{B}) + Q^2 N_A (1/q + \delta)}$$

*for an algorithm  $\mathcal{B}$  of about twice the running time of  $A$ , where  $\text{Adv}^{\text{cr}}(\mathcal{B})$  is its probability of breaking the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b”.*

*Proof.* Only  $H'_R$  is modelled as a random oracle; the digest tree of  $\text{SigHash}$  is computed with BLAKE2b under other personalisations, and only its collision resistance is used. Let  $\epsilon$  be the winning probability. Define an algorithm  $\mathcal{F}$  for the *Crypto Guide*’s Theorem “General forking lemma” (§“Security in the random oracle model and the forking lemma”), with challenge set the range of  $H'_R$ , of size  $h$ , and  $Q$  challenges. On input  $PK$  and challenges  $c_1, \dots, c_Q$ , it runs  $A$ , answering its  $k$ -th distinct query to  $H'_R$  with  $c_k$ ; on the output  $(T, j, x)$  it queries  $x$  itself if  $A$  did not; it returns  $(I, (T, j, x))$ , with  $I$  the index of the query at  $x$  if  $A$  wins and  $I = 0$  otherwise. Its acceptance probability is  $\epsilon$ , so the forking algorithm succeeds with probability

$$\text{frk} \geq \epsilon \left( \frac{\epsilon}{Q} - \frac{1}{h} \right).$$

On success the two runs coincide until the  $I$ -th query is asked, so both query the same string  $x$  at index  $I$ , and both win with that  $x$ , with outputs  $T_1$  and  $T_2$  and challenges  $c_I \neq c'_I$ . The string  $x$  determines  $m$ , since  $u$  has fixed length and the encoding is unambiguous; hence  $\text{SigHash}(T_1) = \text{SigHash}(T_2) = m$ .

If the tuples of randomised validating keys of  $T_1$  and  $T_2$  differ, descent through the two digest trees from their equal roots yields two distinct pairs of personalisation and input with equal BLAKE2b output, each node hashing a uniquely decodable input (ZIP 244, “TxId Digest” and “Signature Digest”; ZIP 229, “Transaction Identifiers, Auth Digests, and Signature Digests”): at the first node whose inputs differ, the two pairs collide. Running the forking algorithm and this descent is  $\mathcal{B}$ .

Otherwise  $T_2$  contains the point  $PK + [\text{ToScalar}(c_I)]B$ , which the first run placed at index  $j$  of  $T_1$ . The second run is independent of  $c_I$ . For a fixed fork index  $i$ , each of the at most  $N_A$  keys of  $T_2$  equals  $PK + [\text{ToScalar}(c_i)]B$  only if  $\text{ToScalar}(c_i)$  takes one value fixed by the second run, which has probability at most  $1/q + \delta$  (*Math Guide*, Theorem “Properties of statistical distance”, part (3)); summing over the keys and over the  $Q$  indices,

$$\text{frk} \leq \text{Adv}^{\text{cr}}(\mathcal{B}) + Q N_A (1/q + \delta).$$

With the lower bound,  $\epsilon^2 - (Q/h)\epsilon - K \leq 0$  for  $K := Q \cdot \text{Adv}^{\text{cr}}(\mathcal{B}) + Q^2 N_A (1/q + \delta)$ , so  $\epsilon \leq (Q/h + \sqrt{(Q/h)^2 + 4K})/2 \leq Q/h + \sqrt{K}$ .  $\square$

The derivation of Construction 5.4 with the digest as message therefore yields a valid threshold spend only with negligible probability: infeasible in the random-oracle model, not impossible. For FROST (Pallas, BLAKE2b-512),  $1/q + \delta < 2^{-253.9}$  and  $Q/h = Q \cdot 2^{-512}$ . Construction 5.4 is the only specified derivation, so no specified randomiser derivation is usable for spend authorisation; the order that a usable one must respect is Proposition 6.12.

**Construction 5.6** (Seed-and-commitments randomiser). After Round One, let  $L$  be the commitment list over the signing set  $I$ . The Coordinator draws a seed  $v$  uniformly from the 32-byte strings and sends it to every member of  $I$  over a confidential and authenticated channel (Definition 2.10, type (b)). The Coordinator and every member of  $I$  compute

$$\hat{\alpha} := H_R(v \parallel \text{enc}(L)),$$

with  $\text{enc}(L)$  the encoding of Definition 4.3; the message is not an input. Two rules are part of the construction and are this volume's own:  $H_R$  in both computations, the Coordinator's and every member's, so that all parties obtain one scalar; and the confidential and authenticated channel for the seed, which the proof of Proposition 7.24, step (iv), shows necessary. Class (Definition 1.1): designed but unspecified.

The interface is that of ePrint 2024/436, Figure 5 and Remark 1, with  $(\alpha, \text{aux}) = (v, \text{enc}(L))$  and every signer applying  $H_R$ ; the encoding of the commitment list is that of RFC 9591, Section 4.3. A seed flow is published only in the revision of ZIP 312 proposed in pull request 895, and the merged ZIP 312 has none. That hashing the signer-chosen commitments hedges a faulty generator at the Coordinator is asserted informally in ePrint 2024/436, Section 7, and proved nowhere; the Coordinator sees  $L$  before it chooses the seed, and it remains trusted for unlinkability (Definition 5.14).

**Lemma 5.7** (Distance of the randomiser from uniform). *Under Assumption 4.2, let an algorithm, the observer, make at most  $q_h$  queries to  $H_R$  and, at one point, choose a byte string  $w$  and receive  $\hat{\alpha} := H_R(u \parallel w)$ , for  $u$  uniform on the  $N_u$ -byte strings and independent of all else. The pair formed by the observer's view and  $\hat{\alpha}$  is within statistical distance*

$$\delta + q_h \cdot 2^{-8N_u}$$

*of the pair in which  $\hat{\alpha}$  is replaced by a uniform element of  $\mathbb{F}_q$  independent of all else. For Construction 5.4,  $N_u = N_s$  and  $w = \text{enc}(L, m)$ ; for Construction 5.6,  $N_u = 32$  and  $w = \text{enc}(L)$ ; with  $N_u = 32$  the bound is  $\delta + q_h \cdot 2^{-256}$ .*

*Proof.* The proof is that of Lemma 4.5 with  $H_R$  in place of  $H_3$  and  $N_u$  bytes in place of 32. The observer queries  $H_R$  at  $u \parallel w$  with probability at most  $q_h \cdot 2^{-8N_u}$ , by the union bound over its queries, each having prefix  $u$  with probability  $2^{-8N_u}$  in the experiment in which the answer used for  $\hat{\alpha}$  is resampled. Outside that event the two experiments coincide, and the resampled answer is within  $\delta$  of uniform; the triangle inequality (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”) gives the bound.  $\square$

At  $q_h = 2^{64}$  and  $N_u = 32$  the bound is below  $2^{-191.99}$ . The randomiser is thus close to uniform, not exactly uniform as ZIP 312, “Rationale”, states.

**Remark 5.8** (Effective randomiser). The randomiser of ZIP 312 is the effective randomiser  $\hat{\alpha}$  of ePrint 2024/436, not its  $\alpha$ . The correspondence carries correctness (Theorem 5.11), not the paper's unforgeability game, whose signing oracle applies  $H_r$  itself to an adversarial  $(\alpha, \text{aux})$  (Definition 6 and Figure 4, with RandShare and RandPK of Figure 5). Only Construction 5.6 presents

the paper’s interface. Wherever the Coordinator sends  $\hat{a}$  itself, as in the wire form of Construction 5.4 or when it forwards a randomiser chosen elsewhere, it chooses the effective randomiser outright and the signers use it unchecked; the paper’s game does not model this. This remark is the volume’s one statement of that fact; later statements cite it.

## 5.4 Authorisation of the signed message

**Definition 5.9** (Message check). In spend authorisation the signed message is  $m = \text{SigHash}(T)$  for a transaction  $T$  (*Ironwood Guide*, Definition “Signature digest”), which alone does not show a signer what it authorises. With  $m$  the Coordinator sends *transaction data*  $D_T$ , further data of  $T$ , over the confidential and authenticated channel of Definition 2.10, type (b). A signer *performs the message check* if, before returning a share, it verifies that  $m$  is the signature digest determined by  $D_T$ , or computes  $m$  from  $D_T$  itself. A signature share counts as an *approval* of the pair  $(\text{rk}, m)$ , with  $\text{rk}$  the shifted group key of its request, only if its signer performed the message check.

ZIP 312, “Round Two - Signature Share Generation”, makes the check a MUST and names as possible data the transaction itself, openings of value commitments and decryptions of note ciphertexts; RFC 9591, Section 7.7 (“Input Message Validation”), recommends such input validation. The honest participants of the security games of §7 perform the check by definition, and their returned shares are the approvals the games count. A signer that omits it authorises whatever the transaction builder chose. Class (Definition 1.1): the obligation is specified; the mechanism by which a signer obtains and checks  $D_T$  is outside ZIP 312 and designed but unspecified.

## 5.5 Re-randomised signing, aggregation and share verification

**Construction 5.10** (Re-randomised signing of ZIP 312). The key material is valid (Definition 2.5); the ciphersuite has the randomiser hash  $H_R$  (Definition 4.1, part (f)).

- (1) *Round One*. Each participant runs Construction 4.4 unchanged and sends its commitments over an authenticated channel (Definition 2.10, type (a)).
- (2) *Randomiser*. The Coordinator chooses the signing set  $I$  and, for each  $i \in I$ , one unused commitment pair, forms the commitment list  $L$ , and obtains  $\hat{a}$  by Construction 5.4 or Construction 5.6, fresh for this run. It sends to each  $P_i, i \in I$ , the tuple  $(m, L, \hat{a}, D_T)$ , or  $(m, L, v, D_T)$  with the seed  $v$  of Construction 5.6, where  $D_T$  is the data of Definition 5.9, over a confidential and authenticated channel (Definition 2.10, type (b)); RFC 9591 requires authentication only.
- (3) *Round Two*. Member  $P_i$  performs the checks of Construction 4.8, its check (1) being the message check of Definition 5.9; in the seed variant it computes  $\hat{a} := H_R(v \parallel \text{enc}(L))$ . It computes  $PK' := PK + [\hat{a}]B$  from its stored group key and runs Construction 4.8 with  $sk_i + \hat{a}$  in place of  $sk_i$  and  $PK'$  in place of  $PK$ . The shifted group key  $PK'$ , not  $PK$ , is the key serialised into  $H_1$  for the binding factors and into  $H_2$  for the challenge (Definition 4.7). Its share is

$$z_i := d_i + e_i \rho_i + \lambda_{i,i} (sk_i + \hat{a}) c,$$

returned as in RFC 9591 over an authenticated, not confidential, channel (Definition 2.10, type (a)).

- (4) *Aggregation.* The Coordinator runs Construction 4.9 with  $PK'$  in place of  $PK$ , and verifies the aggregate under  $PK'$ .
- (5) *Share verification.* The share equation of Construction 4.9 is checked with  $PK_i + [\hat{\alpha}]B$  in place of  $PK_i$  and with the shifted group key:

$$[z_i]B = R_i + [c\lambda_{I,i}](PK_i + [\hat{\alpha}]B).$$

Every signing run uses a fresh randomiser, generated in its own Round Two: a randomiser used for two runs of one key gives both the same shifted key and links them. The output  $(R, z)$  is a signature under  $rk := PK'$ . Class (Definition 1.1): specified (ZIP 312, Draft), the seed variant designed but unspecified.

The construction is that of ZIP 312, “Round One - Commitment”, “Round Two - Signature Share Generation” and “Signature Share Verification and Aggregation”, over RFC 9591, Sections 5.1 to 5.4: the randomised signing, aggregation and share-verification functions are the regular ones with the share, the verification share and the group key shifted by  $\hat{\alpha}$  and by  $[\hat{\alpha}]B$ . Round One keeps the unshifted share in the nonce hash, the randomiser not yet existing. The keys  $PK$  and  $PK_i$  are those stored from key generation, never taken from the messages of the run (Construction 4.9). What the returned shares reveal to a party that sees them is Proposition 7.25.

**Theorem 5.11** (Correctness of re-randomised FROST). *Let the key material be valid, with signing key  $s$  and group key  $PK$ ; let  $\hat{\alpha} \in \mathbb{F}_q$ ,  $I$  a signing set and  $m$  a message that every member of  $I$  accepts. Outside the exceptions below, a run of Construction 5.10 in which every party is honest outputs  $(R, z)$  with*

$$R = [r]B, \quad r := \sum_{i \in I} (d_i + e_i \rho_i), \quad z = r + c(s + \hat{\alpha}),$$

*which the single-party verification accepts on  $m$  under  $PK + [\hat{\alpha}]B$ , and every share satisfies its shifted share equation. The exceptions are those of Theorem 4.11 for the shifted material, together with the identity case of the shifted group key: in a suite whose element serialisation is not total, the run fails when  $PK + [\hat{\alpha}]B = O$ .*

*Proof.* By Definition 5.3 and the paragraph following it, the shifted shares  $sk_j + \hat{\alpha}$ , verification shares  $PK_j + [\hat{\alpha}]B$  and group key  $PK + [\hat{\alpha}]B$  are valid key material for  $s + \hat{\alpha}$ , with sharing polynomial  $f + \hat{\alpha}$ . Steps (3) to (5) of Construction 5.10 are Constructions 4.8 and 4.9 run on that key material. Round One enters those constructions only through the nonces, each an answer of  $H_3$  whichever share enters its input. Theorem 4.11 applied to the shifted material gives  $R = [r]B, z = r + c(s + \hat{\alpha})$  and acceptance under  $PK + [\hat{\alpha}]B$ , outside its cases; and Proposition 4.10, part (ii), applied to it gives the shifted share equations. Both use only validity of the key material. When  $PK + [\hat{\alpha}]B = O$  in a suite whose element serialisation is not total,  $\text{ser}_{\mathbb{G}}(PK')$  is undefined, and so are the binding factors of Definition 4.7; the run fails.  $\square$

ZIP 312, “Rationale”, makes the same argument, and ePrint 2024/436, Equation (2), displays it; the ZIP cites, rather than proves, the coefficient sum one on which it rests, which Lemma 2.4 proves.

**Example 5.12** (The two-of-three run, re-randomised). Every number below is output by the volume’s toy script. The toy curve, the key material, the encodings, the message toy message and the signing set  $I = \{1, 3\}$  are those of Example 4.12. Members 1 and 3 draw fresh Round One nonces: reusing that example’s nonces would reuse a nonce pair (Lemma 4.13). The randomiser is that of Construction 5.6, with a fixed seed and  $H_R$  BLAKE2b-512 under the personalisation of FROST(Pallas, BLAKE2b-512) reduced modulo 967:  $\hat{\alpha} = 565$ . The shifted group key and shares

are

$$PK + [565]B = (268, 139), \quad sk_1 + \hat{a} = 697, \quad sk_3 + \hat{a} = 319,$$

and  $\lambda_{I,1} + \lambda_{I,3} = 1$  modulo 967. With the shifted key in  $H_1$  and  $H_2$ , the binding factors are  $\rho_1 = 480$  and  $\rho_3 = 303$ , the group commitment is  $R = (302, 757)$  and the challenge is  $c = 151$ . The shares are  $z_1 = 151$  and  $z_3 = 652$ , and each satisfies its shifted share equation against  $PK_i + [\hat{a}]B$ . The aggregate is  $z = 803 = r + c(s + \hat{a})$ , and  $[z]B = R + [c](PK + [\hat{a}]B)$  holds.

## 5.6 Trust model of ZIP 312

**Remark 5.13** (Requirements of ZIP 312). ZIP 312, “Requirements”, states three requirements, in lower case, although its “Terminology” invokes the key words of RFC 2119:

- (a) all signatures generated by following the ZIP must verify as Sapling or Orchard spend-authorisation signatures under the appropriate validating key, which Proposition 6.3 proves for the Pallas suite, the Sapling suite being outside the volume;
- (b) they should meet the security criteria of the protocol specification for signatures with re-randomisable keys, Definition 5.1, whose threshold form is an open problem (Table 3);
- (c) the threat model must be taken into account, which Definition 5.14 makes exact.

**Definition 5.14** (Threat model of ZIP 312). The threat model of ZIP 312 consists of two clauses.

- (i) The Coordinator is trusted with the privacy of the transaction, unlinkability included. A rogue Coordinator can break both, but “should not be able to create signed transactions without the approval of”  $t$  participants, “as specified in FROST”; the ZIP’s MIN\_PARTICIPANTS is  $t$ . An approval is a share returned after the message check (Definition 5.9).
- (ii) Every key-share holder is likewise trusted with the privacy of the transaction and with unlinkability.

The clauses are those of ZIP 312, “Threat Model”. The should of clause (i) is lower case, within a threat model that the Requirements say must be taken into account; it is not a must not. ePrint 2024/436, Section 5.1, states the same split: the party choosing the randomiser is trusted with privacy, not with security. The privacy trust is forced by the flow: the Coordinator generates  $\hat{a}$ , every signer receives it, or the seed with  $L$ , and whoever holds it computes  $PK = rk - [\hat{a}]B$  and links the run to the key (the proof of Proposition 7.24, step (iv)). The unforgeability clause has no proof, as §7.5 records (Remark 7.22). In RFC 9591, Section 7, the Coordinator holds no private information; trust in it for privacy is what re-randomisation adds.

ZIP 312, “Non-requirements”, excludes four things: removal of the Coordinator role (RFC 9591, Section 7.5, “Removing the Coordinator Role”); prevention of key-share holders from linking a signing run to its transaction on the chain; a key-generation procedure, for which it refers to RFC 9591; and network privacy.

**Remark 5.15** (Unlinkability as ZIP 312 states it). ZIP 312, “Terminology”, defines unlinkability as statistical independence of signatures from the signers’ long-term keys, ensuring, “for perfectly uniform generation of Randomizers and no leakage of metadata”, that it is “impossible to determine whether two transactions were generated by the same party”. The statement is informal: it names no observer and no view. Definition 7.23 makes it exact against a specified observer, and Lemma 5.7 replaces perfect uniformity.

## 6 The aggregate as a spend-authorisation signature

This section instantiates the re-randomised protocol of §5 over the ciphersuite FROST(Pallas, BLAKE2b-512) and composes it with the Orchard protocol. After the ciphersuite comes the section’s main result: the aggregate is an ordinary RedPallas spend-authorisation signature under the shifted group key, and consensus accepts it for an Action exactly when the effective randomiser is the Action’s randomiser. There follow the sign normalisation that makes a threshold group key a spend validating key; the key tree of ZIP 2005 for an Orchard-protocol threshold key, with the lemma that its common secrets are simulable; the order of randomiser, randomised validating key, signature digest and proof that the main result forces; and the threshold construction of an Action in that order. Throughout,  $q = p_{\text{Vesta}}$ ,  $B = G^{\text{SpendAuth}}$ , the effective randomiser is  $\hat{a}$  (Definition 5.3) and the randomiser of an Action is  $\alpha$ .

### 6.1 The ciphersuite FROST(Pallas, BLAKE2b-512)

For a point  $P$  of the Pallas group write  $\underline{P} := \text{LEBS2OSP}_{256}(P^*)$ , the 32-byte form of the star encoding (*Ironwood Guide*, §“Fields, groups, and encodings”, Definition “Star encoding”), and  $\text{abst}_{\mathbb{P}}$  for its partial inverse.

**Construction 6.1** (FROST(Pallas, BLAKE2b-512)). The ciphersuite FROST(Pallas, BLAKE2b-512) is the following instance of Definition 4.1, for re-randomised signing.

- (a) *Group*. The Pallas group  $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ , of prime order  $q = p_{\text{Vesta}}$  (ZIP 312’s  $r_{\mathbb{P}}$ ) and cofactor 1, with identity  $O$  and generator

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:Orchard}, G)$$

(*Ironwood Guide*, §“GroupHash, domain separation, and nothing-up-my-sleeve generators”; Definition “Spend-authorisation and binding-signature instances”), ZIP 312’s  $G^{\text{Orchard}}$ . A random scalar is uniform on  $\{0, \dots, p_{\text{Vesta}} - 1\}$ .

- (b) *Elements*.  $N_e = 32$  and  $\text{ser}_{\mathbb{G}}(P) := \underline{P}$ , which is total:  $O^* = \text{LE}_{256}(0)$ . The deserialisation maps a 32-byte string  $b$  to  $\text{abst}_{\mathbb{P}}(\text{LEBS2OSP}_{256}^{-1}(b))$ , and fails if that value is  $\perp$  and, in addition, if it is  $O$ . The function  $\text{abst}_{\mathbb{P}}$  itself decodes  $\text{LE}_{256}(0)$  to  $O$  (step 2 of Definition “Star encoding”); the rejection of the identity is the suite’s, not  $\text{abst}_{\mathbb{P}}$ ’s.
- (c) *Scalars*.  $N_s = 32$ ,  $\text{ser}_{\mathbb{F}_q}(k) := \text{l2LEOSP}_{256}(k)$ , and  $\text{deser}_{\mathbb{F}_q}$  fails on a 32-byte string whose little-endian integer is not below  $p_{\text{Vesta}}$ .
- (d) *Hashes*. BLAKE2b-512 with a 16-byte personalisation,  $N_h = 64$ , the roles  $H_1$  to  $H_5$  as in Table 2.
- (e) *Signature encoding and verification* as in Definition 4.1, part (e), with these data.
- (f) *Randomiser hash*. The role  $H_R$  of Table 2.

Class (Definition 1.1): specified (ZIP 312, Draft).

The data are those of ZIP 312, “FROST(Pallas, BLAKE2b-512)”, which takes the group, the order and the encoding  $\text{repr}_{\mathbb{P}}$  from the protocol specification, §“Pallas and Vesta”, and the base point from §“Spend Authorization Signature (Sapling and Orchard)”. The generic element serialisation of RFC 9591, Section 3.1, fails on the identity; this one does not. Three consequences for Theorem 4.11 follow. Its case (2),  $R = O$ , does not arise, the serialisation being total. Its case (1), a zero nonce, remains: a commitment  $D_i$  or  $E_i$  equal to  $O$  is serialised in Round One

| Role                             | Personalisation  | Output         |
|----------------------------------|------------------|----------------|
| $H_1$ , binding factor           | FROST_RedPallasR | ToScalar       |
| $H_2$ , challenge                | Zcash_RedPallasH | ToScalar       |
| $H_3$ , nonce                    | FROST_RedPallasN | ToScalar       |
| $H_4$ , message pre-hash         | FROST_RedPallasM | 64-byte digest |
| $H_5$ , commitment-list pre-hash | FROST_RedPallasC | 64-byte digest |
| $H_R$ , randomiser               | FROST_RedPallasA | ToScalar       |

Table 2: Hash functions of FROST(Pallas, BLAKE2b-512). Each role maps  $x$  to  $\text{BLAKE2b-512}(\text{pers}, x)$ , followed where stated by  $\text{ToScalar}(y) = \text{LEOS2IP}_{512}(y) \bmod p_{\text{Vesta}}$  (*Ironwood Guide*, §“Pseudorandom functions and field reductions”, Definition “Field reductions”). The challenge hash  $H_2$  is, as a function, the RedPallas hash  $H^\circledast(z) = \text{ToScalar}(\text{BLAKE2b-512}(\text{Zcash\_RedPallasH}, z))$  (*Ironwood Guide*, §“The RedPallas signature scheme”, Definition “RedPallas hash”), so the FROST challenge is the challenge that RedPallas validation recomputes; the other five personalisations carry the prefix FROST\_. Source: ZIP 312, “FROST(Pallas, BLAKE2b-512)”.

and rejected by  $\text{deser}_G$  in check (3) of Round Two. The only identity condition left on the output is the consensus rule  $\text{rk} \neq O$  (Corollary 6.4).

The reduction ToScalar is ZIP 312’s “interpreting the 64 bytes as a little-endian integer, and reducing the resulting integer modulo  $G.\text{Order}()$ ”. ZIP 312 records the equality of  $H_2$  and  $H^\circledast$  (protocol specification, §“RedDSA, RedJubjub, and RedPallas”).

**Proposition 6.2** (Random-oracle model of the Pallas ciphersuite). *Assume Assumption 4.2 in its form for a suite built from one hash function under distinct personalisations: the map  $(\text{pers}, x) \mapsto \text{BLAKE2b-512}(\text{pers}, x)$  is modelled as a random oracle. Its restriction to  $\text{pers} = \text{Zcash\_RedPallasH}$  is the Ironwood Guide’s Assumption “The RedPallas hash as a random oracle”. No further assumption is introduced. Then:*

- (i) *the hashes  $H_1, H_2, H_3, H_4, H_5$  and  $H_R$  are independent random oracles;*
- (ii) *each value of  $H_1, H_2, H_3$  and  $H_R$  at a new input is within statistical distance*

$$\delta_{512} := p_{\text{Vesta}}/2^{512} < 2^{-257}$$

*of a uniform scalar, so the suite meets Assumption 4.2 with  $\delta = \delta_{512}$ ;*

- (iii) *the hashes  $H_4$  and  $H_5$  are random oracles onto the 64-byte strings, so  $Q$  queries find a collision of either with probability at most  $Q^2/2^{513}$ ;*
- (iv) *the independence of  $H_{\text{dkg}}$  that Assumptions 3.5 and 4.2 require holds for every instantiation of  $H_{\text{dkg}}$  by BLAKE2b-512 under a 16-byte personalisation distinct from the six of Table 2. No such instantiation is specified (ZIP 312, “Key Generation”).*

*Proof.* (i) The six personalisations are pairwise distinct and all 16 bytes long, hence prefix-free; *Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”, gives independence. (ii) A value at a new input is ToScalar of a uniform 64-byte string, whose distance from uniform is at most  $p_{\text{Vesta}}/2^{512}$  by *Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with  $L = 512$ , the bound that the Ironwood assumption records; and  $p_{\text{Vesta}}/2^{512} = 2^{-258}(1 + e)$  with  $0 < e < 2^{-128}$ . (iii) Birthday bound, *Math Guide*, §“The union bound and a birthday calculation”, Proposition “Birthday bound”:  $\binom{Q}{2}2^{-512} \leq Q^2/2^{513}$ . (iv) Part (i) applied to seven distinct

personalisations. □

At  $Q = 2^{64}$  the collision bound of part (iii) is  $2^{-385}$ .

## 6.2 Validity of the aggregate

**Proposition 6.3** (Aggregates are spend-authorisation signatures). *Let valid key material (Definition 2.5) with group key  $Y$  be given over  $\text{FROST}(\text{Pallas}, \text{BLAKE2b-512})$ , let  $\hat{\alpha} \in \mathbb{F}_q$  be an effective randomiser and  $m$  a message that every member of the signing set accepts (Definition 5.9). Every run of Construction 5.10 in which every party is honest either fails, in the zero-nonce case of Theorem 4.11, or outputs  $(R, z)$  whose encoding*

$$\sigma := \text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$$

*(RFC 9591, Appendix A) is accepted by the validation of the spend-authorisation instance of Red-Pallas (Ironwood Guide, Construction “RedPallas”, part 3, with base  $G^{\text{SpendAuth}}$ ) under  $X := Y + [\hat{\alpha}]G^{\text{SpendAuth}}$  on  $M := m$ . No completed honest run yields an aggregate that validation rejects.*

*Proof.* Theorem 5.11 gives  $[z]B = R + [c]X$  with  $c = H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(X) \parallel m)$  (RFC 9591, Section 4.6), its only exception in this suite being the zero nonce, since  $\text{ser}_{\mathbb{G}}$  is total. Validation of  $\sigma$  under  $X$  on  $m$  proceeds as follows.

1. The first 32 bytes are  $\bar{R} = \underline{R}$ , and  $\text{abst}_{\mathbb{P}}(R^*) = R$  because the star encoding is canonical and total,  $R = O$  included; so the decoded point is  $R \neq \perp$ .
2. The last 32 bytes give  $S = z$ , an integer below  $p_{\text{Vesta}}$  since  $z \in \mathbb{F}_q$ .
3. Validation computes  $H^{\odot}(\bar{R} \parallel \underline{X} \parallel m)$ , where  $\underline{X} = \text{ser}_{\mathbb{G}}(X)$ : the inputs of the FROST challenge in the same order. Since  $H_2 = H^{\odot}$  (Table 2), it recomputes  $c$ .
4. The equation  $[S]B = R + [c]X$ , whose cofactor is 1 on Pallas, is the FROST equation of RFC 9591, Appendix B, which holds.

How  $R$  was produced does not enter validation. These are the conditions that ZIP 312, “Rationale”, lists. □

ZIP 312, “FROST(Pallas, BLAKE2b-512)”, states that the suite is meant to produce signatures “indistinguishable from RedPallas Orchard Spend Authorization Signatures”. This volume reads that phrase as the statement of format and validation above; the distributional statement, for randomisers of Construction 5.6, is Proposition 7.24.

**Corollary 6.4** (The effective randomiser is the Action’s randomiser). *Let  $Y = \text{ak}^{\mathbb{P}} = [s]G^{\text{SpendAuth}}$  be the spend validating key of the key of the note that an Action consumes, and let the Action carry  $\text{rk} = \text{ak}^{\mathbb{P}} + [\alpha]G^{\text{SpendAuth}}$  (Ironwood Guide, §“Randomised validating keys”, Construction “Randomised validating key”). Then*

$$Y + [\hat{\alpha}]G^{\text{SpendAuth}} = \text{rk} \iff \hat{\alpha} = \alpha \text{ in } \mathbb{F}_{p_{\text{Vesta}}}.$$

*Consequently:*

- (i) *if  $\hat{\alpha} = \alpha$ , the aggregate of Proposition 6.3 on the signature digest is valid under  $\text{rk}$ ;*
- (ii) *if  $\hat{\alpha} \neq \alpha$ , with  $\alpha$  and  $\hat{\alpha}$  fixed before the queries to  $H_2$  whose inputs contain either key, the aggregate is valid under  $\text{rk}$  with probability at most  $(Q_2 + 2)(1/p_{\text{Vesta}} + \delta_{512})$  in an experiment with at most  $Q_2$  queries to  $H_2$  (Proposition 5.2, applied to the key  $Y + [\hat{\alpha}]G^{\text{SpendAuth}}$  and the shift  $\alpha - \hat{\alpha} \neq 0$ );*

- (iii) the consensus rule  $rk \neq O$  (protocol specification, §“Action Descriptions”; Ironwood Guide, Construction “Verification”, part (1)) fails exactly when  $\alpha = -s$ . For  $\alpha = \hat{\alpha}$  the output of Construction 5.6, fixed after the key, this has probability at most

$$1/p_{\text{Vesta}} + \delta_{512} + q_h \cdot 2^{-256},$$

with  $q_h$  the number of queries to  $H_R$  made by the party that fixed the key (Lemma 5.7).

Hence, up to these probabilities, consensus accepts the aggregate as the Action’s spend-authorisation signature if and only if the effective randomiser equals the Action’s  $\alpha$ .

*Proof.* The map  $k \mapsto [k]G^{\text{SpendAuth}}$  is injective on  $\mathbb{F}_{p_{\text{Vesta}}}$ , since  $G^{\text{SpendAuth}}$  generates a group of prime order  $p_{\text{Vesta}}$ ; and  $Y + [\hat{\alpha}]G^{\text{SpendAuth}} - rk = [\hat{\alpha} - \alpha]G^{\text{SpendAuth}}$ . Part (i) is Proposition 6.3 with  $X = rk$ . Part (ii) is the cited proposition, whose hypotheses hold: the shifted material is valid key material (Definition 5.3 and the paragraph following it), and  $rk \neq O$  in an accepted Action. For (iii),  $rk = [s + \alpha]G^{\text{SpendAuth}}$  is  $O$  exactly when  $\alpha = -s$ ; a value within statistical distance  $\epsilon$  of uniform takes a fixed value with probability at most  $1/p_{\text{Vesta}} + \epsilon$  (*Math Guide*, Theorem “Properties of statistical distance”, part (3)), and Lemma 5.7 gives  $\epsilon = \delta_{512} + q_h \cdot 2^{-256}$ .  $\square$

For FROST(Pallas, BLAKE2b-512) the term  $1/p_{\text{Vesta}} + \delta_{512}$  of parts (ii) and (iii) is below  $2^{-253.9}$ .

**Remark 6.5** (Consensus is unchanged). No consensus rule refers to FROST or to a threshold key. The rules that bear on spend authorisation,  $rk \neq O$  and validity of the spend-authorisation signature under  $rk$  on the signature digest (*Ironwood Guide*, §“Verification of a transaction”, Construction “Verification”), read only  $rk$ , the 64-byte signature and the digest; validation does not depend on how  $R$  was produced (ZIP 312, “Rationale”). The first requirement of ZIP 312, “Requirements”, that every signature produced by following the ZIP be verified successfully as an Orchard spend-authorisation signature under the appropriate validating key, is met by Proposition 6.3. The remark makes no claim about the security of the signers, which is the subject of §7.

### 6.3 Sign normalisation of a shared key

The spend validating key  $ak^{\mathbb{P}}$  of an Orchard-protocol key has even  $y$ -coordinate: the last bit, the  $\tilde{y}$  bit, of  $\text{repr}_{\mathbb{P}}(ak^{\mathbb{P}})$  is 0. For keys with `use_qsk = false` the protocol specification, §“Orchard Key Components”, produces this by negating `ask` when the bit is 1 (*Ironwood Guide*, §“The spending key and the spend-side secrets”, Construction “Spend validating key and sign normalisation”). For keys with `use_qsk = true`, ZIP 2005’s replacement text for that section (“Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”) requires  $ak^{\mathbb{P}} \in \mathbb{P}^*$  to be obtained “by any suitably secure method that ensures the last bit of  $\text{repr}_{\mathbb{P}}(ak^{\mathbb{P}})$  is 0”; the published specification does not yet carry that text. The requirement is one of key derivation, not a consensus rule: the Action statement admits either sign (protocol specification, §“Action Statement (Orchard)”; *Ironwood Guide*, Remark “Sign of  $ak^{\mathbb{P}}$ ”). A full viewing key records only  $ak = \text{Extract}_{\mathbb{P}}(ak^{\mathbb{P}})$  (*Ironwood Guide*, §“Viewing keys”, Definition “Full viewing key”), from which a transaction builder recovers the point by the even- $y$  lift. For a group key  $PK$  of odd  $y$  the lift yields  $-PK$ ; the builder forms  $rk = -PK + [\alpha]G^{\text{SpendAuth}}$ , which with  $\alpha = \hat{\alpha}$  differs from  $PK + [\hat{\alpha}]G^{\text{SpendAuth}}$  by  $[2]PK \neq O$ , and by Proposition 5.2, part (ii), applied to the key  $PK + [\hat{\alpha}]G^{\text{SpendAuth}}$  and the shift  $-2s \neq 0$ , the aggregate is accepted under  $rk$  with probability at most  $(Q_2 + 2)(1/p_{\text{Vesta}} + \delta_{512})$ . The randomised key  $rk$  carries no parity condition. Class (Definition 1.1): specified (ZIP 2005, Proposed, taking effect with NU6.3, which ZIP 258, Draft, deploys; ZIP 258, “ZIP 2005 activation”).

**Lemma 6.6** (Negation of key material). *Let shares  $sk_j$ , coefficient commitments  $A_0, \dots, A_{t-1}$ , verification shares  $PK_j$  and group key  $PK$  be valid key material for the signing key  $s$  with sharing polynomial  $f$  (Definition 2.5). Then  $-sk_j$ ,  $-A_k$ ,  $-PK_j$  and  $-PK$  are valid key material for  $-s$ , with sharing polynomial  $-f$ . If  $PK \neq O$ , the last bit of  $\text{repr}_{\mathbb{P}}(-PK)$  differs from that of  $\text{repr}_{\mathbb{P}}(PK)$ , and  $\text{Extract}_{\mathbb{P}}(-PK) = \text{Extract}_{\mathbb{P}}(PK)$ .*

*Proof.* The polynomial  $-f$  has degree at most  $t - 1$ , coefficients  $-a_k$  and constant term  $-s$ ; and  $-sk_j = (-f)(j)$ ,  $[-a_k]B = -A_k$ ,  $[-sk_j]B = -PK_j$  and  $[-s]B = -PK$ , which are the conditions of Definition 2.5. For  $PK = (x, y) \neq O$ ,  $-PK = (x, p_{\text{Pallas}} - y)$ . Here  $y \neq 0$ , since a point with  $y = 0$  has order 2 and the group has odd order; and  $p_{\text{Pallas}}$  is odd, so  $y$  and  $p_{\text{Pallas}} - y$  have opposite parities. The last bit of the star encoding is  $y \bmod 2$  (Ironwood Guide, Definition “Star encoding”). The extracted coordinate is  $x$  for both points (Ironwood Guide, §“Fields, groups, and encodings”, Lemma “Coordinate extraction identifies opposite points”).  $\square$

On the toy curve of Example 4.12, the negated material has  $-s = 646$  and shares 835, 57, 246; the group key  $PK = (526, 125)$  becomes  $-PK = (526, 884)$ , with the same  $x$ -coordinate and the parity of  $y$  changed from 1 to 0.

**Construction 6.7** (Sign normalisation of key material). *Input:* valid key material output by Construction 3.2 or by Construction 3.6. Every party computes  $PK$  from public data (Lemma 2.6).

1. If  $PK = O$ , key generation is repeated: ZIP 2005 requires  $\text{ak}^{\mathbb{P}} \in \mathbb{P}^*$ , and neither key generation excludes  $s = 0$ .
2. Otherwise, if the last bit of  $\text{repr}_{\mathbb{P}}(PK)$  is 1, every participant  $P_j$  replaces its share  $sk_j$  by  $-sk_j$ , and every party replaces each  $A_k$ , each  $PK_j$  and  $PK$  by its negative.

The test reads public data only, so all parties decide identically without interaction. *Output:* the resulting key material and  $\text{ak}^{\mathbb{P}} := PK$ . Class (Definition 1.1): the requirement is specified (ZIP 2005, Proposed); the method is this volume’s.

*Correctness.* Validity of the output and even  $y$  of  $\text{ak}^{\mathbb{P}}$  follow from Lemma 6.6;  $PK \neq O$  on output by step (1).  $\square$

## 6.4 The key tree of an Orchard-protocol FROST key

The *Ironwood Guide* constructs only keys with `use_qsk = false` (Remark “Scope of the key derivations”). Write `PRFexpand` for the *Ironwood Guide*’s Construction “Expansion function” and `ToBase`, `ToScalar` for its Definition “Field reductions” (“Pseudorandom functions and field reductions”); `truncate32` returns the first 32 bytes of its input.

**Construction 6.8** (Orchard-protocol threshold key). This is the branch `use_qsk = true` of ZIP 2005’s replacement for §4.2.3 “Orchard Key Components”.

1. Run a key generation (§3) and Construction 6.7; let  $\text{ak}^{\mathbb{P}} := PK$  and  $\text{ak} := \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$ , non-zero since  $\text{ak}^{\mathbb{P}} \neq O$  (Ironwood Guide, Lemma “Extract vanishes only at the identity”).
2. The participants privately agree on a spending key  $sk$ , a 32-byte string drawn uniformly and independently of the key material.

## 3. Every participant computes

$$\begin{aligned}
nk &:= H^{nk}(sk) = \text{ToBase}(\text{PRFexpand}_{sk}([0x07])), \\
qsk &:= H^{qsk}(sk) = \text{truncate}_{32}(\text{PRFexpand}_{sk}([0x0C])), \\
qk &:= H^{qk}(qsk) \\
&= \text{BLAKE3.derive\_key}(\text{Zcash ZIP 2005 qk-derivation v1}, qsk, 32), \\
r_{ivk} &:= H_{qk}^{r_{ivk\_ext}}(ak, nk) \\
&= \text{ToScalar}(\text{PRFexpand}_{qk}([0x0D] \parallel \text{I2LEOSP}_{256}(ak) \parallel \text{I2LEOSP}_{256}(nk))),
\end{aligned}$$

where  $H^{qk}$  is the key-derivation mode of the BLAKE3 hash function with the displayed context string and a 32-byte output, ZIP 2005's derivation, of which no property is used below.

4. The remainder of §4.2.3 is unchanged:  $ivk := \text{Commit}_{r_{ivk}}^{ivk}(ak, nk)$ , the internal keys and the other keys derived from  $(ak, nk, r_{ivk})$ , and the rejection of a key whose  $ivk$  or internal  $ivk$  lies in  $\{0, \perp\}$ . On rejection step (2) is repeated with a new  $sk$ , and  $ak^{\mathbb{P}}$  is kept.
5. The protocol MUST ensure that all participants obtain the same  $ak$ ,  $nk$  and  $r_{ivk}$ , which ZIP 2005 calls  $r_{ivk\_ext}$ .

The spend authorising key  $ask = s$  stays threshold-shared. Figure 1 shows the tree.

Steps (3) and (5), the agreement of step (2) and the rejection of step (4) quote ZIP 2005, “Usage with FROST” and “Changes to the Protocol Specification”, §4.2.3; the uniform, independent draw of  $sk$  in step (2) and keeping  $ak^{\mathbb{P}}$  on rejection in step (4) are this volume's reading of “privately agree” and of the repetition with a new  $sk$ . Step (1) meets the even- $y$  clause of that section by Construction 6.7. Attempts of step (4) are independent,  $sk$  being fresh on each; the volume proves no bound on their rejection probability.

ZIP 2005 names two sources of  $ak^{\mathbb{P}}$  for  $\text{use\_qsk} = \text{true}$ : direct generation of an Orchard-protocol  $ask \in \mathbb{F}_{p_{\text{Vesta}}}^*$  with  $ak^{\mathbb{P}} = [ask]G^{\text{SpendAuth}}$ , which a dealer then shares by Construction 3.2; and FROST distributed key generation in which the spend-authorisation material is  $t$ -of- $n$  shared, here Construction 3.6. Its “Deployment” states that “FROST distributed key generation requires the  $\text{use\_qsk} = \text{true}$  case”. The key-generation section that ZIP 2005 cites for the second method is ZIP 312, “Key Generation”, which leaves key generation out of scope.

The Orchard pool is spend-only by consensus: no transaction increases its value, and every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 258, Draft, “Consensus rules from NU6.3 activation”; *Ironwood Guide*, §“The Orchard protocol and its two pools”). By consensus, therefore, only a party able to authorise spends for an address can create an Orchard-pool note to it, the consumed side of the Action being possibly a dummy note at that address (ZIP 326, “Rationale for key-generation restrictions”). ZIP 326 (Draft), “Wallet key-generation restrictions”, forbids conforming wallets to send Orchard-pool funds to a key with  $\text{use\_qsk} = \text{true}$ ; under that rule such a key, in particular every key from distributed key generation, holds no Orchard-pool notes, and consensus does not enforce the rule. A dealer-shared key with  $\text{use\_qsk} = \text{false}$  may spend notes of either pool. This qualifies the statement of §1.1 that both pools are served.

**Remark 6.9** (Scope of the ZIP 2005 constraints). ZIP 2005, “Usage with FROST”, states its constraints for keys whose  $ak$  is derived jointly by distributed key generation, and this volume reads its MUST as binding those keys. A dealer that shares an  $ask$  derived from a spending key, which ZIP 312, “Key Generation”, permits, produces an ordinary key with  $\text{use\_qsk} = \text{false}$ , known to the dealer and outside that constraint. A dealer that shares a directly generated  $ask$  may use Construction 6.8.

**Remark 6.10** (Custody of the common secrets). Under Construction 6.8 the spend authorising key  $ask$  is threshold-shared and no signing run assembles it, while  $sk$ ,  $nk$ ,  $r_{ivk}$  and  $qsk$  are known to every participant. Each participant MUST treat  $qsk$  as a secret held at least as securely and reliably as  $ask$  (ZIP 2005, “Usage with FROST”).

**Lemma 6.11** (Viewing material and the signing key). (a) For Construction 6.8, the tuple  $V := (sk, nk, qsk, qk, r_{ivk}, ivk, \dots)$ , with every key derived from  $ak$ ,  $nk$  and  $r_{ivk}$ , is a randomised function of  $ak$  alone,  $sk$  being drawn independently of the key material. Hence, for every experiment in which an adversary receives  $V$  together with other inputs, an adversary that receives only the other inputs,  $ak^{\mathbb{P}}$  among them, draws  $sk$  uniformly, computes  $V$  with the same rejection, and runs the first, has identical output distribution.

(b) For a dealer-shared key with  $use\_qsk = \text{false}$  and  $ask = \pm \text{ToScalar}(\text{PRFexpand}_{sk}([0x06]))$  derived from  $sk$ , the spending key determines  $ask$ , so no participant may hold  $sk$ . The participants’  $nk = \text{ToBase}(\text{PRFexpand}_{sk}([0x07]))$  and  $r_{ivk} = \text{ToScalar}(\text{PRFexpand}_{sk}([0x08]))$ , hence the full viewing key, are simulated from  $ak^{\mathbb{P}}$  and independent uniform values with the same rejection, at a cost, per key attempt, of the advantage against the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” plus the sum of the three reduction distances, below  $3 \cdot 2^{-257}$ .

*Proof.* (a) Every component of  $V$  is computed from  $sk$  and  $ak$ , and the rejection test reads only  $sk$  and  $ak$ ; the construction draws  $sk$  independently of the key material, as the simulating adversary does. (b) By the Ironwood Guide’s Lemma “Independence of domain-separated expansions” (“Pseudorandom functions and field reductions”), under Assumption “Pseudorandomness of PRF expansion” and with the three distinct leading bytes  $0x06$ ,  $0x07$ ,  $0x08$ , the triple  $(ask, nk, r_{ivk})$  is replaced by independent uniform values at the stated cost, each reduction distance being below  $2^{-257}$ . After the replacement  $nk$  and  $r_{ivk}$  are independent of  $ask$ , hence of  $ak^{\mathbb{P}}$ , and the simulator draws them and applies the rejection test, which reads  $ak$ ,  $nk$  and  $r_{ivk}$ . No property of  $H^{qk}$  is used.  $\square$

Section 7 uses the lemma to give the corrupt participants the common secrets. Class (Definition 1.1): ZIP 2005’s constraints on FROST key generation are specified (ZIP 2005, Proposed); how the participants privately agree on  $sk$  is designed but unspecified: ZIP 2005, “Usage with FROST”, requires only that they agree privately, no specification gives a protocol for it, and the revision of ZIP 312 proposed in pull request 895 publishes one.

## 6.5 Order of randomiser, validating key and signature digest

**Proposition 6.12** (Order of a threshold spend authorisation). Consider a signing run over an Orchard-protocol threshold key, re-randomised as in Definition 5.3 by an effective randomiser  $\hat{\alpha}$ , whose aggregate consensus accepts as the spend-authorisation signature of an Action with randomiser  $\alpha$  and randomised validating key  $rk$ , in a transaction with signature digest  $m$ . Except with the probabilities of Corollary 6.4 and of Proposition 5.5:

- (i) the randomiser  $\hat{\alpha} = \alpha$  is fixed before  $rk$ , which is the function  $ak^{\mathbb{P}} + [\hat{\alpha}]G^{\text{SpendAuth}}$  of it;
- (ii) the key  $rk$  is fixed before  $m$ , which commits to it (Ironwood Guide, §“Transaction digests and signatures”, Definition “Signature digest”);
- (iii) the digest  $m$  is fixed before any Round Two share, being the message of the binding factors and of the challenge;

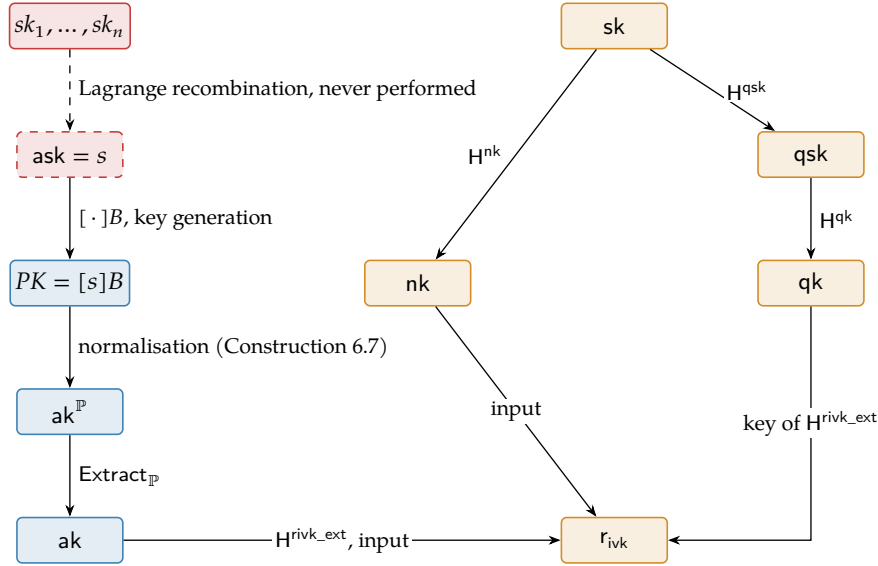


Figure 1: The Orchard-protocol threshold key of Construction 6.8. Red: threshold-shared values; the shares  $sk_1, \dots, sk_n$  determine  $ask$  by Lagrange recombination, which no signing run performs (dashed). Amber: secrets common to all participants, derived from the privately agreed spending key  $sk$ :  $nk$  and  $qsk$  from  $sk$ ,  $qk$  from  $qsk$ , and  $r_{ivk}$  from  $(ak, nk)$  keyed by  $qk$ . Blue: values known to every party, the group key  $PK$ , normalised to  $ak^{\mathbb{P}}$  and extracted to  $ak$ . Every edge is labelled by its operation.

- (iv) Round One precedes  $\hat{\alpha}$  under Construction 5.6, which hashes the commitment list, and need not precede it for a randomiser fixed before signing, as in Remark 6.13, where (i) to (iii) still hold;
- (v) the Action proof takes  $\alpha$  as auxiliary input and  $rk$  as instance, so it follows (i), and it is unconstrained with respect to  $m$  and to Round Two, proofs being authorising data outside the digest.

*Proof.* (i) is Corollary 6.4. (ii) and (iii) are the data dependencies of the cited definition and of Construction 4.8, as run in step (3) of Construction 5.10:  $\rho_i$  takes  $H_4(m)$  and  $c$  takes  $m$ . (iv) is the input of  $H_R$  in Construction 5.6. (v) follows from the *Ironwood Guide's* Definition “Authorising-data digest; effecting and authorising data” and “Construction of a transaction”, where signing can precede proving (ZIP 244, “Authorizing Data Commitment”, field A.3a). A randomiser that is a function of  $m$ , as under Construction 5.4 with the digest as message, would by (i) and (ii) be a fixed point, which Proposition 5.5 bounds; so that derivation admits no order except with that negligible probability.  $\square$

Class (Definition 1.1): the order of Proposition 6.12 under Construction 5.6 is designed but unspecified; no specification composes ZIP 312 with a transaction format.

**Remark 6.13** (Randomisers chosen by a transaction constructor). ZIP 312 derives the randomiser in Round Two (“Randomizer Generation” and “Round Two - Signature Share Generation”), and a derived randomiser equals an  $\alpha$  fixed earlier, as Corollary 6.4 requires, only with probability at most  $1/p_{\text{Vesta}} + \delta_{512} + q_h \cdot 2^{-256}$  (Lemma 5.7). A flow in which the transaction constructor fixes  $\alpha$  and  $rk$  before signing, and the Coordinator sends that  $\alpha$  as the effective randomiser, respects the order of Proposition 6.12, but no specification or published design states it: its design is an open problem (Definition 1.1), and under it the Coordinator chooses the effective randomiser outright (Remark 5.8).

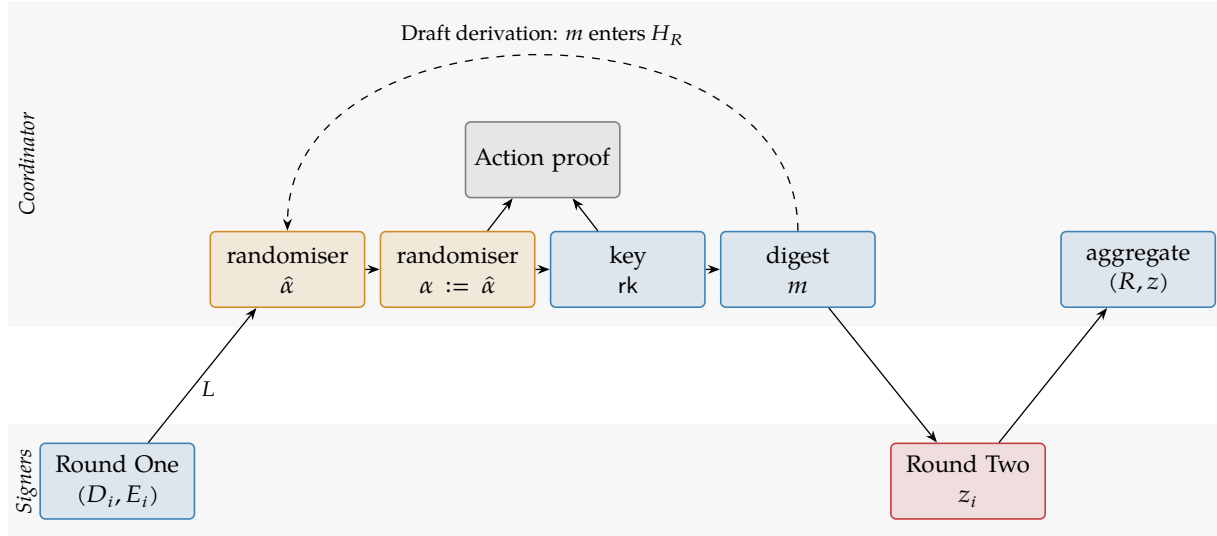


Figure 2: Order of one threshold spend authorisation (Proposition 6.12), in two lanes, Coordinator and signers, time running left to right: Round One commitments  $(D_i, E_i)$ ; the seed-and-commitments randomiser  $\hat{\alpha}$  (Construction 5.6), which hashes the commitment list  $L$ ; the Action’s randomiser  $\alpha$ ; the randomised validating key  $rk$ ; the signature digest  $m$ , which commits to  $rk$ ; the Round Two shares  $z_i$ ; the aggregate  $(R, z)$ . The Action proof is a branch from  $\alpha$  and  $rk$ , connected neither to  $m$  nor to Round Two. The dashed edge closes the cycle of the Draft derivation (Construction 5.4 with  $m$  as message): randomiser,  $rk$ , digest and back, which Proposition 5.5 bounds.

## 6.6 Threshold signing of an Action

The single-signer construction is cited, not rebuilt. Each Action draws  $\alpha$  by  $\text{GenRandom}$ , sets  $\text{rsk} := \text{ask} + \alpha$  and  $\text{rk} := \text{ak}^{\mathbb{P}} + [\alpha]G^{\text{SpendAuth}}$ , proves the Action statement with  $\alpha$  as auxiliary input, computes the signature digest and signs it under  $\text{rsk}$  (*Ironwood Guide*, §“Randomised validating keys”, Construction “Randomised validating key”; “Construction of a transaction”, Construction “Ironwood-pool bundle”, steps (5), (7), (10) and (11); protocol specification, §“Sending Notes (Orchard)” and §“Spend Authorization Signature (Sapling and Orchard)”).

**Construction 6.14** (Threshold spend authorisation of an Action). The transaction builder acts as Coordinator: it creates the Action proof and so already controls the privacy of the transaction (ZIP 312, “Threat Model”). For each Action that consumes a note of an Orchard-protocol threshold key (Construction 6.8), in the order of Proposition 6.12, it proceeds as follows.

1. It runs Round One (Construction 5.10, step (1)) with a signing set  $I$  of at least  $t$  participants.
2. It derives the effective randomiser  $\hat{\alpha}$  by Construction 5.6, the only designed derivation that respects that order: Proposition 5.5 excludes the Draft derivation, and a constructor-chosen  $\alpha$  has no published design (Remark 6.13).
3. It sets  $\alpha := \hat{\alpha}$  and  $\text{rk} := \text{ak}^{\mathbb{P}} + [\alpha]G^{\text{SpendAuth}}$ .
4. Once every Action of the transaction is fixed, it computes the one signature digest  $m$  of the transaction.
5. It runs Round Two of Construction 5.10 with message  $m$ , each signer performing the message check of Definition 5.9, and aggregates.

6. At any point after step (3), it proves the Action statement with  $\alpha$  as auxiliary input (*Ironwood Guide*, §“The Action statement”).

These steps replace the draw of  $\alpha$  and the single-holder signature in the cited construction; every other Action, dummy spends included, is built as there. No party holds  $\text{rsk}$ . The builder alone makes the binding signature (*Ironwood Guide*, §“The binding signature”). Every threshold-spent Action has its own  $\alpha$  and  $\text{rk}$  and its own signing run, with fresh nonces and randomiser, all on the one digest  $m$ ; since  $m$  commits to the  $\text{rk}$  of every Action, the Round One and the randomiser of every threshold-spent Action precede  $m$ , and hence every Round Two (Proposition 6.12, (i) to (iv), applied to each Action). Class (Definition 1.1): designed but unspecified; the composition of ZIP 312 with the transaction is this volume’s own.

*Acceptance.* After normalisation  $\text{ak}^{\mathbb{P}} = PK$ , and  $\alpha = \hat{\alpha}$ ; so Proposition 6.3 and Corollary 6.4, part (i), give a signature valid under  $\text{rk}$  on  $m$ , except in the zero-nonce case and when  $\text{rk} = O$ , whose probabilities Theorem 4.11 and Corollary 6.4, part (iii), bound.  $\square$

The circuit and the consensus rules are unchanged (Remark 6.5). Both pools of the Orchard protocol use the same RedPallas spend authorisation, so one flow serves an Action of either pool (*Ironwood Guide*, §“The Orchard protocol and its two pools”; ZIP 229, “Reuse of the Orchard protocol with minimal changes”); which pool’s notes a key may hold is fixed in §6.4.

## 7 Security

This section states the assumptions and games, then each security statement at the strength its support allows: the cited unforgeability theorem for FROST, with the non-interactive syntax in which it is stated and its correspondence to RFC 9591; its transfer to Orchard-protocol dealer keys, and the open composition with distributed key generation; the claimed unforgeability of the re-randomised scheme and the gaps in its proof; threshold spend authority, proved from a target notion that is itself open; unlinkability against specified outside observers, and the linkage that returned signature shares permit; and the class of the principal constructions and claims of the volume. Throughout,  $\mathbb{G}$  is a group of prime order  $q$  with generator  $B$  and identity  $O$ ; a count of oracle queries is written  $Q$ , never  $q$ .

### 7.1 One-more discrete logarithms

**Definition 7.1** (One-more discrete logarithm problem). The *one-more discrete logarithm game* for  $\mathbb{G}$  is a search game in the sense of the *Crypto Guide*, §“Security as a game”, Definition “Security game and advantage”. The challenger keeps a challenge counter  $k$  and a query counter  $\ell$ , both initially 0, and offers two oracles.

Chal(): set  $k := k + 1$ , draw  $x_k \xleftarrow{\$} \mathbb{F}_q$ , and return  $X_k := [x_k]B$ .

DLog( $X$ ), for any  $X \in \mathbb{G}$ : if  $X$  was queried before, return the stored answer; otherwise set  $\ell := \ell + 1$ , store and return the discrete logarithm of  $X$  to base  $B$ .

The adversary outputs  $(y_1, \dots, y_k) \in \mathbb{F}_q^k$  and *wins* if  $y_j = x_j$  for every  $j \leq k$  and  $\ell < k$ . Its advantage  $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(A)$  is its probability of winning. The case of  $\ell + 1$  challenges and at most  $\ell$  queries is the  $\ell$ -OMDL problem; the case of one challenge and no query is the discrete logarithm problem of the *Crypto Guide*, §“The discrete logarithm problem”.

The *Crypto Guide* names the problem in prose only. The game is that of ePrint 2022/833, Figure 9, in additive notation; its challenge queries are those that Theorem 7.9 counts.

**Assumption 7.2** (One-more discrete logarithms on Pallas). Let  $\mathbb{G}$  be the Pallas group, of order  $q = p_{\text{Vesta}}$ , with generator  $B = G^{\text{SpendAuth}}$  (*Ironwood Guide*, §“Fields, groups, and encodings”; “GroupHash, domain separation, and nothing-up-my-sleeve generators”). Every efficient adversary has negligible advantage  $\text{Adv}_{\mathbb{G}}^{\text{omdl}}$  (*Math Guide*, §“Polynomial, exponential, and negligible functions”) in the game of Definition 7.1.

By its case of one challenge, the assumption implies the *Ironwood Guide*’s Assumption “Discrete logarithms on Pallas”.

**Definition 7.3** (Algebraic one-more discrete logarithm problem). Let  $\mathbb{G}$ ,  $q$  and  $B$  be as in Definition 7.1 and let  $\ell \geq 0$  be an integer. In the  $\ell$ -AOMDL game the challenger draws  $x_0, \dots, x_\ell \xleftarrow{\$} \mathbb{F}_q$  and gives the adversary  $X_i := [x_i]B$  for  $0 \leq i \leq \ell$ . Its oracle takes a *declaration*  $(\alpha, \beta_0, \dots, \beta_\ell) \in \mathbb{F}_q^{\ell+2}$  and returns

$$\alpha + \sum_{i=0}^{\ell} \beta_i x_i,$$

the discrete logarithm of  $[\alpha]B + \sum_i [\beta_i]X_i$ ; it answers at most  $\ell$  declarations. The adversary wins by outputting  $(x_0, \dots, x_\ell)$ , and  $\text{Adv}_{\mathbb{G}}^{\ell\text{-aomdl}}(A)$  is its probability of winning.

The game is the intended one of ePrint 2024/436, Assumption 1 and Figure 1. The restriction lies on the oracle’s queries, which are declared combinations of  $B$  and the challenges, and not on the adversary, so no statement about algebraic adversaries is needed; the oracle of Definition 7.1 answers any element. The printed Figure 1 is ill-formed in three clauses: its oracle takes  $\beta_1, \dots, \beta_\ell$  but returns an expression in  $\beta_0$ ; it never checks the queried element against the declared combination; and its query counter is never incremented, so its final test, that the count is below  $\ell + 1$ , bounds nothing. This volume reads the game as stated above.

**Lemma 7.4** (OMDL hardness implies AOMDL hardness). *For every  $\mathbb{G}$  of prime order  $q$ , every  $\ell \geq 0$  and every adversary  $A$  in the  $\ell$ -AOMDL game of Definition 7.3, there is an adversary  $A'$  in the game of Definition 7.1, making  $\ell + 1$  challenge queries and at most  $\ell$  oracle queries, with*

$$\text{Adv}_{\mathbb{G}}^{\text{omdl}}(A') = \text{Adv}_{\mathbb{G}}^{\ell\text{-aomdl}}(A),$$

*running in the time of  $A$  plus  $\ell + 2$  scalar multiplications per declaration.*

*Proof.* Algorithm  $A'$  queries Chal  $\ell + 1$  times, obtaining  $X_1, \dots, X_{\ell+1}$ , and runs  $A$  on them as  $X_0, \dots, X_\ell$ . On a declaration  $(\alpha, \beta_0, \dots, \beta_\ell)$  it queries DLog on  $[\alpha]B + \sum_i [\beta_i]X_i$  and returns the answer, which is  $\alpha + \sum_i \beta_i x_i$ , the value the AOMDL oracle returns. It outputs the output of  $A$ . The view of  $A$  is that of the AOMDL game, the query counts agree, and  $A'$  wins exactly when  $A$  wins.  $\square$

The assumption of ePrint 2024/436, Theorem 1, is therefore no stronger than the OMDL assumption of the analysis RFC 9591 cites (ePrint 2022/833). Hence the analysis of ePrint 2024/436 needs no assumption beyond that of ePrint 2022/833.

## 7.2 Security of FROST signing

**Definition 7.5** (Threshold signatures with leader requests). A *threshold signature scheme with leader requests* TS has  $n$  servers, the participants of Definition 2.1 with identifiers  $1, \dots, n$ , a

threshold  $t$ , and the following algorithms, each with access to a random oracle  $h$ .

1. *Key generation*  $\text{Kg}[h]$ , run by a trusted party, outputs a verification key  $vk$ , public auxiliary information  $vks$  (the paper's *aux*), which holds the verification shares, and secret keys  $sk_1, \dots, sk_n$ .
2. *Token generation*: server  $i$  issues a first-round *token*  $pp$  and keeps a secret state for it.
3. A *leader request*  $lr$ , formed by the *leader*, a party holding no secret of the scheme, carries a message  $lr.\text{msg}$ , a signer set  $lr.\text{SS} \subseteq \{1, \dots, n\}$ , and a map  $lr.\text{PP}$  from  $lr.\text{SS}$  to tokens. The scheme is an *echo scheme*: requests carry the tokens.
4. *Partial signing*  $\text{PS}[h](lr, i)$ : server  $i$  answers  $lr$  with a partial signature computed from the state of the token  $lr.\text{PP}(i)$ , or with the failure symbol  $\perp$  if  $lr.\text{PP}(i)$  is not an unused token of its own; each token is thus answered at most once.
5. *Aggregation*  $\text{Agg}[h]$  maps partial signatures to a signature  $sig$ .
6. *Verification*  $\text{Vf}[h](vk, M, sig)$ .
7. *Strong verification*  $\text{SVf}[h](vk, lr, sig)$ , which accepts, for each pair  $(vk, lr)$ , at most one signature.

The syntax is that of ePrint 2022/833, Section 3.1, restricted to what this volume uses. The leader is the Coordinator of Definition 2.8.

**Definition 7.6** (Threshold unforgeability levels). Let TS be an echo scheme (Definition 7.5). The game has the following procedures.

*Init*( $C$ ): require  $C \subseteq \{1, \dots, n\}$  and  $|C| < t$ ; draw  $h$ ; run  $(vk, vks, sk_1, \dots, sk_n) \leftarrow \text{Kg}[h]$ ; set  $HS := \{1, \dots, n\} \setminus C$ ; return  $vk, vks$  and  $sk_j$  for  $j \in C$ .

*Token oracle* on  $i \in HS$ : server  $i$  issues a token, which is added to the set  $PP_i$  and returned.

*Partial-signing oracle* on  $(i, lr)$ ,  $i \in HS$ : record  $lr$ ; run  $\text{PS}[h](lr, i)$ ; if the answer is not  $\perp$ , add  $i$  to  $S_2(lr)$ ; return the answer.

*Random oracle*: return  $h(x)$ .

*Fin*( $M, sig$ ): for every recorded  $lr$  set

$$S_3(lr) := \{i \in HS \cap lr.\text{SS} : lr.\text{PP}(i) \in PP_i\}, \quad S_4(lr) := HS \cap lr.\text{SS}.$$

The adversary loses unless  $\text{Vf}[h](vk, M, sig)$  holds.

The adversary acts as the leader. The *trivial-forgery predicates* are

$$\begin{aligned} \text{tf}_2(lr) : & \quad |S_2(lr)| \geq t - |C|, \\ \text{tf}_3(lr) : & \quad \text{tf}_2(lr) \text{ and } S_2(lr) = S_3(lr), \\ \text{tf}_4(lr) : & \quad \text{tf}_2(lr) \text{ and } S_2(lr) = S_4(lr), \end{aligned}$$

and, for  $i \in \{2, 3, 4\}$ , the *trivial strong-forgery predicate*  $\text{tsf}_i(lr, vk, sig)$  is  $\text{tf}_i(lr)$  and  $\text{SVf}[h](vk, lr, sig)$ . The game TS-UF- $i$  is won by a valid  $(M, sig)$  such that no recorded  $lr$  with  $lr.\text{msg} = M$  satisfies  $\text{tf}_i(lr)$ ; TS-SUF- $i$  replaces  $\text{tf}_i(lr)$  by  $\text{tsf}_i(lr, vk, sig)$ . The advantage  $\text{Adv}_{\text{TS}}^{\text{ts-suf-}i}(A)$  is the probability of winning.

The games are those of ePrint 2022/833, Figures 2 and 3, the predicates transcribed exactly. Under TS-SUF-3 a forgery on  $M$  is trivial only if one request  $lr$  for  $M$  was answered by at least  $t - |C|$  honest servers, those servers are exactly the honest servers of  $lr.ss$  listed with a token they issued, and  $sig$  is the unique signature that strong verification associates with  $(vk, lr)$ . Only these levels are defined: the volume uses TS-SUF-3 and the failure of TS-UF-4.

**Construction 7.7** (FROST1 in the non-interactive syntax). Let  $t \leq n < q$ , and let  $h_j(x) := h(j, x)$  for one random oracle  $h$ . The scheme  $\text{FROST1}[\mathbb{G}]$  is the following echo scheme.

1. Kg: draw  $a_0, \dots, a_{t-1} \xleftarrow{\$} \mathbb{F}_q$ ; set  $sk_i := \sum_j i^j a_j$  and  $vk_i := [sk_i]B$  for each  $i$ ,  $vk := [a_0]B$  and  $vks := (vk_1, \dots, vk_n)$ .
2. Token of server  $i$ : draw  $u, v \xleftarrow{\$} \mathbb{F}_q$ ; the token is  $pp := ([u]B, [v]B)$ , and  $(u, v)$  is kept for it.
3. For a request  $lr$  and each  $i \in lr.ss$  let  $(U_i, V_i) := lr.pp(i)$  and  $b_i := h_1(vk, lr, i)$ ; let
 
$$R := \sum_{i \in lr.ss} (U_i + [b_i]V_i), \quad c := h_2(vk, lr.msg, R).$$
4. PS( $lr, i$ ): if  $lr.pp(i)$  is an unused token of  $i$  with secret  $(u, v)$ , erase  $(u, v)$  and return  $(R, z_i)$  with  $z_i := u + b_i v + c \lambda_{lr.ss, i} sk_i$ ; otherwise return  $\perp$ .
5. Agg: if all partial signatures carry one  $R$ , return  $(R, \sum_i z_i)$ .
6. Vf( $vk, M, (R, z)$ ): accept if and only if  $[z]B = R + [h_2(vk, M, R)] vk$ .
7. SVf( $vk, lr, (R^*, z^*)$ ): accept if and only if  $R^*$  equals the  $R$  computed from  $lr$  and  $[z^*]B = R^* + [c] vk$ .

The construction is ePrint 2022/833, Figure 8, in additive notation; the paper's token secrets  $r, s$  and binding factors  $d_i$  are written  $u, v$  and  $b_i$  here, those letters being taken. FROST2 (Crites, Komlo and Maller, ePrint 2021/1375) replaces the factors  $b_i$  by one  $b := h_1(vk, lr)$ ; ePrint 2022/833, Section 5, proves FROST2 TS-SUF-2-secure and not TS-UF-3-secure. RFC 9591 specifies FROST1: its binding factor takes the identifier (Section 4.4), and its Section 7.2 ("Optimizations") rules the shared factor NOT RECOMMENDED, because it removes the guarantee that the set of participants that started Round One is the set that produced the signature.

**Proposition 7.8** (RFC 9591 signing realises FROST1). *Let a ciphersuite (Definition 4.1) whose element serialisation is total, as that of FROST(Pallas, BLAKE2b-512) is, meet Assumption 4.2 with distance  $\delta$  and digest length  $N_h$  bytes. Map RFC 9591 signing (Constructions 4.4, 4.8 and 4.9, Definition 4.7) over key material of Construction 3.2 at identifiers  $1, \dots, n$  to the syntax of Definition 7.5 as follows: the Coordinator is the leader; the Round One pair  $(D_i, E_i)$  is the token; the request  $(m, L)$  is  $lr$  with  $lr.msg = m$ ,  $lr.ss$  the identifiers of  $L$  and  $lr.pp(i)$  the pair of  $i$  in  $L$ ;  $vk := PK$  and  $vks := (PK_1, \dots, PK_n)$ , from which the coefficient commitments are computed, since  $t$  verification shares determine the committed polynomial in the exponent (Lemmas 2.6 and 2.4). For every adversary  $A$  in the TS-SUF-3 game of Definition 7.6 for RFC 9591 signing under this map whose requests list only identifiers in  $\{1, \dots, n\}$ , making  $q_s$  token queries and  $q_h$  hash queries, there is an adversary  $A'$  against  $\text{FROST1}[\mathbb{G}]$  (Construction 7.7) with the same query counts and about the same running*

time, such that, with  $Q := q_s + q_h + 1$ ,

$$\begin{aligned} \text{Adv}_{\text{RFC}}^{\text{ts-suf-3}}(A) &\leq \text{Adv}_{\text{FROST1}[\mathbb{G}]}^{\text{ts-suf-3}}(A') \\ &\quad + 2q_s(\delta + (q_h + 2q_s) \cdot 2^{-256}) + 3Q^2 \cdot 2^{-8N_h} + (q_h + (n+1)q_s + 1)\delta. \end{aligned}$$

A request naming another identifier lies outside the FROST1 game and is not covered, and so is a suite whose serialisation is not total: its abort on  $R = \emptyset$  follows hash evaluations that consume no token. The correspondence rests on four facts.

- (a) The binding-factor input  $\text{ser}_{\mathbb{G}}(PK) \parallel H_4(m) \parallel H_5(\text{enc}(L)) \parallel \text{ser}_{\mathbb{F}_q}(i)$  determines  $(vk, lr, i)$ , except on a collision of  $H_4$  or  $H_5$  or on a digest used before its preimage is queried.
- (b) The challenge input  $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK) \parallel m$  is an injective reordering of  $(vk, M, R)$ , the two element encodings having fixed length.
- (c) The separated hashes  $H_1$  and  $H_2$  realise  $h_1$  and  $h_2$  as independent oracles.
- (d) Hedged nonces replace the uniform token secrets of FROST1 at the cost of Lemma 4.5 per nonce, a hop the reduction needs because it does not know the honest shares that enter  $H_3$ .

*Proof.* Game hops, with additive losses (Crypto Guide, §“Security as a game”, Remark “Game-hopping”).

Game  $G_0$  is the RFC game.

Game  $G_1$ : every honest nonce is a uniform scalar. The  $2q_s$  nonces are replaced one at a time, in the order in which they are drawn. At each hop Lemma 4.5 applies with the rest of the game as the observer: its queries to  $H_3$  are the  $q_h$  queries of  $A$  and at most  $2q_s$  evaluations by honest servers, each at an input with a fresh 32-byte prefix. Hence  $|\Pr[G_0] - \Pr[G_1]| \leq 2q_s(\delta + (q_h + 2q_s) \cdot 2^{-256})$ . Game  $G_2$  aborts on a collision of  $H_4$  or of  $H_5$ , or when a hash input contains an output value of  $H_4$  or  $H_5$  before the query that produces it. Each of the two oracles receives at most  $Q$  queries, from  $A$  and one per answered request, at most  $q_s$  of them. By Assumption 4.2 a collision of either has probability at most  $Q^2 \cdot 2^{-8N_h}$  in total; each new answer equals one of at most  $Q$  digest strings already present in inputs with probability at most  $Q \cdot 2^{-8N_h}$ , which over  $Q$  queries and both oracles gives  $2Q^2 \cdot 2^{-8N_h}$ .

Game  $G_3$  answers  $H_1$  and  $H_2$  at distinct inputs by uniform scalars. Under Assumption 4.2 each answer is an independent sample within distance  $\delta$  of uniform, and the distinct inputs number at most  $q_h$  queries of  $A$ , at most  $n$  binding factors and one challenge per answered request, and one challenge of the final verification; replacing the answers one at a time costs  $(q_h + (n+1)q_s + 1)\delta$  (Math Guide, Theorem “Properties of statistical distance”).

In  $G_3$ , algorithm  $A'$  plays the FROST1 game. It relays token queries. It simulates  $H_3$ ,  $H_4$  and  $H_5$  lazily and records their queries. It answers an  $H_1$  query  $\text{ser}_{\mathbb{G}}(PK') \parallel u \parallel v \parallel \text{ser}_{\mathbb{F}_q}(i)$  by  $h_1(vk, lr, i)$  when  $PK' = PK$  and  $u, v$  are recorded outputs of  $H_4$  on  $m$  and of  $H_5$  on  $\text{enc}(L)$ , with  $lr$  formed from  $(m, L)$ , and by a fresh value otherwise; by (a) and the abort rule of  $G_2$  the request is unique. It answers an  $H_2$  query  $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(PK') \parallel m$  by  $h_2(vk, m, R)$  when  $PK' = PK$ , and by a fresh value otherwise; by (b) the input determines  $(m, R)$ , and by (c) the two simulations are independent. The checks of Construction 4.8, namely the message check, the presence of the member’s own unused pair in  $L$ , deserialisation of every commitment and the order of the identifiers, are decided from public data. Algorithm  $A'$  therefore queries the partial-signing oracle exactly when the RFC participant would answer, does not query the oracle when the participant would abort, leaving the token unused in both games, since an aborting participant keeps its pair; it never submits a token already answered, and returns  $z_i$ , discarding  $R$ . The sets  $S_2$ ,  $S_3$  and  $S_4$  of the two games coincide; a valid RFC signature  $\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{F}_q}(z)$  is a valid FROST1 signature  $(R, z)$ ; and strong verification agrees. So  $A'$  wins when  $A$  wins in  $G_3$ .  $\square$

**Theorem 7.9** (Unforgeability of FROST). *Let  $\mathbb{G}$  have prime order  $q$ , and let  $\text{FROST1}[\mathbb{G}]$  be Construction 7.7 with  $n$  servers and threshold  $t$ . For every adversary  $\mathcal{A}$  in the TS-SUF-3 game of Definition 7.6 making at most  $q_s$  token queries, the first-round queries, and at most  $q_h$  random-oracle queries, there is an adversary  $\mathcal{B}$  in the game of Definition 7.1, making at most  $2q_s + t$  challenge queries, such that, with  $Q := q_s + q_h + 1$ ,*

$$\text{Adv}_{\text{FROST1}[\mathbb{G}]}^{\text{ts-suf-3}}(\mathcal{A}) \leq 4nQ\sqrt{\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}) + 6Q/q},$$

*only the two terms  $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B})$  and  $6Q/q$  lying under the root. Algorithm  $\mathcal{B}$  runs in about twice the time of  $\mathcal{A}$ , plus at most  $6nQ + 4q_s + 2n^2$  exponentiations and group operations. For some parameters, for instance  $n = 20$  and  $t = 3$ , the scheme  $\text{FROST1}[\mathbb{G}]$  is not TS-UF-4-secure. The hypotheses are those of the game: static corruption of fewer than  $t$  servers, trusted key generation, and the random-oracle model.*

*Proof.* Cited: ePrint 2022/833, Theorem 5.4, whose bound, query counts and running time are those stated, the root spanning only  $\text{Adv}_{\mathbb{G}}^{\text{omdl}}(\mathcal{B}) + 6Q/q$ ; the TS-UF-4 separation is its Section 5.4, by an explicit attack at  $n = 20$  and  $t = 3$ .  $\square$

RFC 9591, Section 7 (“Security Considerations”), claims security against existential unforgeability under chosen-message attacks as defined in that analysis, assuming that the shares are generated and distributed securely, by a trusted dealer or by a distributed key generation protocol, and that at most the Coordinator and  $\text{MIN\_PARTICIPANTS} - 1$  participants are corrupted. For dealer keys in a suite with total element serialisation and requests listing only identifiers in  $\{1, \dots, n\}$ , the claim rests on Theorem 7.9 at level TS-SUF-3, through Proposition 7.8; with distributed key generation it is open (Remark 7.13). It does not rest on the theorem of the original paper, which concerns an interactive variant.

**Remark 7.10** (Role of the binding factors). The share  $z_i$  is bound through  $\rho_i$  to  $PK$ , the message, the commitment list  $L$  and the identifier (Definition 4.7). Changing the message, the signing set or any commitment in  $L$  changes some  $\rho_j$ , hence  $R$  and  $c$ , except on a collision of  $H_1$ ,  $H_4$  or  $H_5$ ; a share moved into another run fails its share equation, and the aggregate fails verification. Without such binding, concurrent sessions of two-round Schnorr signing admit forgeries: in time subexponential in the bit length of  $q$ , by an attack that combines the challenges of concurrent sessions (Drijvers, Edalatnejad, Ford, Kiltz, Loss, Neven and Stepanovs, “On the Security of Two-Round Multi-Signatures”, IEEE Symposium on Security and Privacy 2019), which applies to  $t$ -of- $n$  threshold signing with up to  $t - 1$  corrupt participants (ePrint 2020/852, Section 2.5); and in polynomial time once more than  $\log_2 q$  sessions are open concurrently (Benhamouda, Lepoint, Loss, Orrù and Raykova, “On the (in)security of ROS”, EUROCRYPT 2021). The remark explains Theorem 7.9 and adds no claim; the security of FROST rests on that theorem.

### 7.3 Unforgeability and key generation

**Lemma 7.11** (Sign normalisation and unforgeability). *Consider a game over dealer key material (Construction 3.2) that decides its winning condition from public data, the adversary’s output and the transcript of its oracles, as the games of Definition 7.6 do, over a group in which exactly half of the  $q - 1$  non-identity points pass the parity test of Construction 6.7; for Pallas this test is the last bit of  $\text{repr}_{\mathbb{P}}$ . An adversary with advantage  $\epsilon$  in the game whose key material is normalised by Construction 6.7 is, run unchanged, an adversary with advantage at least*

$$\frac{q-1}{2q} \epsilon$$

in the game with unnormalised dealer key material.

*Proof.* The reduction plays the unnormalised game, runs the adversary unchanged on it, and stops unless  $PK \neq O$  and  $PK$  passes the parity test; since  $s$  is uniform, this event has probability  $(q-1)/(2q)$ , and it is decided from public data before the adversary receives anything else. On the event normalisation changes nothing. The normalised key material is distributed as unnormalised dealer key material conditioned on the event: step (1) of Construction 6.7 redraws a zero secret, and negation (Lemma 6.6) maps the sharings whose group key fails the test bijectively onto those whose group key passes it, the coefficients staying uniform. The conditioned view is therefore that of the normalised game, and the win carries over.  $\square$

For  $q = p_{\text{Vesta}}$  the factor is  $1/2 - 1/(2q)$  with  $1/(2q) = 2^{-255}$  to the precision stated: normalisation costs a factor 2 in advantage, up to that term.

**Corollary 7.12** (Unforgeability for Orchard-protocol dealer keys). *Assume Assumption 7.2, and Assumption 4.2 as Proposition 6.2 instantiates it for FROST(Pallas, BLAKE2b-512) (Construction 6.1). Then Theorem 7.9, through Proposition 7.8, holds for RFC 9591 signing against adversaries whose requests list only identifiers in  $\{1, \dots, n\}$ , with the effective randomiser zero on dealer key material (Construction 3.2) normalised by Construction 6.7 and completed in either of two ways, the corrupt participants holding the common secrets:*

- (a) *by Construction 6.8 (`use_qsk = true`), with  $sk$  drawn independently of the key material and given to every participant; the TS-SUF-3 advantage of such an adversary is at most  $2q/(q-1)$  times the sum of the bound of Theorem 7.9 and the loss of Proposition 7.8, with  $q = p_{\text{Vesta}}$ , the simulation of Lemma 6.11, part (a), being exact;*
- (b) *for a dealer that shares an ask derived from a spending key that no participant holds (`use_qsk = false`; Ironwood Guide, Construction “Spend-side secrets”), with the further terms of Lemma 6.11, part (b), of the Ironwood Guide’s Assumption “Pseudorandomness of PRF expansion” and Lemma “Rejection in key generation”, and of the bias below  $2^{-257}$  of ToScalar (Definition “Field reductions” there), by a hybrid that replaces ask with a uniform scalar.*

*The corollary covers signing with the randomiser zero only; the re-randomised protocol is the subject of §7.4.*

*Proof.* Hybrids, then the cited statements. For (a), Lemma 6.11, part (a), replaces the adversary that receives  $sk$  and the values derived from it by one that draws  $sk$  itself and computes them from  $ak = \text{Extract}_{\mathbb{P}}(PK)$ , rejection included, with no loss. For (b), Lemma 6.11, part (b), simulates  $nk$  and  $r_{ivk}$  from independent uniform values; the key-derivation hybrid then replaces  $ask$ , before normalisation, by a uniform scalar, at the cost of Assumption “Pseudorandomness of PRF expansion”, of Lemma “Rejection in key generation” and of the ToScalar bias. These hybrids apply because the win is efficiently decidable from the key: the deciding algorithm answers the signing queries with the shares. What remains is the game of Definition 7.6 over normalised dealer keys. Lemma 7.11 passes to unnormalised dealer keys, which are the output of  $Kg$  of FROST1; Proposition 7.8 passes such an adversary to FROST1[Pallas]; and Theorem 7.9 with Assumption 7.2 bounds that advantage.  $\square$

**Remark 7.13** (Distributed key generation and unforgeability). Theorem 7.9 assumes trusted key generation, and no checked source proves FROST1 unforgeable with a distributed key generation, in particular with Construction 3.6. RFC 9591, Section 7, lists secure generation by a dealer or by a distributed protocol as an assumption, not as a result. The nearest results are two. Crites, Komlo and Maller (ePrint 2021/1375, Theorem 7) prove FROST2 with their key generation unforgeable under the one-more discrete logarithm assumption and a further assumption: that an

algorithm outputting a group element with a valid Schnorr proof of knowledge for it knows the element's discrete logarithm, in that an extractor computes the logarithm from the algorithm's code and random coins. The original paper (ePrint 2020/852) proves, under discrete logarithms, an interactive variant with its own key generation, not the specified scheme. Classification (Definition 1.1): unforgeability of FROST1 with distributed key generation is an open problem.

## 7.4 Unforgeability of re-randomised FROST

**Definition 7.14** (Re-randomisable threshold unforgeability). The game TRUF, for the instance of Definition 5.3 given by FROST with effective-randomiser map  $H_r$ , runs as follows.

1. The adversary chooses  $(n, t)$  and a static set  $cor$  with  $|cor| \leq t - 1$ .
2. The challenger runs centralised key generation, a Shamir sharing of a uniform  $x$  with  $PK = [x]B$ , and gives  $PK$ , every  $PK_i$  and  $x_j$  for  $j \in cor$ .
3. Oracle  $\text{Sign}(k, ssid)$ , for honest  $k$ , opens session  $ssid$  and returns Round One commitments  $\rho_k = (R_k, S_k)$ .
4. Oracle  $\text{Sign}'(k, ssid, m, S, \alpha, aux, (\rho_i)_{i \in S})$ , once per open session: add  $m$  to the set  $\mathcal{Q}_m$ ; set  $\hat{\alpha} := H_r(\alpha, aux)$ ,  $\bar{x}_k := \frac{x_k}{\hat{\alpha}} + \hat{\alpha}$  and  $\bar{PK} := PK + [\hat{\alpha}]B$ ; return the Round Two share  $z_k$  computed with  $\bar{x}_k$  and  $\bar{PK}$ .
5. The adversary outputs  $(m^*, \sigma^*, \alpha^*, aux^*)$  and wins if  $m^* \notin \mathcal{Q}_m$  and  $\sigma^*$  is valid on  $m^*$  under  $PK$  or under  $PK + [H_r(\alpha^*, aux^*)]B$ .

The randomisers are adversarial, and the oracle applies  $H_r$  itself. The advantage  $\text{Adv}^{\text{truf}}(A)$  is the probability of winning.

The game is that of ePrint 2024/436, Definition 6 and Figure 4, in additive notation and with the evident reading of its interface; the symbol  $\rho_k$  is the paper's name for a commitment pair, not a binding factor.

**Remark 7.15** (Scope of the game). Four clauses bound what TRUF covers.

- (a) TRUF is existential: its oracle records messages only, and a new signature on a signed message does not win. It is therefore weaker than Definition 5.1, which records message–signature pairs, and Theorem 1 of ePrint 2024/436 would not, as printed, give the strong notion that ZIP 312, “Requirements”, asks for (Remark 5.13, (b)).
- (b) The choice  $\alpha = 0$  does not give the identity shift, since  $H_r(0, aux)$  need not be 0, contrary to the preprint's statement that an adversary obtains unrandomised signatures by submitting the randomiser zero. The game has no unrandomised signing oracle and does not contain ordinary unforgeability.
- (c) Freshness is on the message alone. All Actions of a transaction sign one digest, so a forgery under a second Action's randomised key on a digest already signed for another Action is not a win.
- (d) A Coordinator that chooses the effective randomiser outright (Remark 5.8) is outside the game, whose oracle applies  $H_r$ .

Each clause is a reason why the target notion of §7.5, Definition 7.18, is stated separately.

**Remark 7.16** (The claimed unforgeability of re-randomised FROST). Theorem 1 of ePrint 2024/436 claims that Rerandomized-FROST is unforgeable in the sense of Definition 7.14, in the random-oracle model under the assumption of Definition 7.3, with its Equation (3) as printed:

$$\text{Adv} \leq \sqrt{Q \cdot \text{Adv}_D^{\text{aomdl}}(\lambda) + 2Q^2/q - \text{negl}(\lambda)},$$

all three terms under one root, with  $Q := q_h + q_s$  counting random-oracle and signing queries and  $q$  the group order ( $p$  in the paper). No number is used from it. Its assumptions are static corruption and centralised key generation (Section 2.3 there); the paper states that its definition can be adapted to distributed key generation, and gives no proof. The abstract’s “same security assumptions underlying plain FROST” is correct only in the sense of Lemma 7.4.

**Remark 7.17** (Gaps in the printed reduction). The reduction of ePrint 2024/436, Section 6, has five gaps, one per clause.

- (a) The simulated verification shares do not interpolate to  $PK$  as printed. For  $|C| = t - 1$  and  $L_j(i)$  the Lagrange coefficient of the node  $j$  of  $\{0\} \cup C$  at  $i$ , the required relation is

$$PK_i = [L_0(i)] PK + \sum_{j \in C} [L_j(i)] [x_j] B.$$

- (b) The Round Two logarithm query omits the Lagrange coefficient: the share  $z_k = r_k + s_k a_k + c \lambda_k \bar{x}_k$  needs the logarithm of  $R_k + [a_k] S_k + [c \lambda_k] \bar{P} K_k$ .
- (c) The random-oracle simulation is not a consistent lazy sampler: the table of  $H_r$  stores no outputs, index and message are mixed in the binding-factor table, one branch returns  $c$  in place of  $a_k$ , and an unrandomised case  $\alpha = \perp$  is used that neither  $\mathbb{F}_q$  nor the game defines.
- (d) The forking step reprograms the signature hash at a key  $PK'$  left undefined, does not show that the reprogrammed query is the challenge query of both forgeries when their randomised keys differ, and derives no bound, so Equation (3) is not derived.
- (e) Nonce extraction is unsound: the printed numerator has sign errors and uses the randomisers in place of their  $H_r$  images; and when  $c_0 = c_1$  and  $a_0 = a_1$  but  $h_0 \neq h_1$ , the two responses give one linear equation in two unknowns.

With the query counter of Definition 7.3 restored, these remain material gaps: a repair may exist, but no complete reduction is given, and the claim of Remark 7.16 is recorded, not used as a theorem. Classification (Definition 1.1): unforgeability of re-randomised FROST in the game of Definition 7.14 is an open problem.

## 7.5 Threshold spend authority

**Definition 7.18** (Threshold unforgeability under re-randomisation). Fix a key generation  $K$  for threshold parameters  $(t, n)$ , followed by Construction 6.7, and the ciphersuite FROST(Pallas, BLAKE2b-512), whose hashes are available as Assumption 4.2 models them.

1. *Corruption.* The adversary names a set  $C$  of fewer than  $t$  participants before key generation and takes their part in  $K$ : it receives every message  $K$  delivers to a member of  $C$  and, when  $K$  is interactive, sends the messages of the members of  $C$  in each round after seeing the honest participants’ messages of that round. The honest participants, and a dealer, run  $K$  as specified and abort as it prescribes. For Construction 3.2 the adversary receives the shares of  $C$  and the coefficient commitments.
2. *Keys.* After a run of  $K$  without abort it receives  $PK$  and every  $PK_j$ , and it controls the

Coordinator.

3. *Signing.* It obtains Round One commitments from honest participants (Construction 4.4) and Round Two shares on requests  $(m, L, \hat{\alpha}, D_T)$  with transaction data  $D_T$  (Definition 5.9), in which it supplies the effective randomiser  $\hat{\alpha}$  directly, as the wire form of ZIP 312 lets a Coordinator do; each honest participant runs Construction 5.10 with that  $\hat{\alpha}$  and  $D_T$ .

4. *Output.* The adversary outputs  $(\theta^*, \alpha^*, m^*, \sigma^*)$  with  $\theta^* \in \{1, -1\}$ , and wins if  $\sigma^*$  is valid on  $m^*$  under

$$\text{rk}^* := [\theta^*] PK + [\alpha^*] B$$

and fewer than  $t - |C|$  honest participants returned shares to requests whose message is  $m^*$  and whose shifted group key  $PK + [\hat{\alpha}]B$  equals  $\text{rk}^*$ .

The scheme *satisfies the notion* for  $K$  if every efficient adversary wins with negligible probability.

The notion is the target for ZIP 312, “Round Two - Signature Share Generation”; no source states it. Naming  $(\theta^*, \alpha^*)$  relates  $\text{rk}^*$  to  $PK$ , so a key whose logarithm the adversary knows is no win;  $\theta^* = -1$  admits the opposite point that an Action may witness (*Ironwood Guide*, Remark “Sign of  $\text{ak}^{\mathbb{P}}$ ”). Freshness is on the pair  $(\text{rk}, m)$ , the counterpart of the *Ironwood Guide*’s Definition “Spend-authority experiment”, whose queries  $(\alpha, m)$  fix  $\text{rk}$ .

**Definition 7.19** (Threshold spend-authority experiment). The adversary names a static set  $C$  of fewer than  $t$  participants. An Orchard-protocol threshold key is generated by  $K$ , the adversary taking the part of  $C$  as in Definition 7.18, normalised by Construction 6.7, and completed in one of two ways:

- (a) by Construction 6.8 ( $\text{use\_qsk} = \text{true}$ ), with  $\text{sk}$  drawn uniformly and independently of the key material and given to every participant, the ideal form of the agreement on  $\text{sk}$ ; or
- (b) when  $K$  is a dealer that shares an  $\text{ask}$  derived from a spending key, by the *Ironwood Guide*’s Construction “Spend-side secrets” from that spending key, which no participant holds ( $\text{use\_qsk} = \text{false}$ ).

The adversary controls the Coordinator and the participants of  $C$ , receives their state, the common secrets included, and every public field, may create notes to any address of the key, and runs signing runs of Construction 5.10 with the honest participants, supplying the message, the transaction data they check (Definition 5.9) and the effective randomiser. A *note of the key* is a note whose transmission key is  $[\text{ivk}]g_d$  for the key’s  $\text{ivk}$ . The adversary *wins* by outputting a block chain containing an accepted Action whose witness consumes a note of the key and whose signature is valid under its  $\text{rk}$  on a digest  $m$  such that fewer than  $t - |C|$  honest participants returned shares to requests with message  $m$  and shifted key  $\text{rk}$ .

The experiment is the formal content of the clause of ZIP 312, “Threat Model”, that a rogue Coordinator “should not be able to create signed transactions without the approval of”  $t$  participants (Definition 5.14). Freshness is on  $(\text{rk}, m)$  because every Action of a transaction signs one digest.

**Theorem 7.20** (Threshold spend authority from the target notion). *Suppose that re-randomised FROST over FROST(Pallas, BLAKE2b-512) with keys from  $K$  satisfies Definition 7.18. Then, under the Ironwood Guide’s Assumptions “Knowledge soundness of the Action proof”, “Discrete logarithms*

on Pallas” and “GroupHash as a random oracle”, and, for keys completed as in Definition 7.19, (b), its Assumption “Pseudorandomness of PRF expansion”, no efficient adversary wins Definition 7.19 except with negligible probability. The statement is conditional: its premise is an open problem (Remark 7.22).

*Proof.* The proof cites statements only. Let  $A$  be an efficient adversary in Definition 7.19.

(1) *Simulation.* A reduction  $\mathcal{B}$  plays Definition 7.18 with the same  $C$ , relaying the part of  $A$  in  $K$ . It completes the key. For  $\text{use\_qsk} = \text{true}$  it draws  $\text{sk}$  and computes  $\text{nk}$ ,  $\text{qsk}$ ,  $\text{qk}$ ,  $r_{\text{ivk}}$  and  $\text{ivk}$  from  $\text{sk}$  and  $\text{ak} = \text{Extract}_{\mathbb{P}}(PK)$  (Lemma 6.11, part (a)); for  $\text{use\_qsk} = \text{false}$  it simulates  $\text{nk}$  and  $r_{\text{ivk}}$  (Lemma 6.11, part (b)), at a negligible cost. It gives  $A$  the state of  $C$  and every public field, and simulates the chain and every other honest party. It relays each signing run to its oracles with the message, transaction data, commitment list and effective randomiser of  $A$ ; the honest oracles perform the check of Definition 5.9, and a failed check aborts identically in both games. The view of  $A$  is thus that of the experiment, and the number of honest shares returned for a pair  $(rk, m)$  is the same in both games.

(2) *Extraction.* On the block chain output by  $A$ ,  $\mathcal{B}$  runs the extractor of Assumption “Knowledge soundness of the Action proof” and finds an Action that meets the winning condition, stopping if there is none. Its witness gives  $\text{ak}_w^{\mathbb{P}}$ ,  $\alpha_w$  and  $\text{ivk}_w$  with  $\text{rk} = \text{ak}_w^{\mathbb{P}} + [\alpha_w]G^{\text{SpendAuth}}$  and the consumed note’s transmission key  $[\text{ivk}_w]g_d$ , where  $\text{ivk}_w = \text{Commit}_{r_{\text{ivk}_w}}^{\text{ivk}}(\text{Extract}_{\mathbb{P}}(\text{ak}_w^{\mathbb{P}}), \text{nk}_w)$  (Ironwood Guide, Definition “Orchard Action statement”).

(3) *The witnessed key.* The note is a note of the key, so  $[\text{ivk}_w]g_d = [\text{ivk}]g_d$ . The point  $g_d \neq O$  has prime order  $p_{\text{Vesta}}$ , and  $\text{ivk}_w$  and  $\text{ivk}$  are integers below  $p_{\text{Pallas}} < p_{\text{Vesta}}$ , so  $\text{ivk}_w = \text{ivk}$ . The Ironwood Guide’s Proposition “Binding of  $\text{ivk}$  to  $(\text{ak}, \text{nk})$ ” gives  $\text{Extract}_{\mathbb{P}}(\text{ak}_w^{\mathbb{P}}) = \text{Extract}_{\mathbb{P}}(PK)$ , except with negligible probability, and its Lemma “Coordinate extraction identifies opposite points” gives  $\text{ak}_w^{\mathbb{P}} = [\theta] PK$  for some  $\theta \in \{1, -1\}$ .

(4) *Output.* The Action is accepted, so its signature  $\sigma$  is valid under  $\text{rk}$  on  $m$ , and  $\text{rk} = [\theta] PK + [\alpha_w]B$ . Algorithm  $\mathcal{B}$  outputs  $(\theta, \alpha_w, m, \sigma)$ , which wins Definition 7.18 by the winning condition of the experiment and step (1).

The corrupt participants’  $\text{nk}$  and the other common secrets enter only through the simulation of Lemma 6.11, and no step uses their secrecy; there the proof departs from the single-holder case. The losses are the failure probability of the extractor, the binding term and, for  $\text{use\_qsk} = \text{false}$ , the simulation term, all negligible.  $\square$

**Corollary 7.21** (Bundle binding for threshold keys). *The Ironwood Guide’s Proposition “Bundle binding” is stated for single holders under its hypothesis (S), that the holder signs under  $\text{rsk}_i$  only  $\text{SigHash}(T)$ . Its threshold counterpart is the hypothesis*

$(S_t)$  *for each threshold-signed Action  $i$  of a transaction  $T$ , with randomised validating key  $\text{rk}_i$ , fewer than  $t - |C|$  honest participants return shares to requests whose shifted group key is  $\text{rk}_i$  and whose message is not  $\text{SigHash}(T)$ .*

*If the premise of Theorem 7.20 and  $(S_t)$  hold, then under the Ironwood Guide’s Assumption “Collision resistance of BLAKE2b” and, for keys completed as in Definition 7.19, (b), its Assumption “Pseudorandomness of PRF expansion”, an adversary in Definition 7.19 that receives  $T$  outputs, only with negligible probability, an accepted transaction that contains an Action with key  $\text{rk}_i$  and whose effecting data differ from those of  $T$ .*

*Proof.* Let  $T'$  be the output and  $m' := \text{SigHash}(T')$ . If  $m' = \text{SigHash}(T)$ , descending the digest tree of the Ironwood Guide’s Definition “Signature digest”, whose nodes hash unambiguous encodings under their personalisations, from the equal roots to a node whose inputs differ gives a collision, which has negligible probability by Assumption “Collision resistance of BLAKE2b” there. Otherwise  $m' \neq \text{SigHash}(T)$ , and by  $(S_t)$  fewer than  $t - |C|$  honest participants returned shares for the

pair  $(rk_i, m')$ ; the signature  $\sigma'$  of  $T'$  for that Action, valid under  $rk_i$  on  $m'$ , is fresh on the pair. The reduction of Theorem 7.20, which relayed the effective randomiser  $\alpha_i$  with  $rk_i = PK + [\alpha_i]B$  and simulated the common secrets by Lemma 6.11, outputs  $(1, \alpha_i, m', \sigma')$ , a win in Definition 7.18.  $\square$

Hypothesis  $(S_t)$  does not follow from Definition 5.9: a Coordinator may send a second request with the same effective randomiser together with its own matching transaction data, and the check passes. It is an approval policy that the corollary assumes. The corollary is conditional, as the theorem is.

**Remark 7.22** (Status of threshold spend authority). The premise of Theorem 7.20 is an open problem for every randomiser flow. Theorem 7.9 has no randomisation. The game of Definition 7.14 applies the randomiser hash inside its oracle, which only the flow of Construction 5.6 matches; wherever the Coordinator sends the effective randomiser, as in the wire form of ZIP 312 or when it forwards a randomiser fixed elsewhere, the flow is outside that game (Remark 5.8). The preprint’s freshness is on messages only, and its proof is incomplete (Remark 7.17). Composition with distributed key generation is open (Remark 7.13). The *Ironwood Guide*’s Theorem “Spend authority” does not apply: it fixes a key with `use_qsk = false` and the signing oracle of one holder. Unconditional threshold spend authority is an open problem (Definition 1.1); the clause of ZIP 312 is itself a “should”.

## 7.6 Unlinkability

Neither ePrint 2024/436 nor ZIP 312 proves threshold unlinkability. ZIP 312, “Rationale”, argues by matching distributions: its randomiser generation “generates randomizer uniformly at random as required by RedDSA.GenRandom”, and signing is compatible with the randomisation, signing and validation of the protocol specification. Its “Requirements” defer the formal notion to the criteria of the protocol specification for signatures with re-randomisable keys. The randomiser is close to uniform, not uniform (Lemma 5.7). This subsection states a notion against specified observers and proves it.

**Definition 7.23** (Outsider unlinkability). Fix threshold parameters  $(t, n)$ , the identifiers  $1, \dots, n$  and a key generation  $K$  with these parameters. The game generates two threshold keys independently by  $K$  and gives both group keys to an *observer*, which is neither the Coordinator nor a key-share holder and makes at most  $q_h$  random-oracle queries. The observer chooses a number  $N$  of run positions, for each position  $k$  a signing set  $I_k$  of at least  $t$  identifiers, and two assignments  $a^0, a^1$  of the run positions to the two keys. For  $b \in \{0, 1\}$ , run position  $k$  signs with signing set  $I_k$  under key  $a^b(k)$  by Construction 5.10, the observer choosing the message  $m_k$ , a signature digest, after receiving the run’s randomised validating key  $rk_k$ . Signing sets and their sizes, and messages as functions of the view, thus do not depend on the assignment. The *view* of a run is taken at one of three levels:

the *chain level*:  $rk_k$ , the aggregate and  $m_k$ ;

the *commitment level*: the chain level with the commitment list of Round One, which RFC 9591, Section 5, sends without confidentiality;

the *share level*: the commitment level with the returned signature shares, which RFC 9591 also sends without confidentiality.

The scheme is *outsider-unlinkable at a level with distance  $\epsilon$*  if the observer’s views under  $a^0$  and  $a^1$ , its oracle answers included, are within statistical distance  $\epsilon$  (*Math Guide*, §“Statistical distance”).

The Coordinator and the share holders are excluded because ZIP 312 trusts them with unlinkability (Definition 5.14).

**Proposition 7.24** (Unlinkability against outside observers). *Let signing be over FROST(Pallas, BLAKE2b-512), with distance  $\delta < 2^{-257}$  (Proposition 6.2). Assume:*

- (1) *an honest Coordinator and honest signers following Construction 5.10;*
- (2) *a randomiser fresh per run and hidden from the observer, drawn by Construction 5.6, hence within the distance of Lemma 5.7;*
- (3) *the seed and the transaction data on the confidential and authenticated channel of Definition 2.10, type (b), and Round One and the returned shares on the authenticated channels of RFC 9591.*

*Let  $Q_h := q_h + N + 2 \sum_k |I_k|$ , which bounds the number of distinct inputs at which the observer and the honest parties together query  $H_3$  and  $H_R$ . Then the scheme is outsider-unlinkable (Definition 7.23) at the chain and commitment levels with distance at most*

$$2 \sum_{k=1}^N (1 + 2|I_k|)(\delta + Q_h \cdot 2^{-256}).$$

*The hypotheses that the observer is neither the Coordinator nor a signer of the runs, and that the randomiser channel is confidential, are necessary.*

*Proof.* Sufficiency is proved by hybrids (i) to (iii), from each assignment towards one common game. The argument is statistical and programs no oracle, so it has no abort term.

(i) Each randomiser  $\hat{a}_k$  is replaced by a uniform scalar, one run at a time, at  $\delta + Q_h \cdot 2^{-256}$  per run, by Lemma 5.7 applied with the rest of the game as the observer; its queries to  $H_R$  are among the  $Q_h$ , each other honest evaluation having its own fresh 32-byte prefix. Then every  $rk_k = PK + [\hat{a}_k]B$  is uniform and independent of the group keys and of the other runs (Ironwood Guide, Proposition “Unlinkability of randomised validating keys”, part (i)).

(ii) Each honest nonce is replaced by a uniform scalar, at  $\delta + Q_h \cdot 2^{-256}$  per nonce, by Lemma 4.5 applied in the same way, whatever the share. The commitments are then uniform and independent of the keys, and so is the event of a zero nonce and the abort it causes.

(iii) The rest of the view is a function of these values, the observer’s choices and the oracle. For run  $k$ :  $\rho_i$  is  $H_1$  at an input holding  $\text{ser}_{\mathbb{G}}(rk_k)$ ,  $H_4(m_k)$ ,  $H_5$  of the list encoding and  $i$ ;  $R = \sum_{i \in I_k} (D_i + [\rho_i]E_i)$ ;  $c = H_2(\text{ser}_{\mathbb{G}}(R) \parallel \text{ser}_{\mathbb{G}}(rk_k) \parallel m_k)$ ; and  $z$  is the unique scalar with  $[z]B = R + [c]rk_k$  (Theorem 5.11). The binding-factor input holds  $rk_k$  and not  $PK$  because Construction 5.10 substitutes the shifted key; with  $PK$  there, the check that  $R$  is the sum of the  $D_i + [\rho_i]E_i$  under a candidate  $PK$  would test whether a run belongs to that key. After (i) and (ii) the view has one distribution under  $a^0$  and under  $a^1$ . The triangle inequality (Math Guide, §“Statistical distance”, Theorem “Properties of statistical distance”) over both chains of hybrids, each of  $N$  randomiser hops and  $2 \sum_k |I_k|$  nonce hops, gives the bound.

(iv) *Necessity.* The Coordinator and every signer of a run hold its seed with  $L$ , hence its randomiser, and so does any party that reads a randomiser channel without confidentiality. Each computes  $ak^{\mathbb{P}} = PK = rk - [\hat{a}]G^{\text{SpendAuth}}$  and so links the run to every run of the key, distinguishing two assignments that differ on it, except when the two independent group keys coincide. A randomiser reused across two runs of one key gives both the same shifted key (ePrint 2024/436, Section 7). Hence the exclusions and the confidential randomiser channel are necessary; this part is the volume’s one statement of why that channel is confidential.  $\square$

The proposition is scoped against the Ironwood Guide’s Definition “Public leakage and privacy preconditions” (“Privacy”). A key with `use_qsk = true` violates (P1), which requires `use_qsk =`

false; every randomiser of ZIP 312 violates (P2), which requires  $\alpha$  drawn by GenRandom; and (P4) excludes an adversary holding  $nk$ , which every participant of Construction 6.8 holds. The *Ironwood Guide*'s Theorem "Privacy of an Action" and the key-dependent clauses of its Theorem "End-to-end security" therefore cover no threshold-signed Action, and for the other fields of such an Action this volume claims nothing.

**Proposition 7.25** (Linkage through returned signature shares). *Let  $t \geq 2$ . At the share level of Definition 7.23 every run of a key is linked. From a share  $z_i$  of run  $k$ , the commitment list, the public  $rk_k$  and  $m_k$ , the observer computes  $\rho_i$ ,  $R$ ,  $c$  and  $\lambda_{I_k,i}$ , and then the tag*

$$\left[ \frac{1}{\lambda_{I_k,i} c} \right] ([z_i]B - D_i - [\rho_i]E_i) - rk_k = PK_i - PK,$$

*which is the same for every run of the key; the division is defined except when  $c = 0$ , which has probability at most  $\delta + 1/q$ . From the at least  $t$  tags of one run, interpolation in the exponent gives  $PK_j - PK$  for every identifier  $j$ , so runs with disjoint signing sets are linked too. For two keys generated independently by Construction 3.2, the tags at one identifier agree with probability  $1/q$ . The chain and commitment levels are therefore the strongest that the channels of RFC 9591 and ZIP 312 support: unlinkability against an observer of the network needs confidential channels for the returned shares, which neither requires. For  $t = 1$  the tag is  $O$  and carries no key.*

*Proof.* An honest share is  $z_i = d_i + e_i \rho_i + \lambda_{I,i} c (sk_i + \hat{\alpha})$  (Construction 5.10), so

$$[z_i]B - D_i - [\rho_i]E_i = [\lambda_{I,i} c] (PK_i + [\hat{\alpha}]B);$$

dividing by  $\lambda_{I,i} c$  and subtracting  $rk = PK + [\hat{\alpha}]B$  gives  $PK_i - PK$ . Here  $\lambda_{I,i} \neq 0$  for distinct non-zero identifiers, and  $c$  is an answer of  $H_2$ , which takes the value 0 with probability at most  $1/q + \delta$  (*Math Guide*, Theorem "Properties of statistical distance", part (3)). With  $f$  the sharing polynomial, the tags are the values  $[g(j)]B$  of  $g := f - f(0)$ , of degree at most  $t - 1$ ;  $t$  of them determine  $[g(j)]B$  for every  $j$  by Lagrange interpolation in the exponent (*Math Guide*, Theorem "Lagrange interpolation"), the map  $a \mapsto [a]B$  being linear. For a dealer key with  $t \geq 2$ ,  $g(j) = \sum_{k \geq 1} a_k j^k$  is uniform for  $j \neq 0$ , since  $a_1$  is uniform and independent of the other coefficients; so the tags of two independent keys at  $j$  agree with probability  $1/q$ . For  $t = 1$ ,  $f$  is constant and  $g = 0$ .  $\square$

On the toy curve of Example 4.12, two re-randomised runs of the key with randomisers 565 and 908 have randomised keys (268,139) and (436,331), and both give the tags  $PK_1 - PK = (433,783)$  and  $PK_3 - PK = (933,275)$ .

## 7.7 Status of the constructions

Table 3 classifies the principal constructions and claims of the volume by Definition 1.1, with its source and the section that states it.

| Object and source                                                                                                                                                                                                                            | Home                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| <i>Specified</i>                                                                                                                                                                                                                             |                                   |
| FROST signing, aggregation and share verification; RFC 9591                                                                                                                                                                                  | §4                                |
| Trusted-dealer key generation; RFC 9591, Appendix C                                                                                                                                                                                          | §3.1                              |
| Even- $y$ requirement on the spend validating key of an Orchard-protocol key; ZIP 2005 (Proposed)                                                                                                                                            | §6.3                              |
| Re-randomised signing, aggregation and share verification; FROST(Pallas, BLAKE2b-512); the threat model; ZIP 312 (Draft), which leaves key generation out of scope                                                                           | §§5.5, 6.1, 5.6                   |
| Randomiser derivation of ZIP 312 (Draft); with the signature digest as message it yields a valid threshold spend only with negligible probability (Proposition 5.5), so no randomiser derivation usable for spend authorisation is specified | §5.3                              |
| Derivation with <code>use_qsk = true</code> and the constraints on FROST keys; ZIP 2005 (Proposed), taking effect with NU6.3 (ZIP 258)                                                                                                       | §6.4                              |
| Message-check obligation; ZIP 312                                                                                                                                                                                                            | §5.4                              |
| Consensus rule under which the aggregate is accepted; protocol specification, §“Action Descriptions”                                                                                                                                         | §6.2                              |
| <i>Designed but unspecified</i>                                                                                                                                                                                                              |                                   |
| Distributed key generation for Zcash; ePrint 2020/852, specified by no RFC or ZIP                                                                                                                                                            | §3.2                              |
| Agreement on the common spending key                                                                                                                                                                                                         | §6.4                              |
| Sign normalisation of shared key material (Construction 6.7), this volume’s own                                                                                                                                                              | §6.3                              |
| Seed-and-commitments randomiser and the order under it                                                                                                                                                                                       | §§5.3, 6.5                        |
| Message-check mechanism                                                                                                                                                                                                                      | §5.4                              |
| Threshold spend authorisation of an Action under the seed-and-commitments randomiser                                                                                                                                                         | §6.6                              |
| <i>Open problem</i>                                                                                                                                                                                                                          |                                   |
| Unforgeability of FROST with distributed key generation                                                                                                                                                                                      | §7.3                              |
| Unforgeability of re-randomised FROST in the game of ePrint 2024/436, claimed with a gapped proof; and in the target notion, Definition 7.18                                                                                                 | §§7.4, 7.5                        |
| Threshold strong unforgeability under re-randomised keys, Definition 5.1                                                                                                                                                                     | §5.6                              |
| Unconditional threshold spend authority                                                                                                                                                                                                      | §7.5                              |
| Composition of a constructor-chosen $\alpha$ with ZIP 312                                                                                                                                                                                    | §6.5                              |
| <i>Established</i>                                                                                                                                                                                                                           |                                   |
| TS-SUF-3 of FROST1 under OMDL in the random-oracle model; ePrint 2022/833, Theorem 5.4                                                                                                                                                       | Theorem 7.9                       |
| <i>Proved in this volume</i>                                                                                                                                                                                                                 |                                   |
| Correctness                                                                                                                                                                                                                                  | Theorems 4.11, 5.11               |
| Validity of the aggregate                                                                                                                                                                                                                    | Proposition 6.3, Corollary 6.4    |
| Correspondence to FROST1                                                                                                                                                                                                                     | Proposition 7.8                   |
| OMDL hardness implies AOMDL hardness                                                                                                                                                                                                         | Lemma 7.4                         |
| No randomiser derived from the signature digest                                                                                                                                                                                              | Proposition 5.5                   |
| Distance of nonces and randomisers from uniform                                                                                                                                                                                              | Lemmas 4.5, 5.7                   |
| Replay and identifiable abort                                                                                                                                                                                                                | Propositions 4.14, 4.10           |
| Validity of dealt and jointly generated key material                                                                                                                                                                                         | Propositions 3.3, 3.8             |
| Correctness of sign normalisation of a dealt or jointly generated sharing                                                                                                                                                                    | Lemma 6.6, Construction 6.7       |
| Transfer of unforgeability to Orchard-protocol dealer keys                                                                                                                                                                                   | Lemmas 7.11, 6.11, Corollary 7.12 |
| Order of a threshold spend authorisation                                                                                                                                                                                                     | Proposition 6.12                  |
| Nonce-reuse criterion                                                                                                                                                                                                                        | Lemma 4.13                        |
| Threshold spend authority and bundle binding from the target notion                                                                                                                                                                          | Theorem 7.20, Corollary 7.21      |
| Outsider unlinkability at the chain and commitment levels                                                                                                                                                                                    | Proposition 7.24                  |
| Linkage at the share level                                                                                                                                                                                                                   | Proposition 7.25                  |

Table 3: Classification of the principal constructions and claims of the volume (Definition 1.1), one row per object, with its source and the section or statement that gives it. Specified objects are those of an RFC, a ZIP or the protocol specification, with the ZIP’s status; designed objects are published or constructed here without a specification; open problems have no proof in any checked source; established claims rest on a published proof; the last group lists the principal statements proved in this volume.